



人工神经网络：仿生联结智慧启

Artificial Neural Network: Intelligence with Brain-inspired Neuron



授课对象：计算机科学与技术专业 二年级

课程名称：人工智能（专业必修）

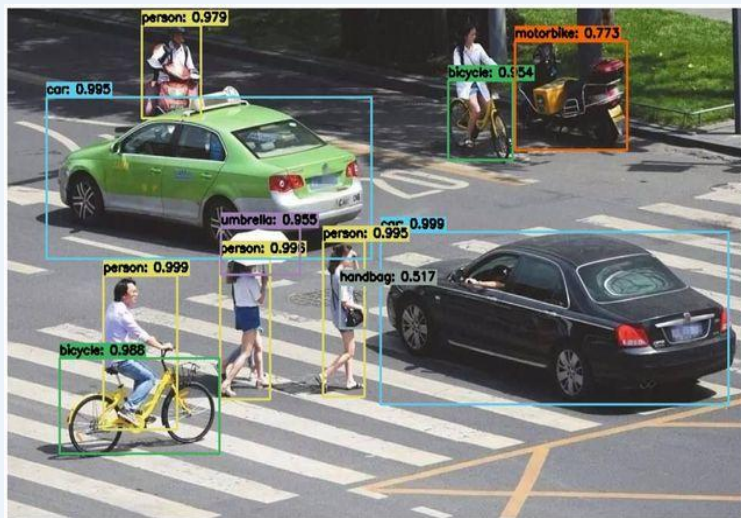
节选内容：第5章 机器学习

课程学分：3学分





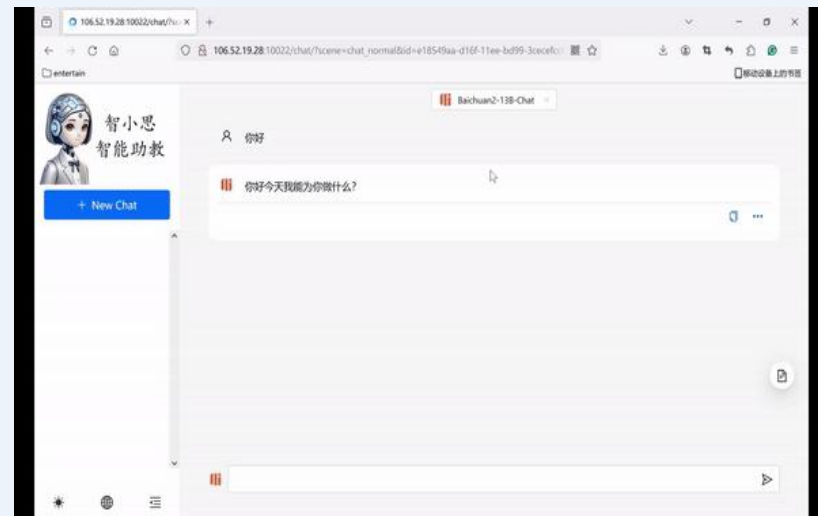
前情知识回顾：机器学习概述



计算机视觉
(Computer Vision)



语音识别
(Speech Recognition)



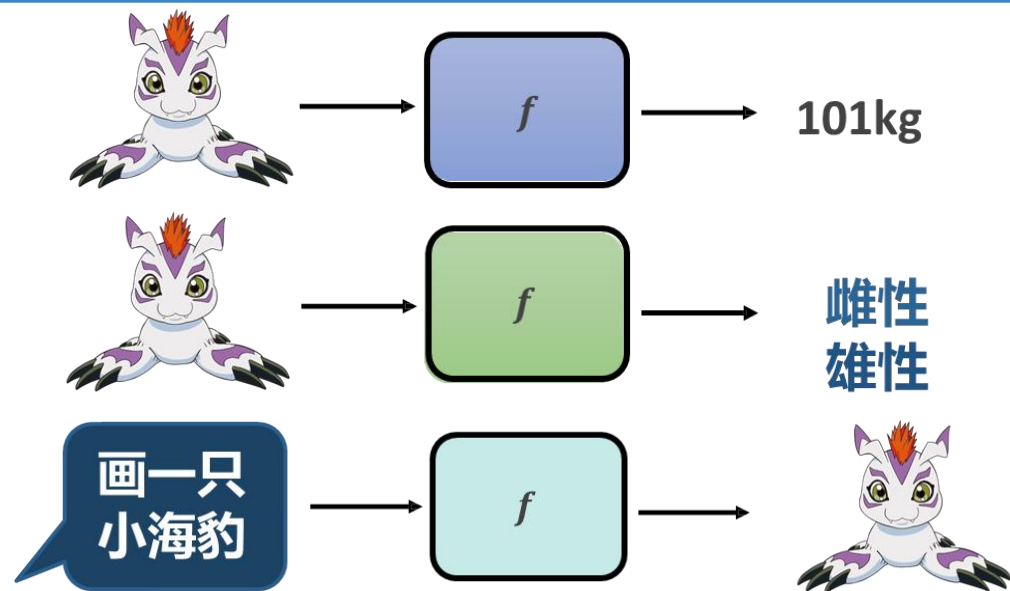
自然语言处理
(Natural Language Processing)
课程人工智能助教“智小思”



前情知识回顾：机器学习概述

≈ 机器自动找一个函数

- 回归 (Regression): 函数的输出是一个数值
- 分类 (Classification): 函数的输出是一个类别
- 生成 (Generation): 函数的输出是一个数据



机器学习的核心目标是**通过数据学习一个函数 $f(x)$** ，完成从输入到输出的映射

关键：如何设计**函数结构**和**优化策略**来更好地完成这个映射

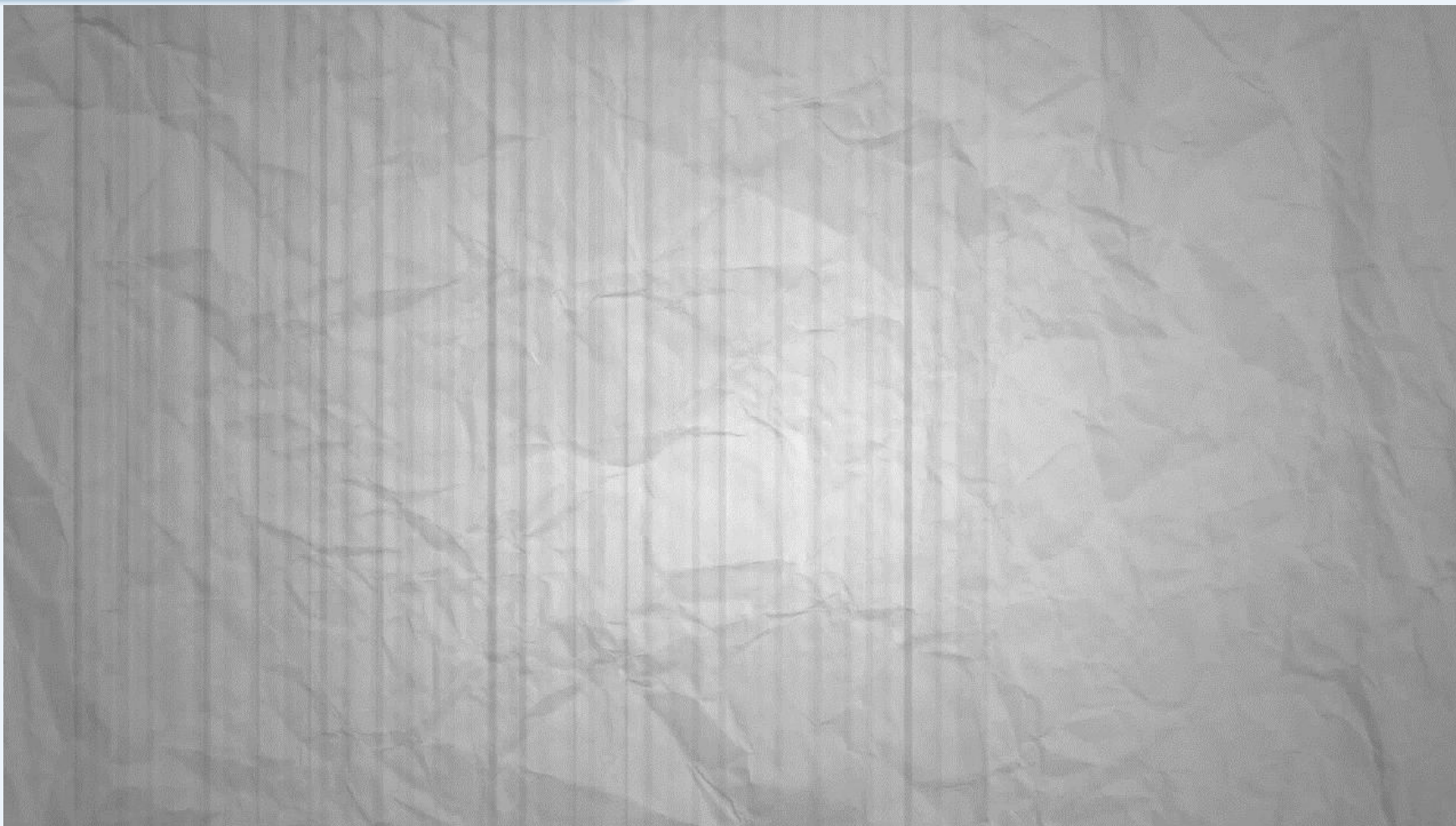


人工神经网络的发展历史

- Origins: Algorithms that try to **mimic the brain**
- Widely used in 80s and early 90s; popularity diminished in late 90s
- Resurgence: **State-of-the-art technique** for many applications in recent years

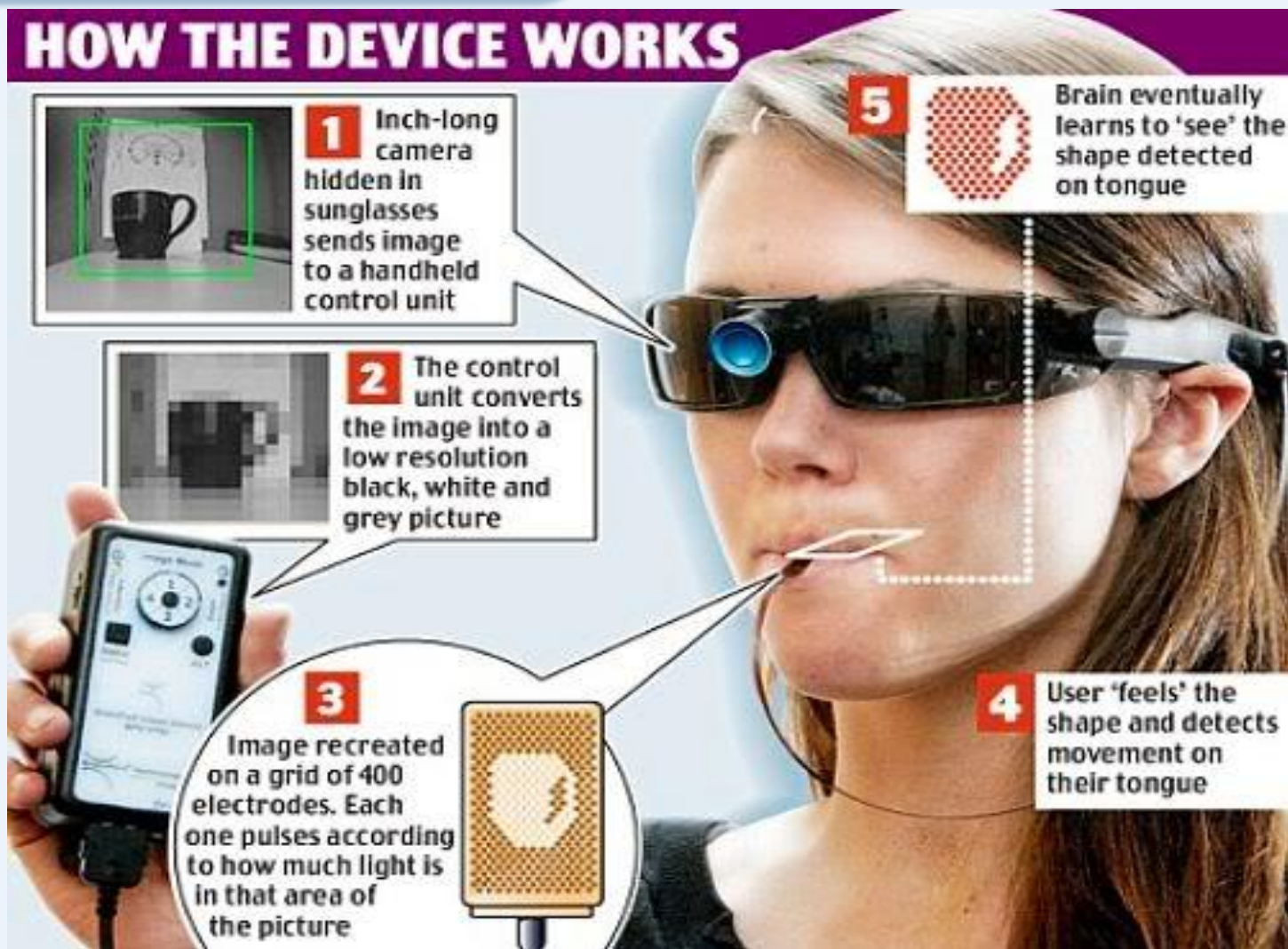


Brain vs ANN





人类如何用脑





神经元

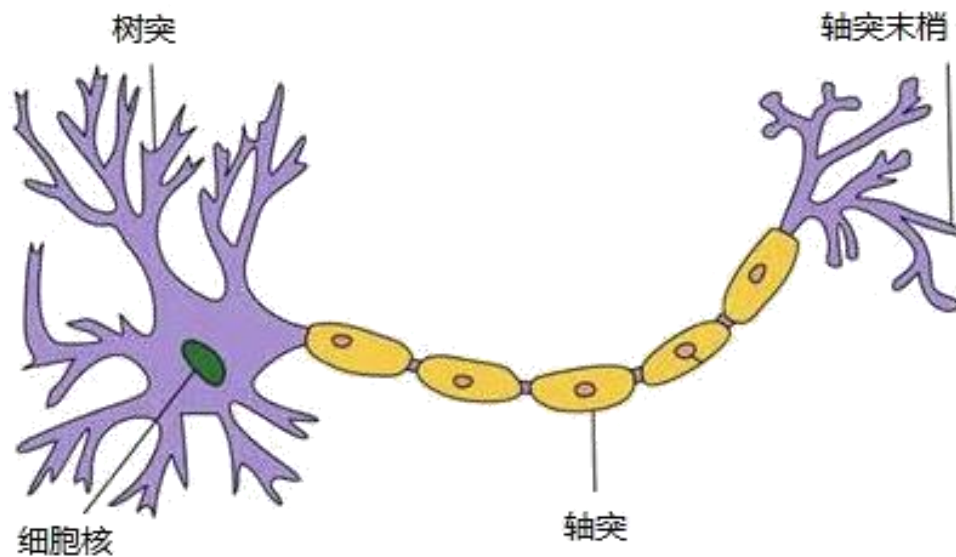
- 人脑中的神经网络是一个非常复杂的组织，由很多具有适应性的**简单同构单元**组成的广泛并行互连的网络，它的组织能够模拟生物神经系统对真实世界所作出的交互反应
- 这一简单的结构就是**神经元**
- 成人的大脑中估计有 1000 亿个神经元



神经元

- 1904年生物学家已经知晓**神经元**的组成结构
- 一个神经元通常具有多个**树突**，主要用来接受传入信息；而**轴突**只有一条，轴突尾端有许多**轴突末梢**，跟其他神经元的树突产生连接（**突触**），从而给其他多个神经元传递信号

大脑运作方式的启发



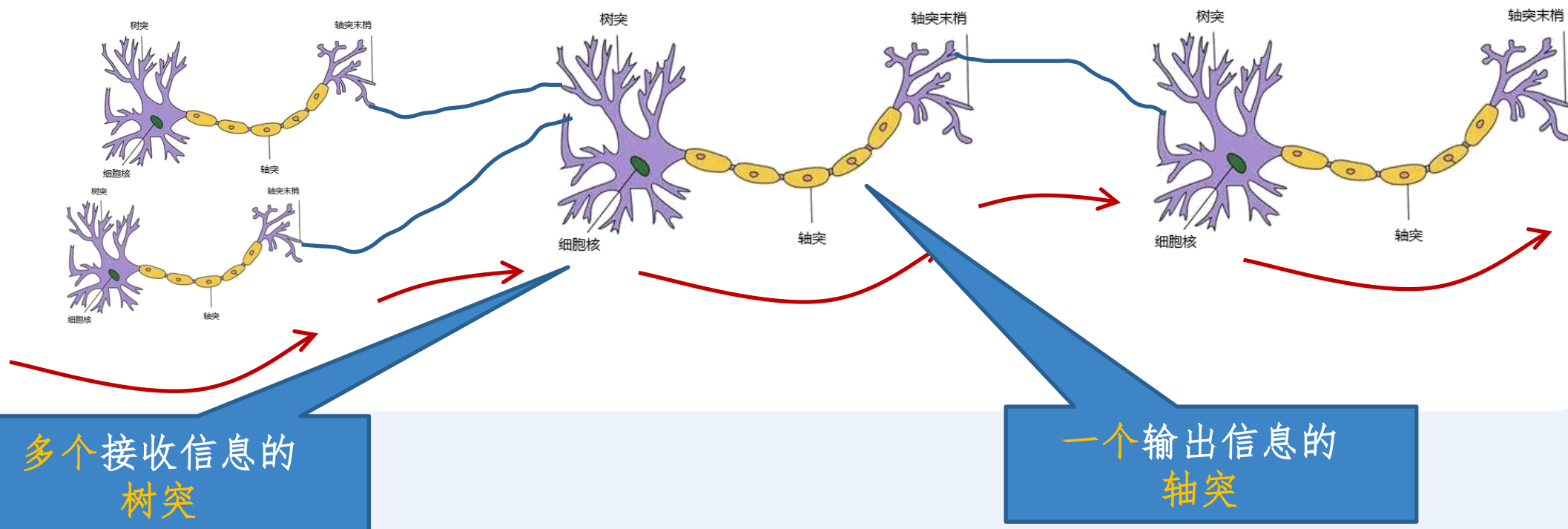
神经元结构图



人脑神经元的信息传递

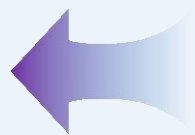
- 当一个神经元接收的信号**累积到一定程度**时，它会释放神经递质，该神经递质会经过**树突前端**前往**轴突末梢**，依靠突触刺激下一个神经元，实现多个神经元间**信息传递**

信息的传递





人脑神经元模型MP



人脑神经元的信息传递

- 1943年，心理学家McCulloch和数学家Pitts参考了生物神经元的结构，发表了抽象的**神经元模型MP**
- 类比生物结构：输入可以类比为神经元的树突，而输出可以类比为神经元的轴突，计算则可以类比为细胞核。**神经元模型是一个包含输入，输出与计算功能的模型**

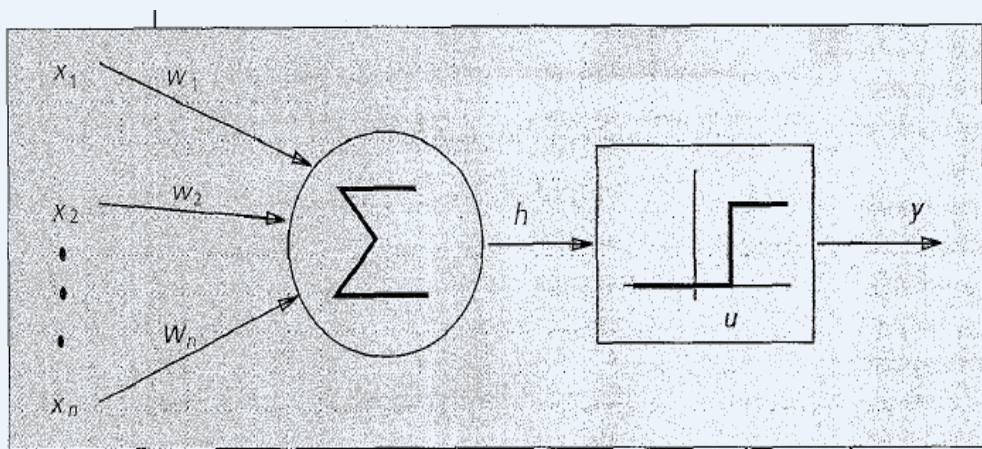
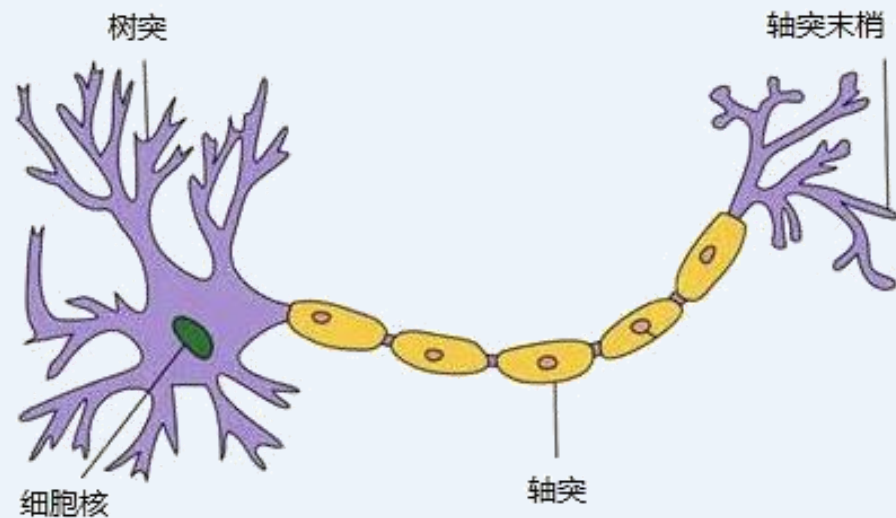
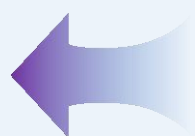
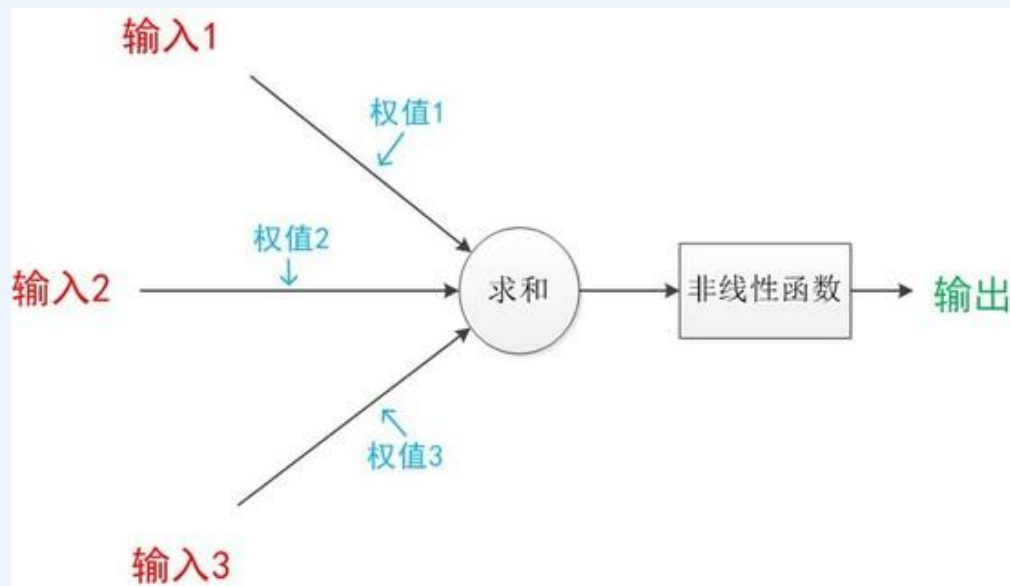


Figure 2. McCulloch-Pitts model of a neuron.

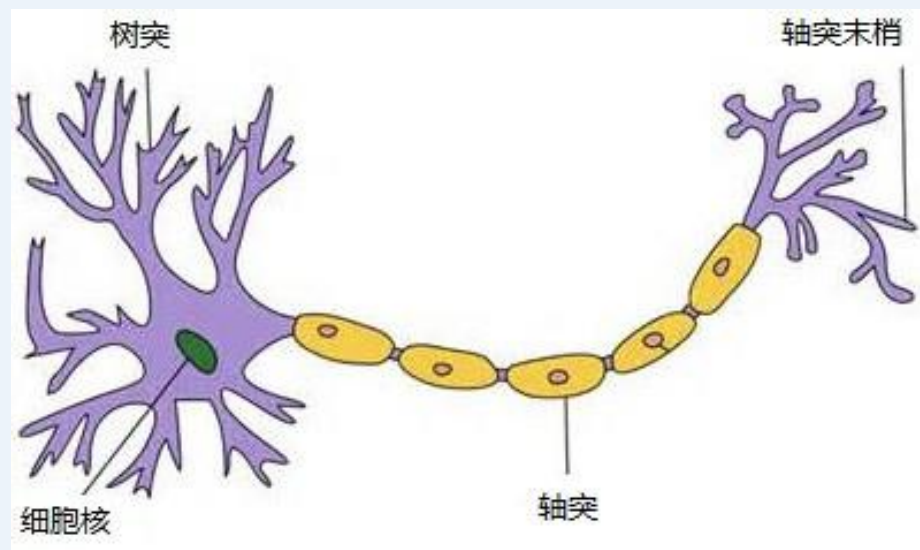


人工神经元模型MP

- 类比生物结构：输入可以类比为神经元的树突，而输出可以类比为神经元的轴突，计算则可以类比为细胞核。**神经元模型是一个包含输入，输出与计算功能的模型。**
- 下图是一个典型的神经元模型：包含有3个输入，1个输出，以及2个计算功能
- 注意中间的箭头线。这些线称为“连接”。每个上有一个“权值”

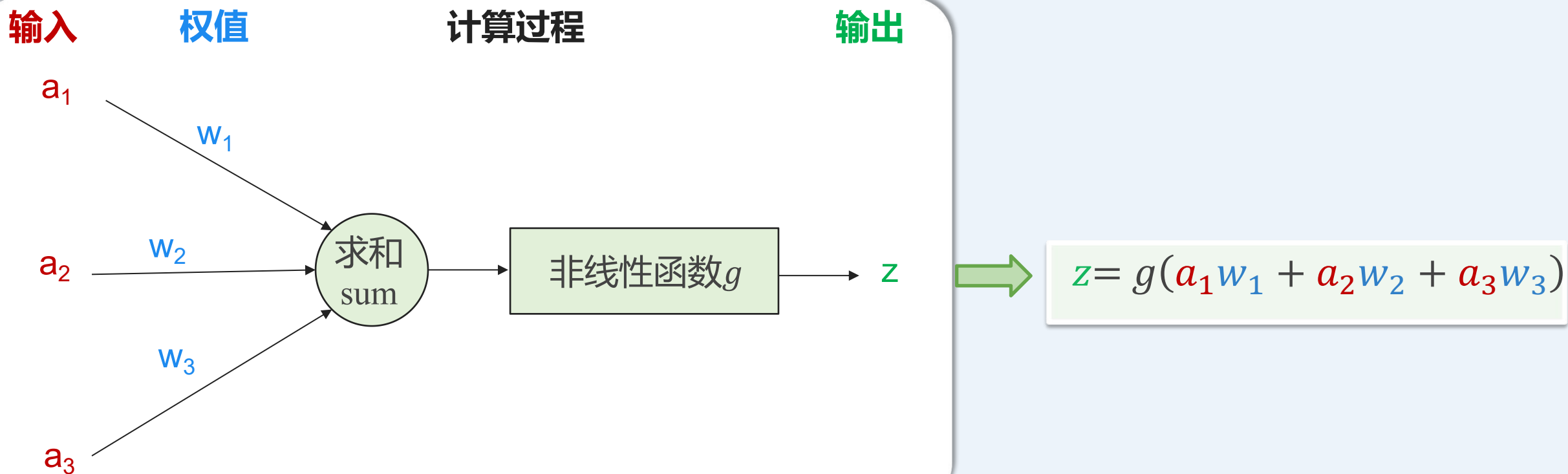


神经元模型



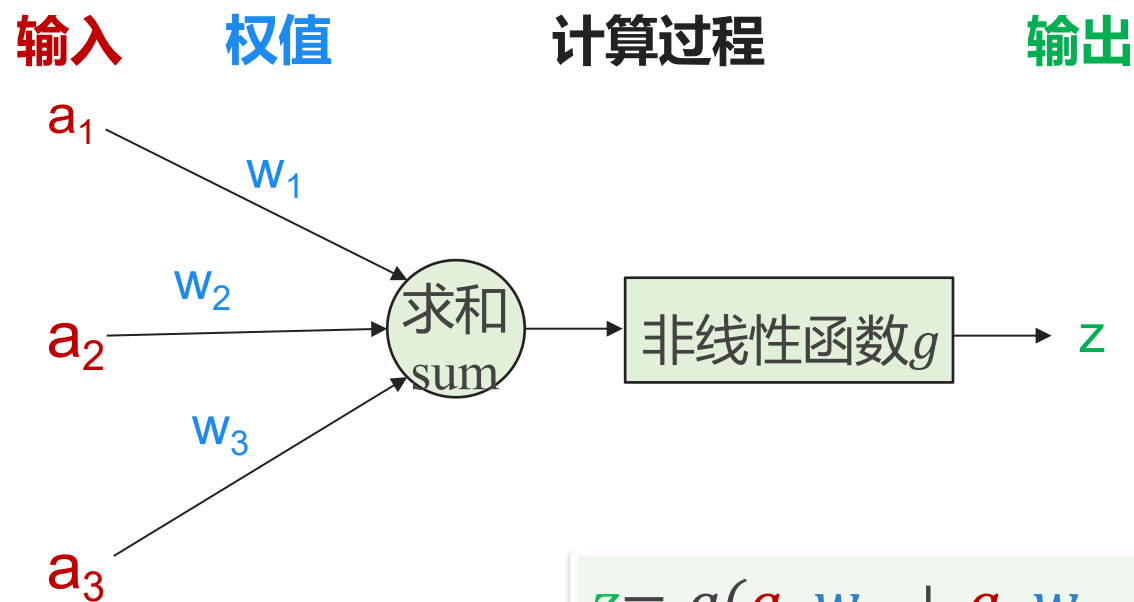
人工神经元模型MP

- 一个典型的神经元模型，包含有N个输入，1个输出，以及2个计算功能，每个连接上有一个权值

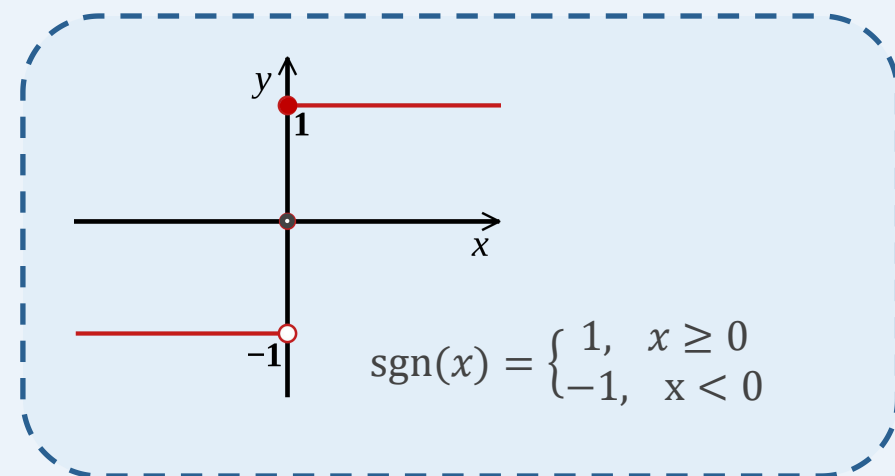


人工神经元模型MP

- 神经元的计算公式：用 $a = [a_1, a_2, a_3]^T$ 表示输入，用 $W = [w_1, w_2, w_3]$ 表示权值，经过加权计算末端信号为 $W \times a$ ，再叠加**非线性函数g**，即是输出值z
- 在MP模型里，**非线性函数g**设置是**符号函数（sgn）**
- 由于起到类似神经元的“激活”作用，也称作**激活函数**（Active Function）

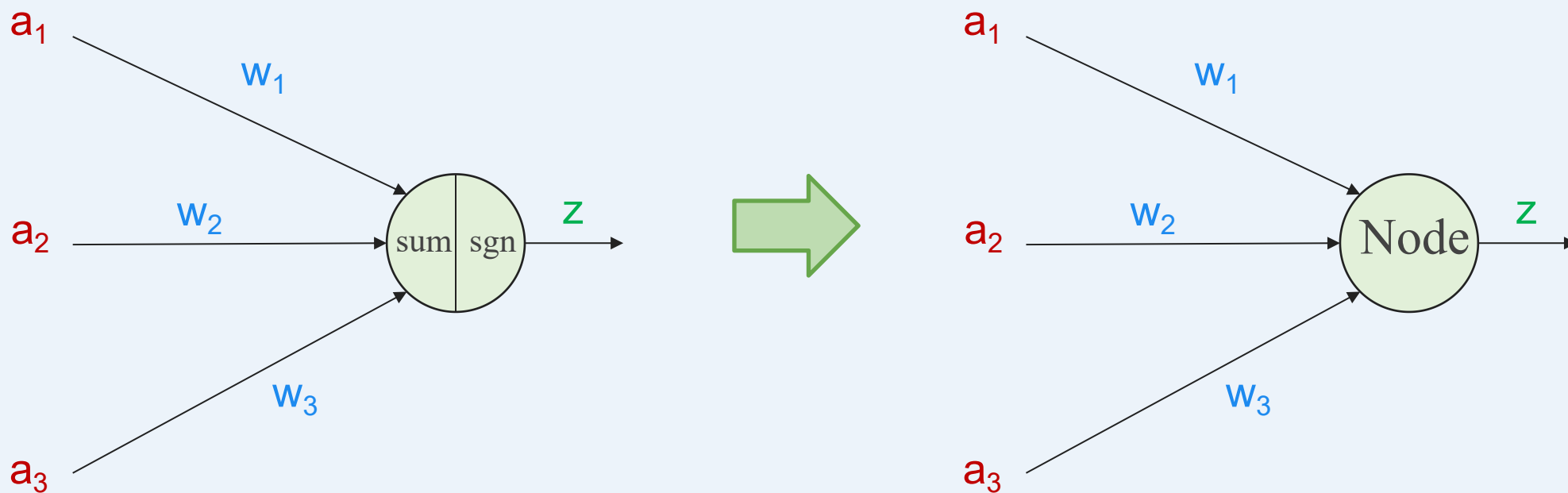


$$z = g(a_1 w_1 + a_2 w_2 + a_3 w_3) = g(W \times a)$$



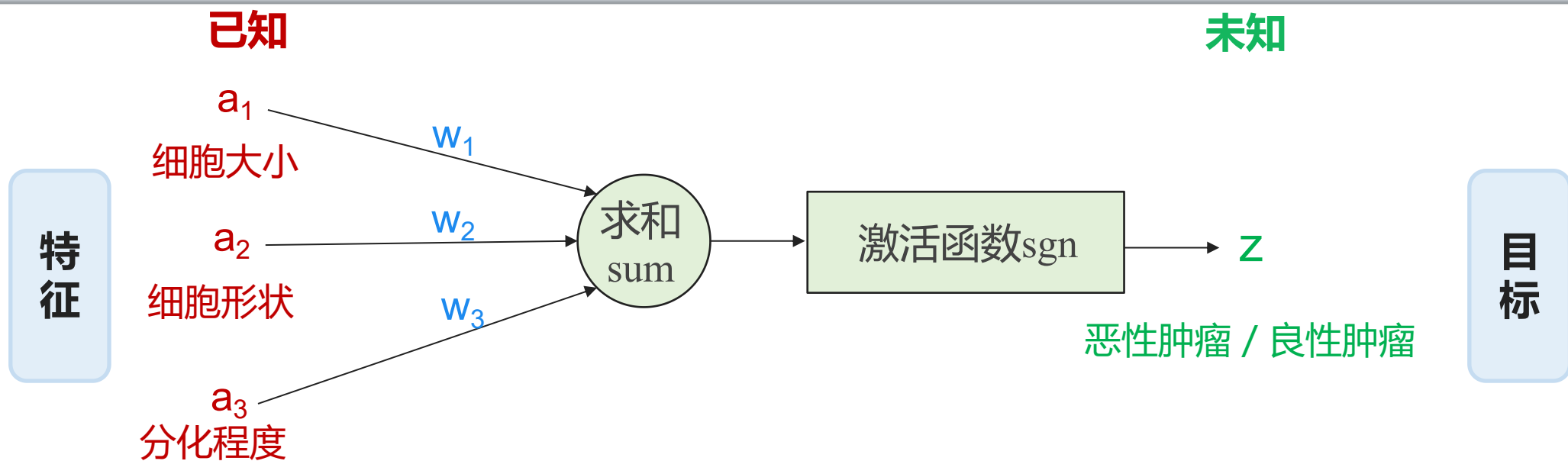
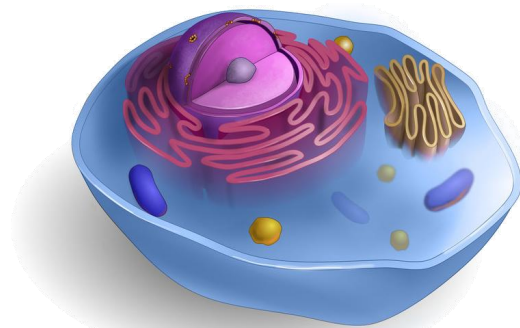
人工神经元模型MP

- 对神经元模型的图进行简化：将求和函数sum与激活函数sgn合并到一个圆圈里，代表神经元的内部计算。此时，一个“节点”就是一个神经元



MP神经元的理解

- 神经元模型可以这样理解：
 - 根据3个**已知属性**预测**未知属性**
 - 神经元模型中：3个已知属性的值是 a_1 , a_2 , a_3 , 未知属性的值是 z
 - 已知的属性称为**特征**, 未知属性称为**目标**



MP神经元的缺陷

- 1943年发布的MP模型，虽然简单，但已经建立了神经网络大厦的地基。然而，MP模型中，**权重的值都是预先设置的**，因此不能进行学习

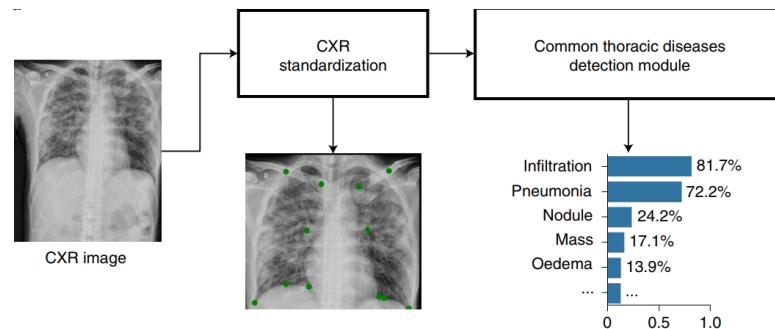


Donald Olding Hebb

- 1949年心理学家Hebb发现人脑神经细胞的**突触上的强度是可以变化的**，提出了**Hebb学习率**。这启发计算科学家们开始考虑用调整权值的方法来让**机器自我学习**，为后面的学习算法奠定了基础
- 基于这个启发，才**诞生了感知机（Perceptron）**

启发：学科交叉

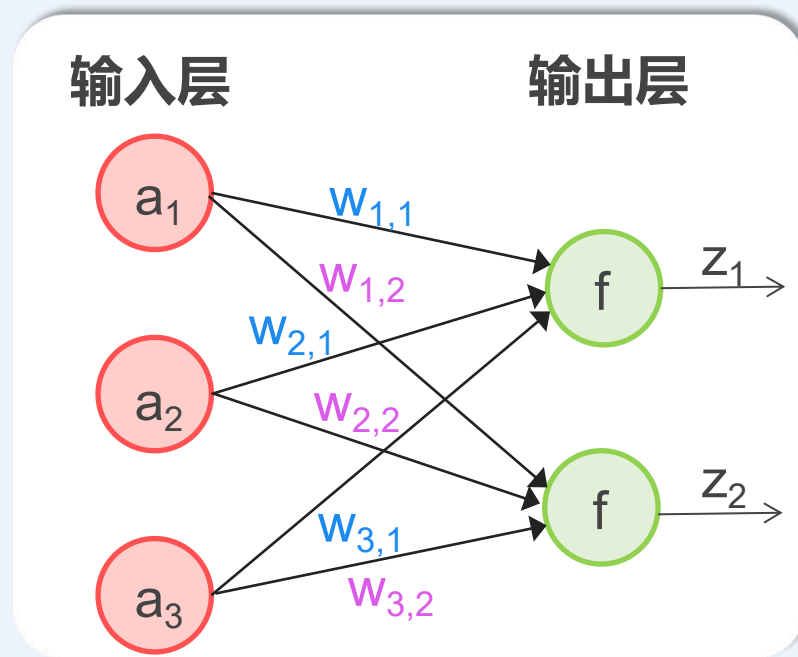
- 各个学科之间有相通之处，互相启发
- 人工智能亦助力其它医学、生物学等多学科发展



广东省某医院：基于胸片的AI诊断系统

感知机模型

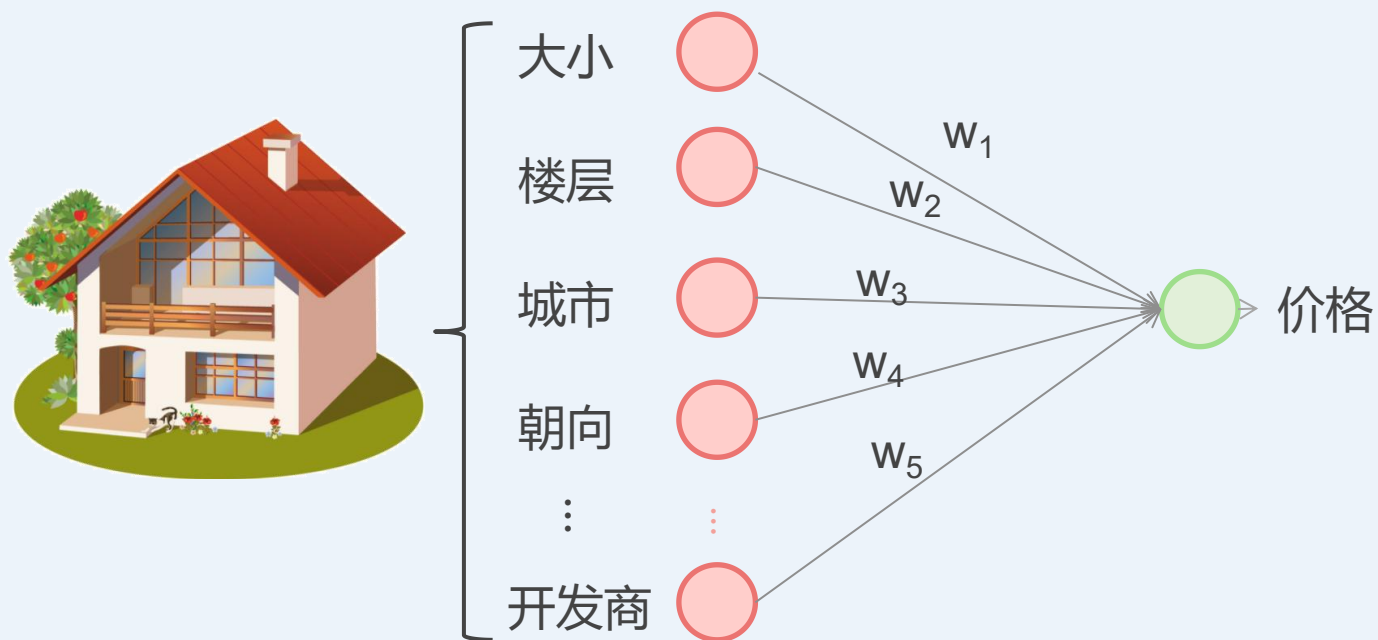
- **感知机 (Perceptron)**：由两层神经元构成
 - **输入层**：在MP模型的输入位置添加神经元节点，只**传输数据**，不做计算
 - **输出层**：对前面一层的输入**进行计算**



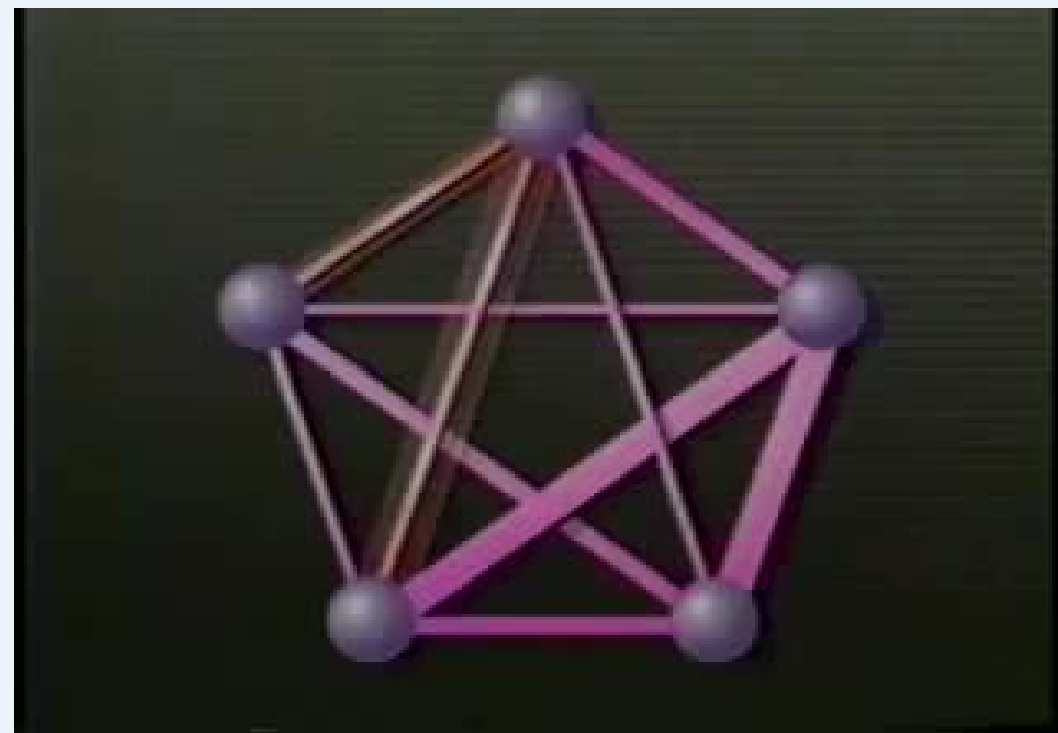
感知机拥有一个计算层，也称为“单层神经网络”

感知机模型

- 感知机可用于**回归问题**和**分类问题**



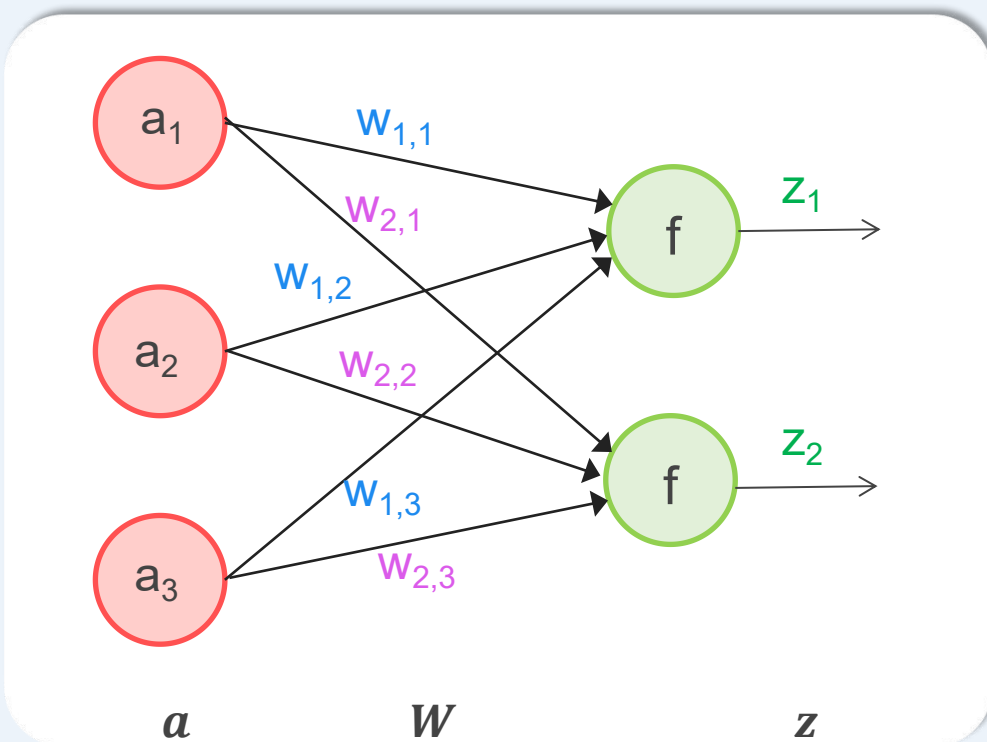
感知机的房价回归预测



感知机的早期应用：性别分类

感知机模型数学化公式

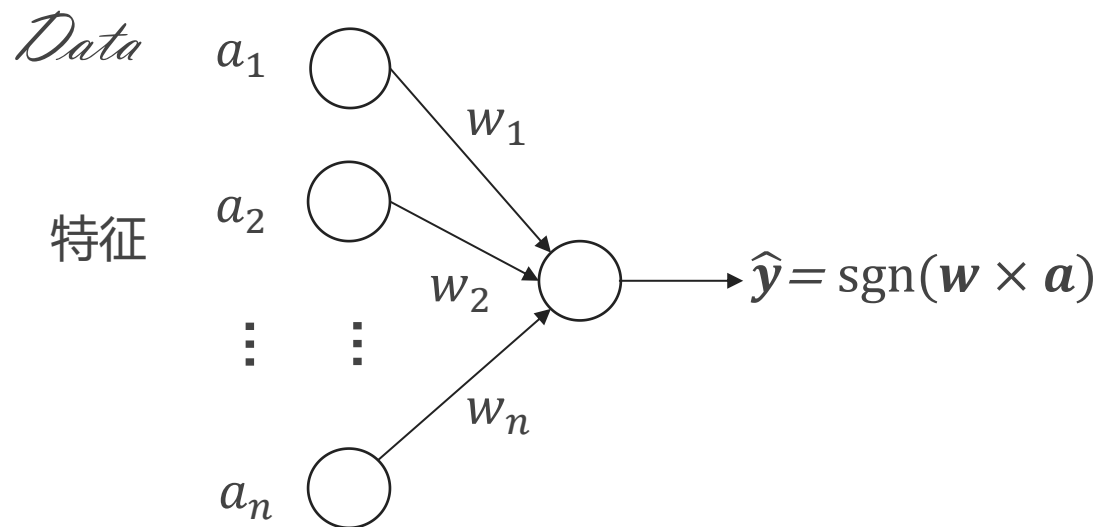
- 输入向量 $a = [a_1, a_2, a_3]^T$ ，输出向量 $z = [z_1, z_2]^T$ ，系数矩阵 $W \in \mathbb{R}^{2 \times 3}$
- 则用矩阵乘法表达感知机的计算公式为： $z = g(W \times a)$
- 该公式也是神经网络中从前一层到后一层的运算过程



$$z_1 = g(a_1 w_{1,1} + a_2 w_{1,2} + a_3 w_{1,3})$$
$$z_2 = g(a_1 w_{2,1} + a_2 w_{2,2} + a_3 w_{2,3})$$

感知机模型数学化公式

- 与神经元MP模型不同，感知机中的权重是通过**训练**得到的



$$\text{sgn}(z) = \begin{cases} 1, & \text{if } z \geq 0 \\ -1, & \text{otherwise} \end{cases}$$

$$\min_{\mathbf{w}} |\mathbf{y} - \hat{\mathbf{y}}|$$

感知机训练法则

$$W_t \leftarrow W_t + \Delta W_t$$

$$\Delta W_t = \eta(y - \hat{y})\mathbf{a}$$

y : 是真实的目标

$\hat{y}$: 是感知机的输出

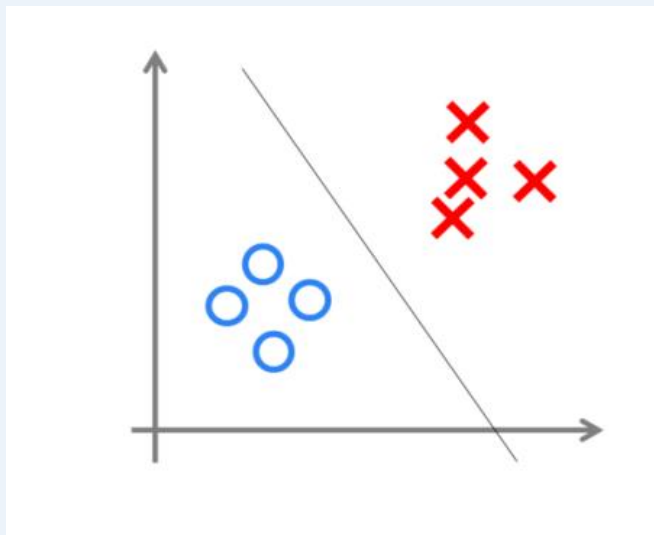
η : 学习速率 (如 0.1)

$\mathbf{a}$: 训练数据

感知机潜藏的缺陷

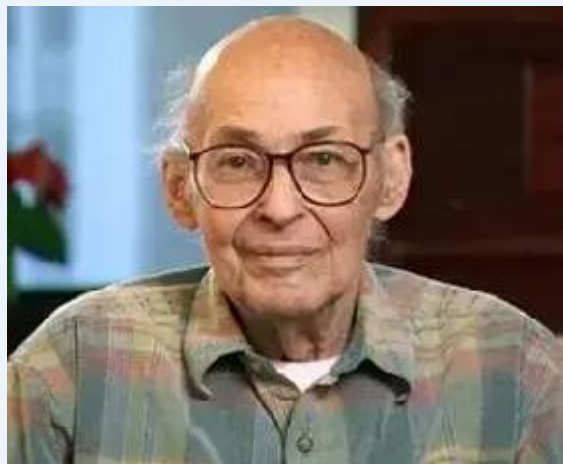
$$w \cdot x = 0$$

- 根据以前的知识我们知道，感知机类似一个 **逻辑回归** 模型，可以做线性分类任务。可以把感知机当做n维空间中的决策超平面，对于超平面一侧的实例感知机输出 **1**，对于另一侧的实例输出 **0**
- 我们可以用 **决策分界** 来形象的表达分类的效果。决策分界就是在二维的数据平面中划出一条直线，当数据的维度是3维的时候，就是划出一个平面，当数据的维度是n维时，就是划出一个n-1维的超平面。下图显示了在二维平面中划出决策分界的效果，也就是感知机的分类效果。

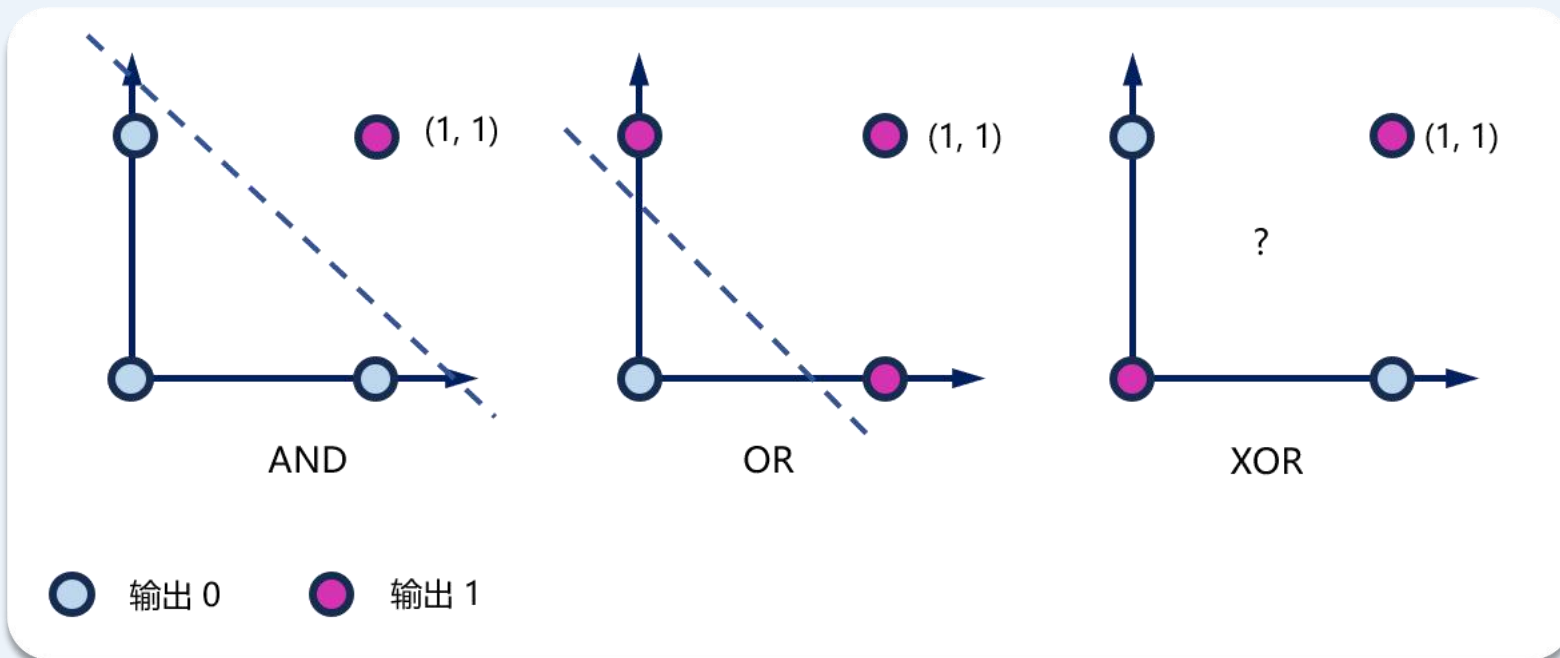


感知机的缺陷

- Minsky在1969年出版了一本叫《Perceptron》的书，里面用详细的数学证明了感知器的弱点，尤其是单层感知器对**XOR（异或）**这样的简单分类任务都无法解决



Minsky

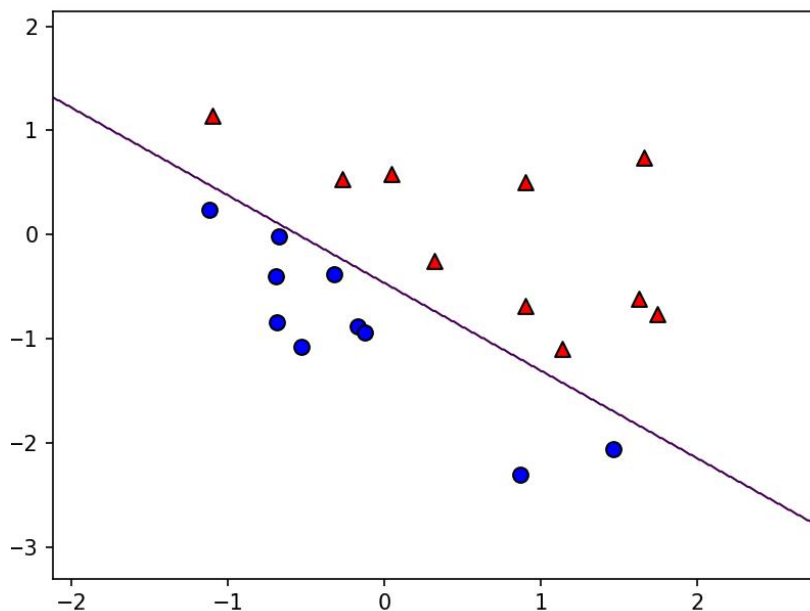


- 代码参考: <https://pastebin.ubuntu.com/p/8dw4zyXfY8/>

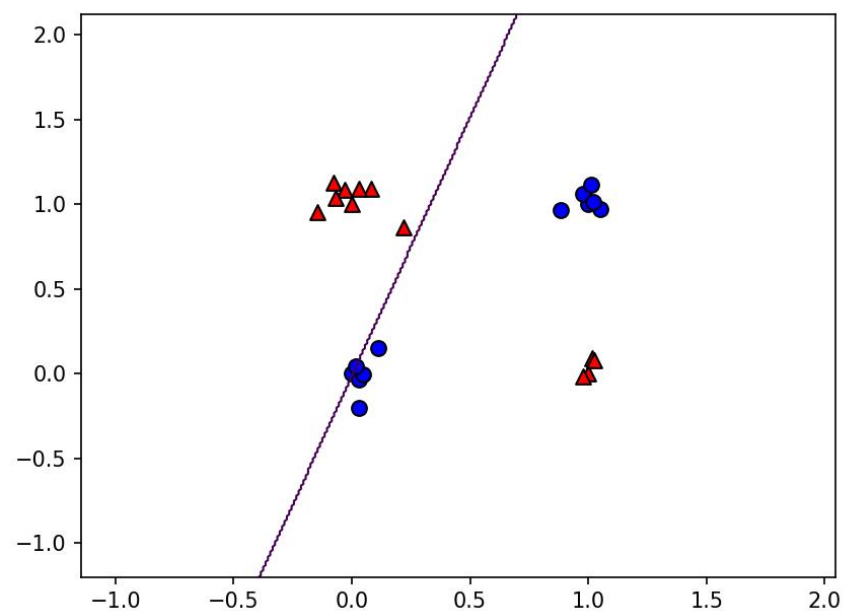
感知机的缺陷

- Minsky在1969年出版了一本叫《Perceptron》的书，里面用详细的数学证明了感知器的弱点，尤其是单层感知器对**XOR（异或）**这样的简单分类任务都无法解决

线性分类任务



XOR任务



- 代码参考: <https://pastebin.ubuntu.com/p/8dw4zyXfY8/>

感知机的缺陷

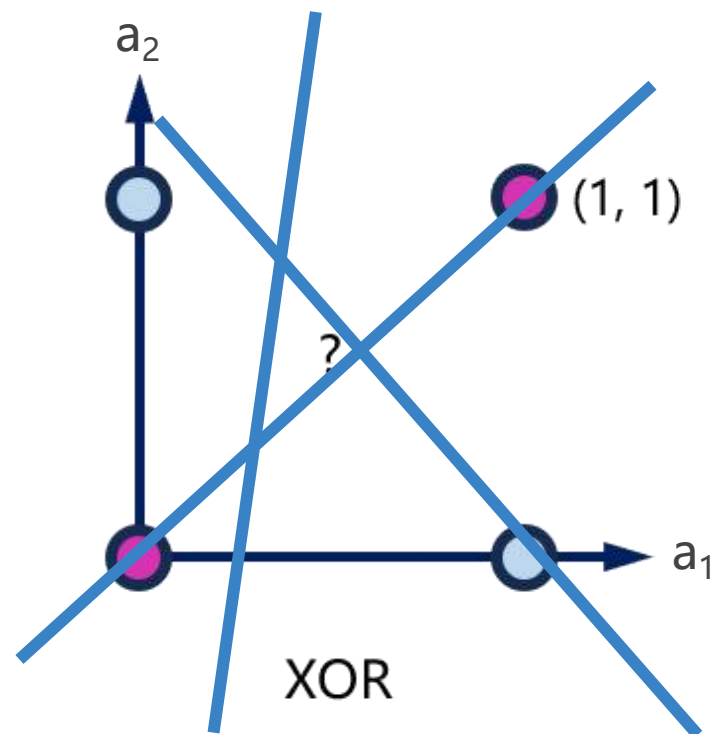
• 实验结论：感知机只能做简单的线性分类任务

- 给定感知机 $z = \text{sgn}(W \times a)$
- 考虑 $W \times a = 0$, 即 $w_1 * a_1 + w_2 * a_2 = 0$
- 不在直线上的点 a 代入 $W \times a$, 结果 >0 或 <0 经过 sgn 函数的作用就会划分出1或-1两个分类

一条直线



决策平面



怎么解决XOR（异或）问题呢？

多层感知机

- 单层神经网络无法解决异或问题。但是当增加一个计算层以后，两层神经网络不仅可以解决异或问题，而且具有非常好的非线性分类效果。不过两层神经网络的计算是一个问题，没有一个较好的解法
- 1986 年，Rumelhart 和 Hinton 等人提出了反向传播（Backpropagation，BP）算法，解决了两层神经网络所需要的复杂计算量问题，从而带动了业界使用两层神经网络研究的热潮
- 这时候的Hinton还很年轻，30年以后，正是他重新定义了神经网络，带来了神经网络复苏



David Rumelhart

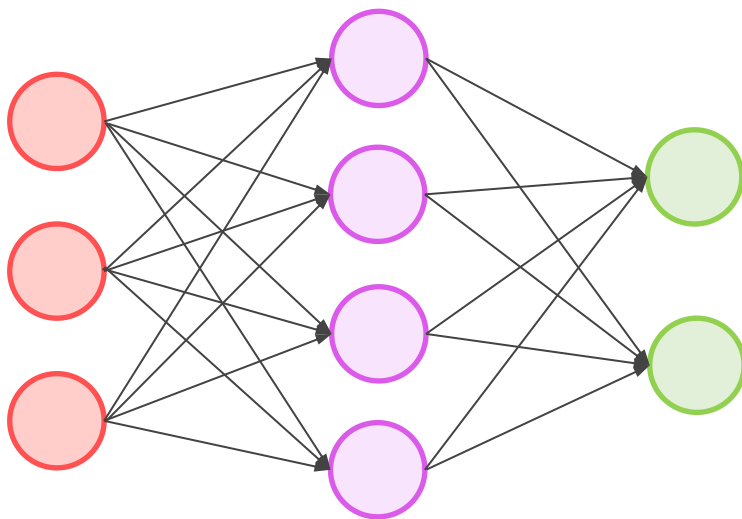


Geoffery Hinton

多层感知机模型（多层神经网络/MLP）

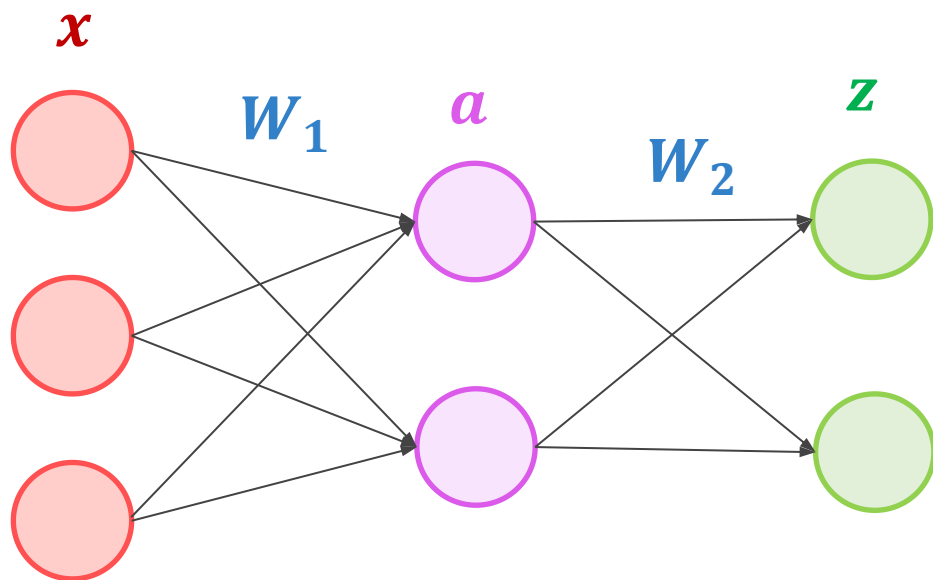
- **多层神经网络**：即**多层感知机（Multi-Layer Perceptron，简称MLP）**
- 包含三个层次：一个**输入层**，一个或多个中间层（也叫**隐藏层**，hidden layer）和一个**输出层**
- 输入层与输出层的节点数是固定的，中间层则可以自由指定

输入层 隐藏层 输出层



多层感知机模型数学化表达

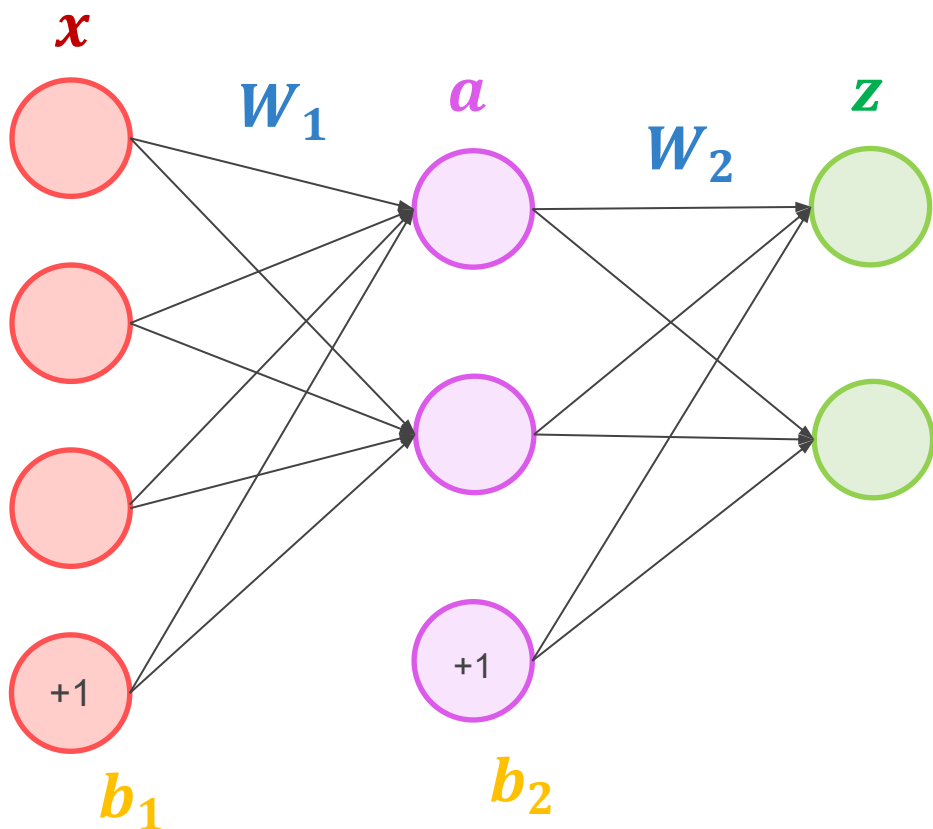
- 使用向量和矩阵来表示神经网络中的变量。 x , a , z 是网络中传输的向量数据。 W_1 和 W_2 是网络的矩阵参数
- 矩阵运算具有简洁性，节点数增多也不受影响



$$\begin{aligned}
 a &= g(W_1 \times x) \\
 z &= g(W_2 \times a) \\
 &= g(W_2 \times g(W_1 \times x))
 \end{aligned}$$

多层感知机模型数学化表达

- MLP通常还会引入**偏置单元**，它与后一层的所有节点都有连接
- 记所有偏置值为向量 b ，则MLP的矩阵运算公式为：



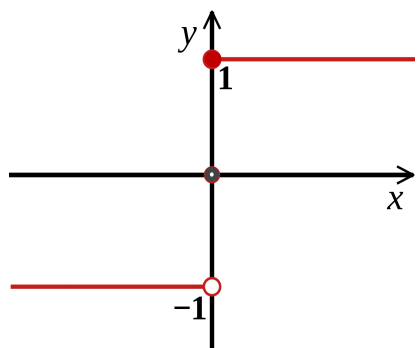
$$\begin{aligned}
 a &= g_1(W_1 \times x + b_1) \\
 z &= g_2(W_2 \times a + b_2) \\
 &= g_2(W_2 \times g_1(W_1 \times x + b_1) + b_2)
 \end{aligned}$$

多层感知机模型激活函数

- 两层神经网络不再使用sgn函数作为激活函数，而是使用平滑函数sigmoid

不同的激活函数

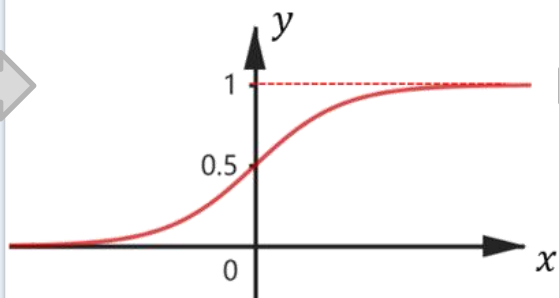
$$\text{sgn}(x) = \begin{cases} 1, & x > 0 \\ -1, & x < 0 \end{cases}$$



(a) sgn

用于MP神经元, 感知机

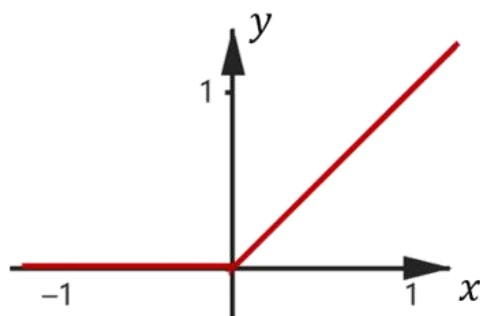
$$\text{sigmoid}(x) = \frac{1}{1 + e^{-x}}$$



(b) sigmoid

多用于多层神经网络(MLP)

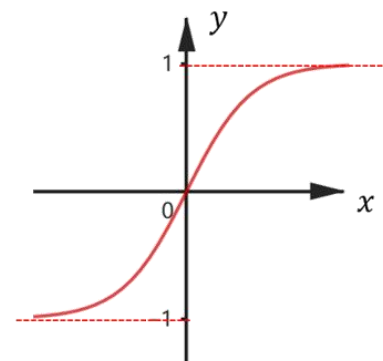
$$\text{ReLU}(x) = \max(0, x)$$



(c) ReLU

多用于深层神经网络(DNN)

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$



(d) tanh

多用于深层神经网络(DNN)

多层神经网络的拟合原理

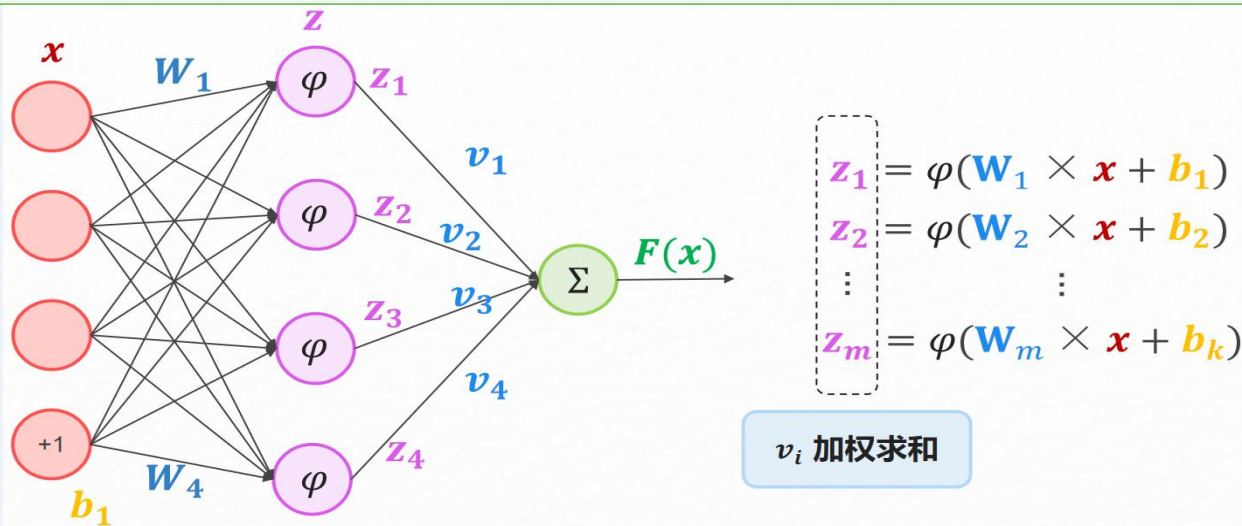
通用近似定理(Universal Approximation Theorem)[Cybenko,1989, Hornik et al., 1989]:

令 $\varphi(\cdot)$ 是一个非常数、有界、单调递增的连续函数, $\mathbb{I}_d$ 是一个d维的单位超立方体 $[0,1]^d$, $C(\mathbb{I}_d)$ 是定义在 $\mathbb{I}_d$ 上的连续函数集合。对于任何一个函数 $f(x) \in C(\mathbb{I}_d)$, 存在一个整数 m , 和一组实数 $v_i, b_i \in \mathbb{R}$ 以及实数向量 $W_i \in \mathbb{R}^d, i = 1, \dots, m$, 可以定义函数:

$$F(x) = \sum_{i=1}^m v_i \varphi(W_i^T x + b_i)$$

作为函数 f 的近似实现, 即:

$$|F(x) - f(x)| < \epsilon, \forall x \in L_d \quad \text{其中} \epsilon > 0 \text{ 是一个很小的正数。}$$





多层神经网络的拟合原理

通用近似定理(Universal Approximation Theorem)[Cybenko,1989, Hornik et al., 1989]:

令 $\varphi(\cdot)$ 是一个非常数、有界、单调递增的连续函数, $\mathbb{I}_d$ 是一个d维的单位超立方体 $[0,1]^d$, $C(\mathbb{I}_d)$ 是定义在 $\mathbb{I}_d$ 上的连续函数集合。对于任何一个函数 $f(x) \in C(\mathbb{I}_d)$, 存在一个整数 m , 和一组实数 $v_i, b_i \in \mathbb{R}$ 以及实数向量 $W_i \in \mathbb{R}^d, i = 1, \dots, m$, 可以定义函数:

$$F(x) = \sum_{i=1}^m v_i \varphi(W_i^T x + b_i)$$

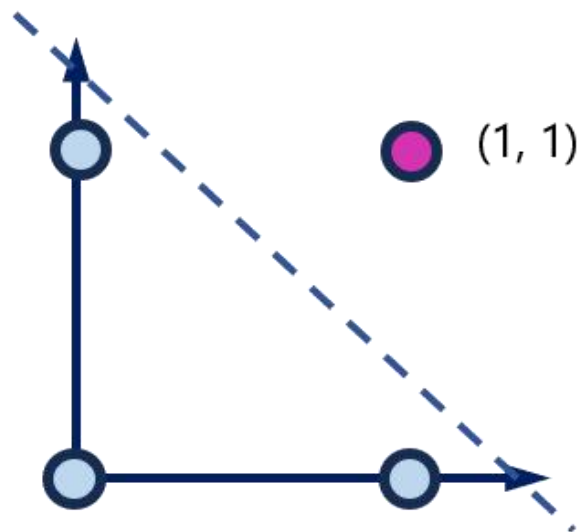
作为函数 f 的近似实现, 即:

$$|F(x) - f(x)| < \epsilon, \forall x \in L_d \quad \text{其中} \epsilon > 0 \text{ 是一个很小的正数。}$$

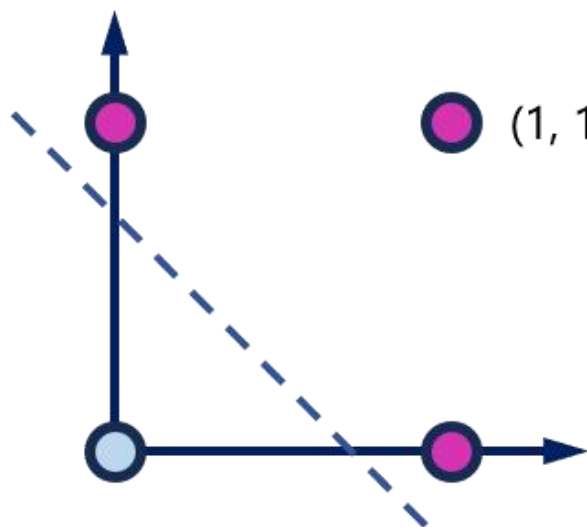
- 多层神经网络的本质就是通过矩阵参数与激活函数来拟合特征与目标之间的真实函数关系
- 因此, 多层神经网络也能够解决XOR异或问题!

多层感知机模型效果：XOR解决之道

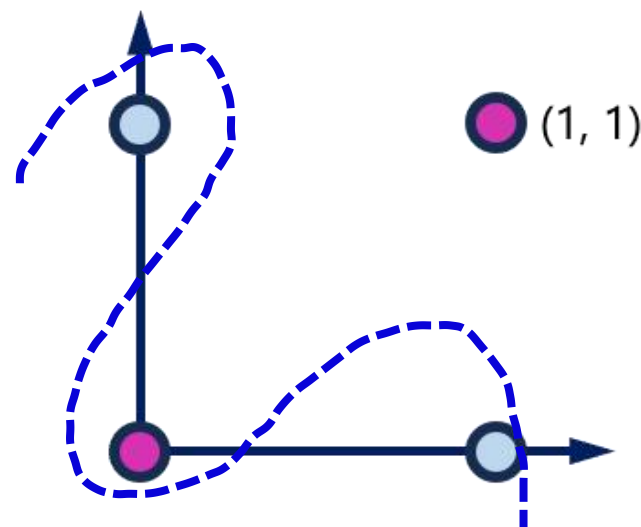
为什么多层神经网络就可以解决XOR异或决策问题？



AND



OR



XOR



输出 0



输出 1

多层感知机模型效果：XOR解决之道

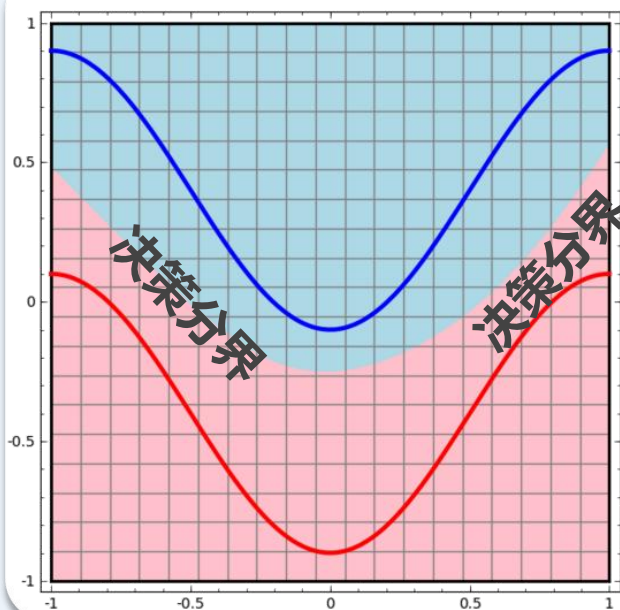
例子

用红色的线与蓝色的线代表数据

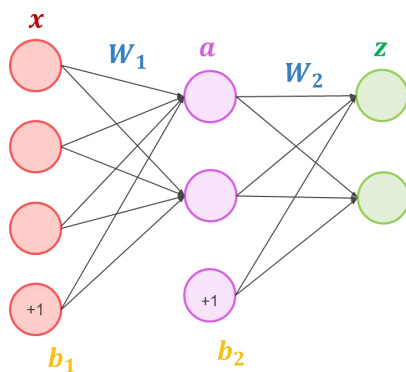
- 红色区域和蓝色区域代表由一个两层神经网络划开的区域，两者的分界线就是决策分界

- 可见决策分界是非常平滑的曲线，而且分类的很好

$$z = g_2(W_2 \times g_1(W_1 \times x + b_1) + b_2)$$



拟合



问题

Q: 单层网络只能做线性分类任务。而两层神经网络中的两层（隐藏层和输出层）也都是线性层，应该只能做线性分类任务。

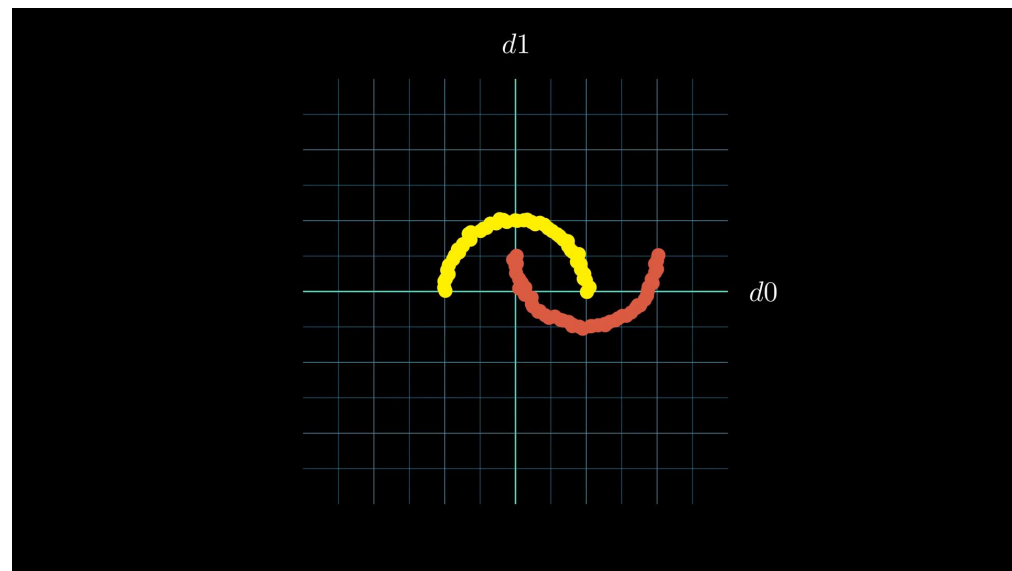
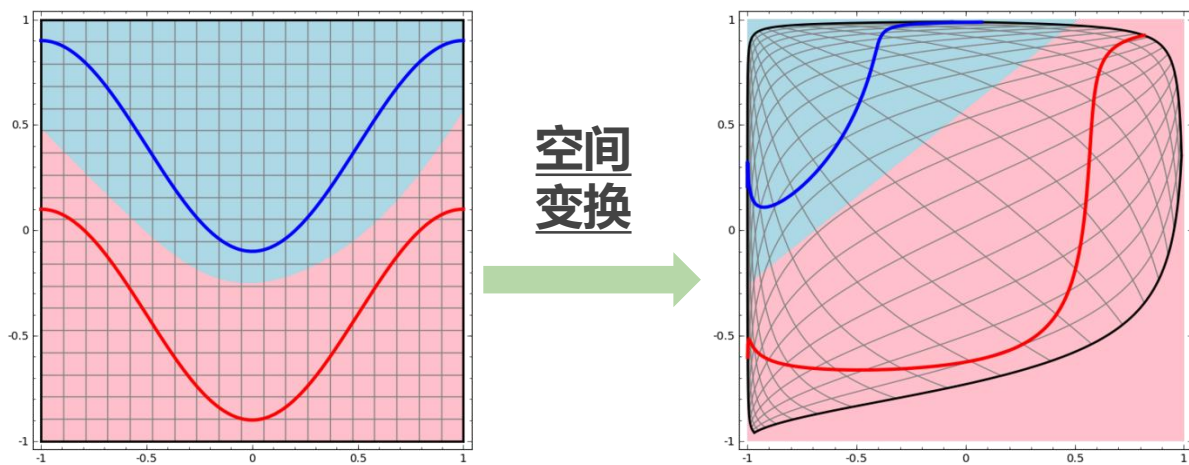
为什么两个线性层的结合就可以做非线性分类任务？

反过来想，如果 g_1 是一个线性函数会发生什么？

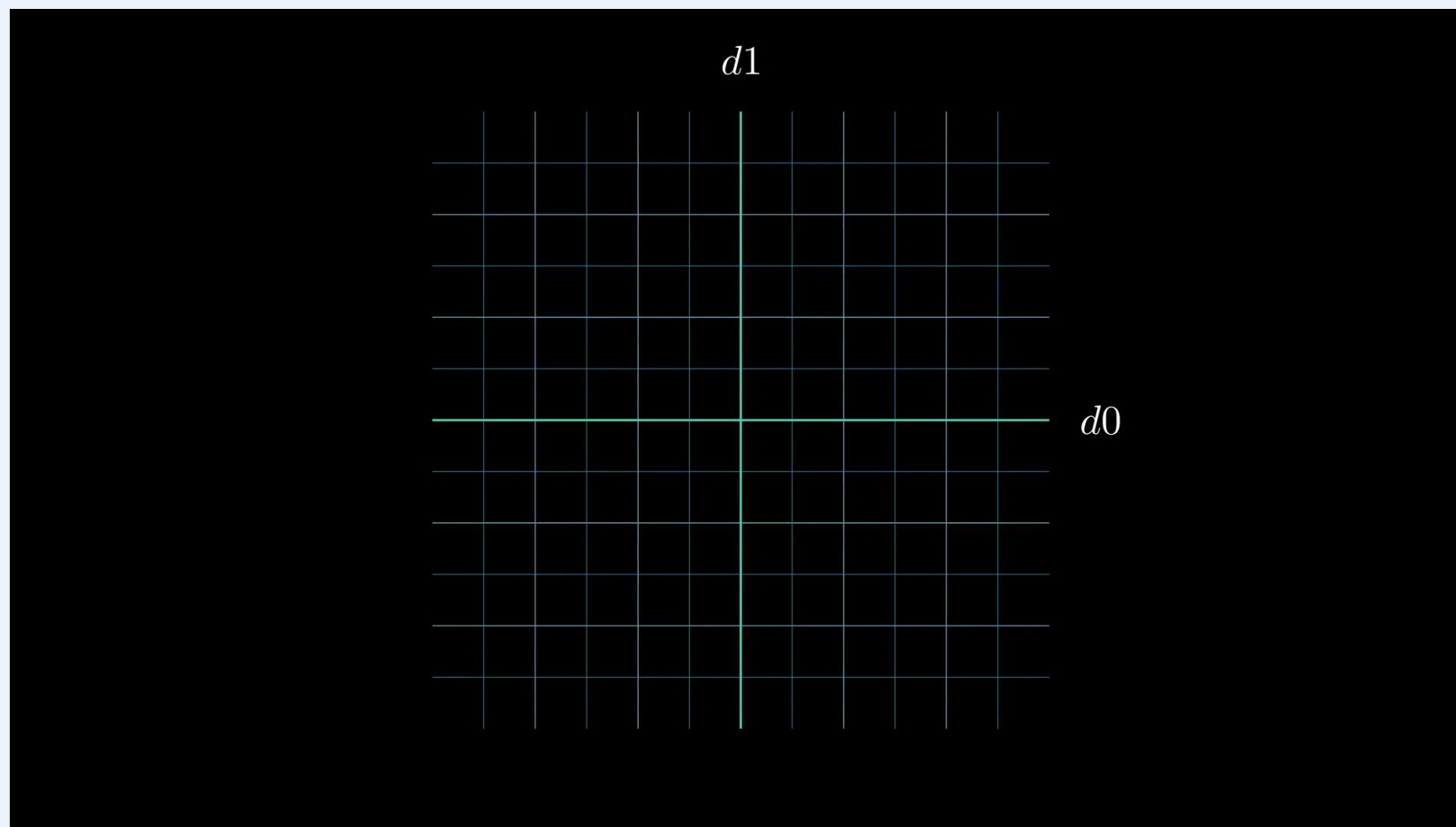
多层感知机模型效果：XOR解决之道

空间变换

- **关键在于**：从输入层到隐藏层时，数据在**激活函数作用**下发生了**空间扭曲**
- 两层神经网络中，隐藏层对原始的数据进行了一个空间变换，使其可以被线性分类，然后输出层划分出了一个线性分类分界线，对其进行分类



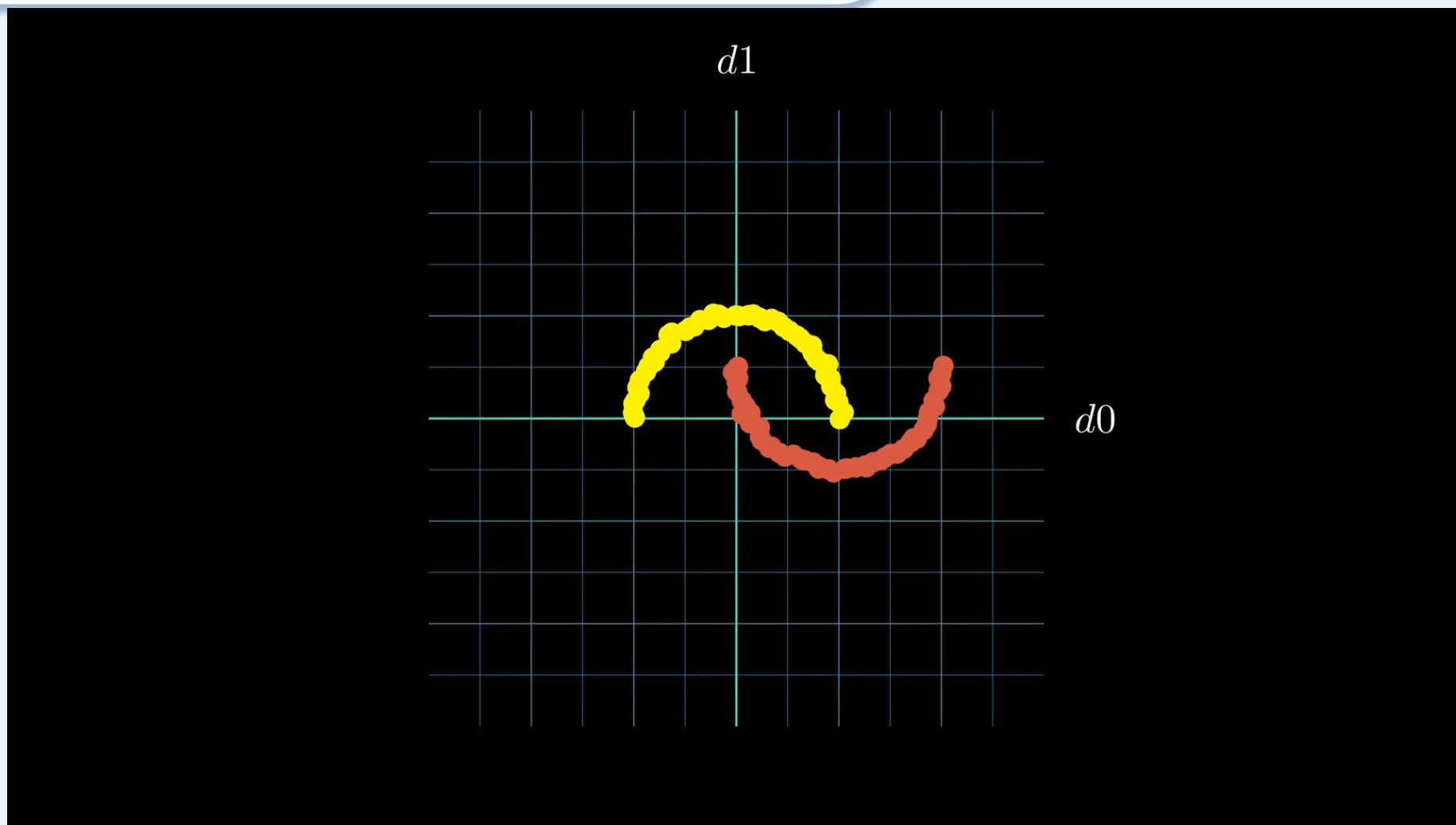
多层感知机效果



$$\begin{aligned}z^{(1)} &= \tanh(\mathbf{W}_1 \mathbf{x} + \mathbf{b}_1) \\z^{(2)} &= \tanh(\mathbf{W}_2 \mathbf{z}^{(1)} + \mathbf{b}_2) \\y &= \tanh(\mathbf{W}_3 \mathbf{z}^{(2)} + \mathbf{b}_3)\end{aligned}$$

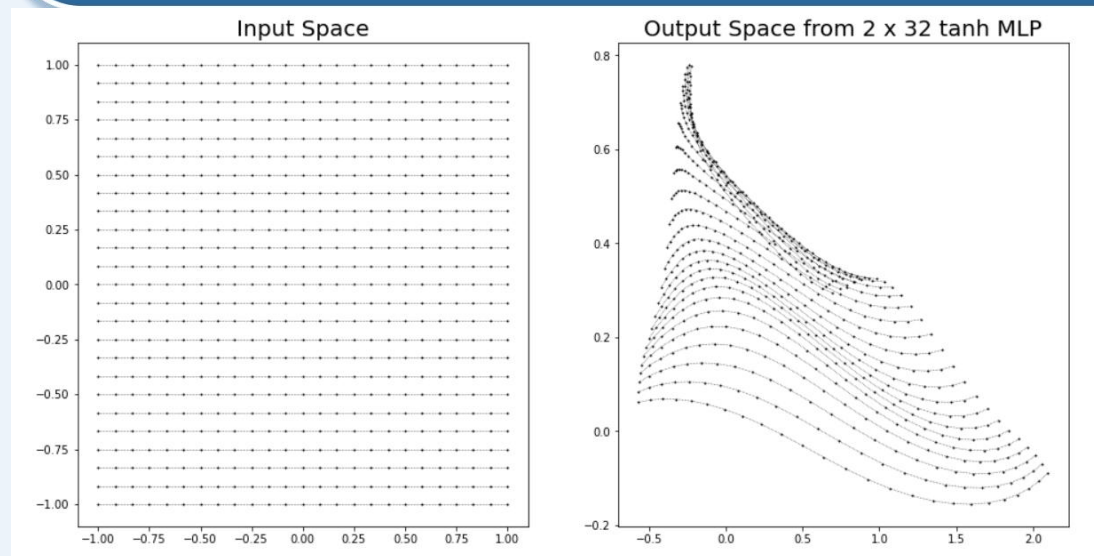
Neural Network Transformation Visualization

多层感知机效果



Neural Network Decision Boundary Visualization

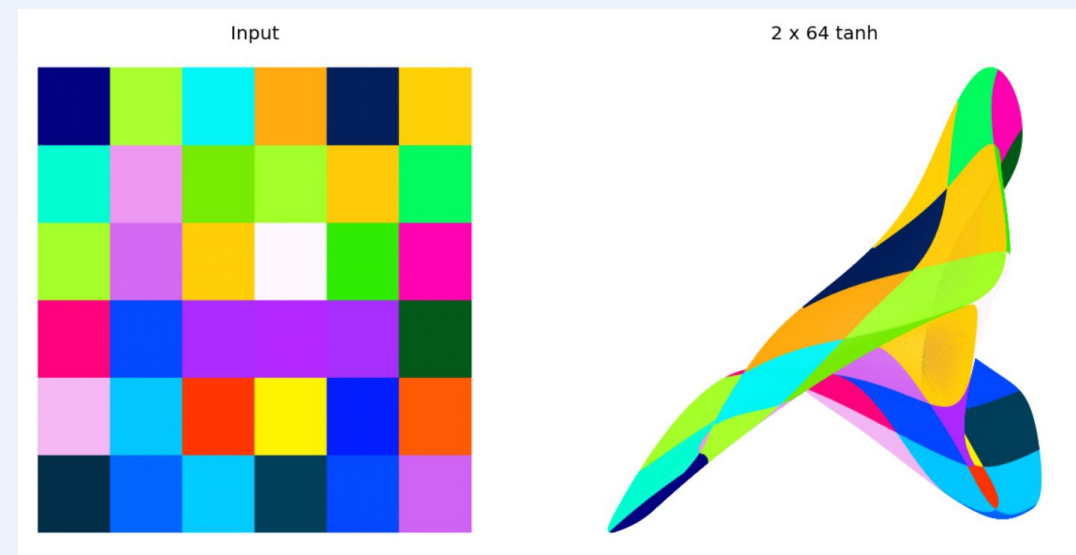
多层感知机效果

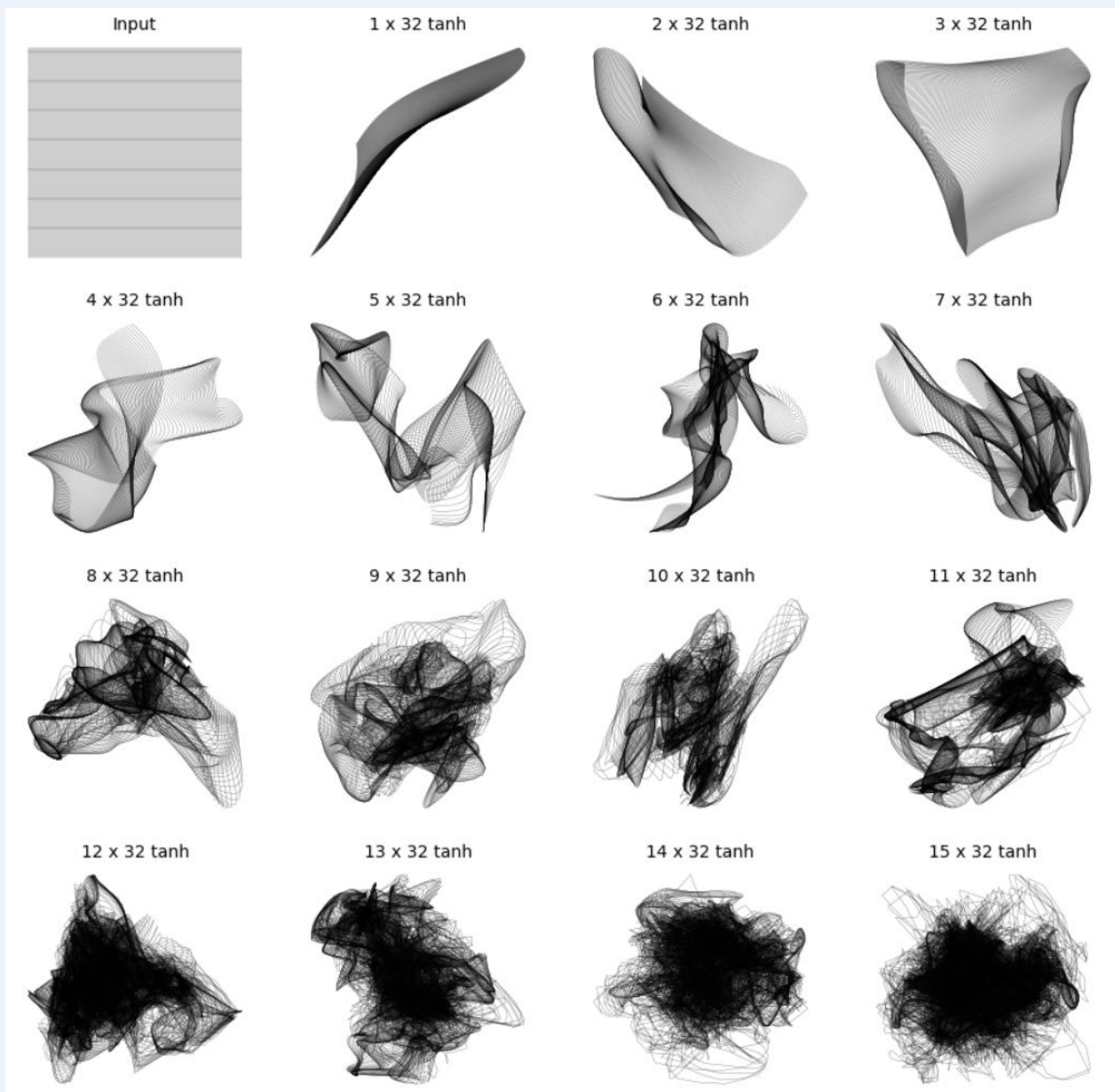


```
units_per_layer = 64
num_layers = 2
activation_fn = 'tanh'
```

```
units_per_layer = 32
num_layers = 2
activation_fn = 'tanh'
```

https://colab.research.google.com/github/murphyka/ml_colabs/blob/main/Simple_MLP_Visualization.ipynb#scrollTo=HpMd cCE7NoB0

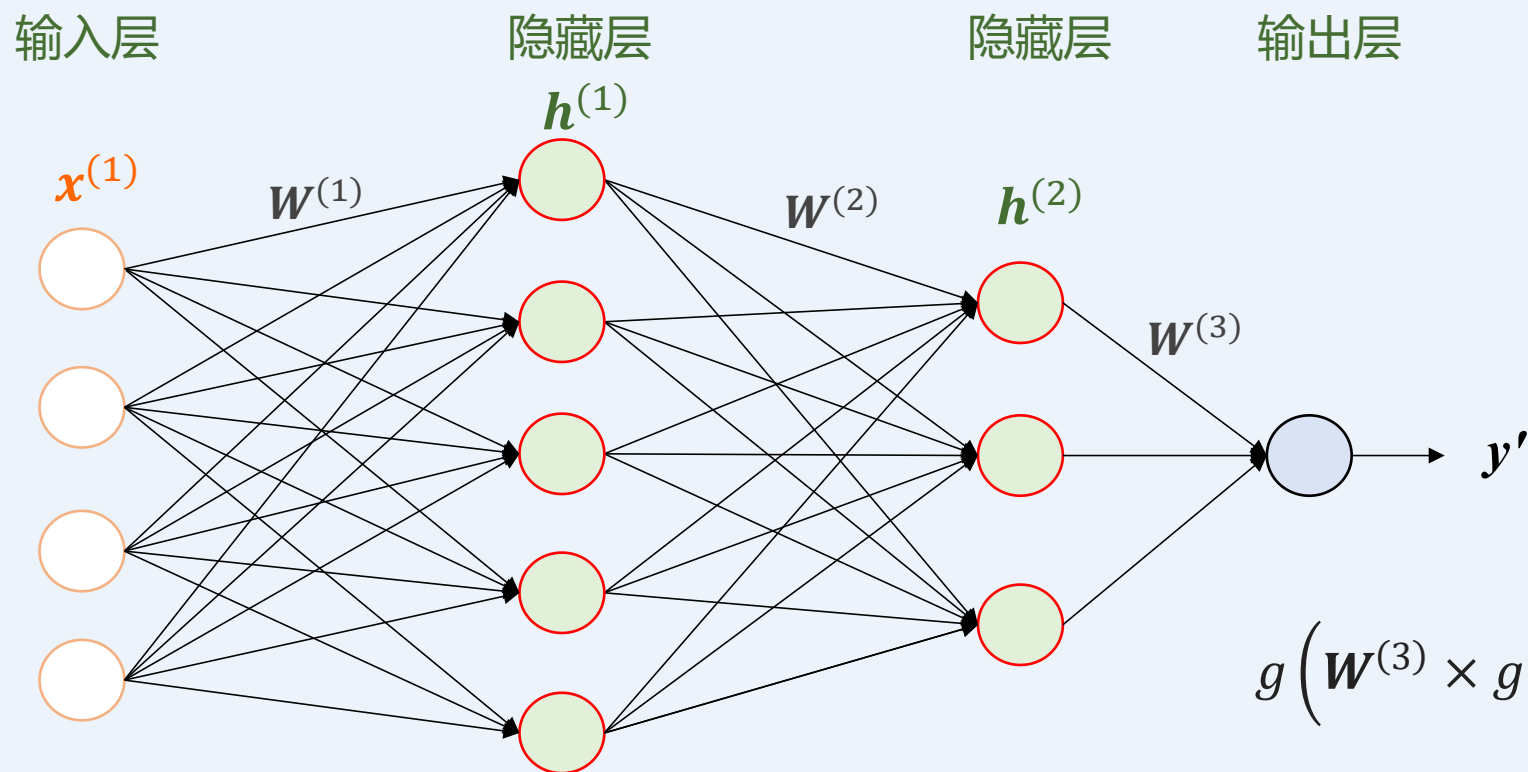




多层感知机效果

- 两层神经网络可以做非线性分类的关键--隐藏层
- 矩阵公式中矩阵和向量相乘以及非线性函数sigmoid/tanh/..., 本质上就是对向量的坐标空间进行一个变换。因此, 隐藏层的作用就是使得数据的原始坐标空间**从线性不可分, 转换成了线性可分**
- 两层神经网络通过两层的线性模型模拟了数据内真实的非线性函数。因此, 多层的神经网络的**本质就是复杂函数拟合**
- 设计一个神经网络时, 输入层的节点数需要与特征的维度匹配, 输出层的节点数要与目标的维度匹配。如何决定这个自由层的节点数呢? 一般根据经验来设置
- Grid Search (网格搜索): 预先设定几个可选值, 通过切换这几个值来看整个模型的预测效果, 选择效果最好的值作为最终选择

多层感知机结构与数学化



g 是激活函数

$$g(W^{(1)} \times x^{(1)}) = h^{(1)}$$

$$g(W^{(2)} \times h^{(1)}) = h^{(2)}$$

$$g(W^{(3)} \times h^{(2)}) = y'$$

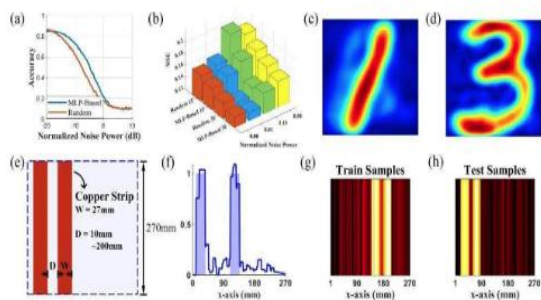
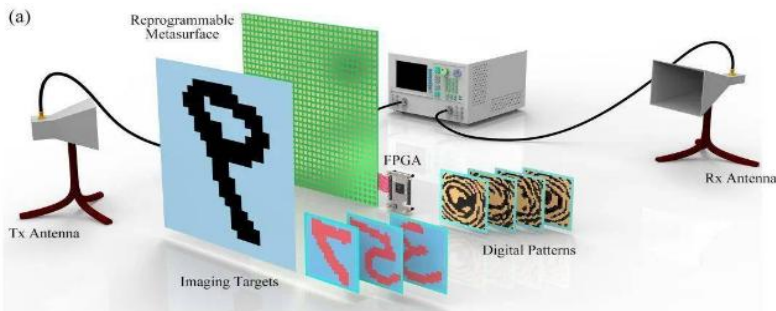
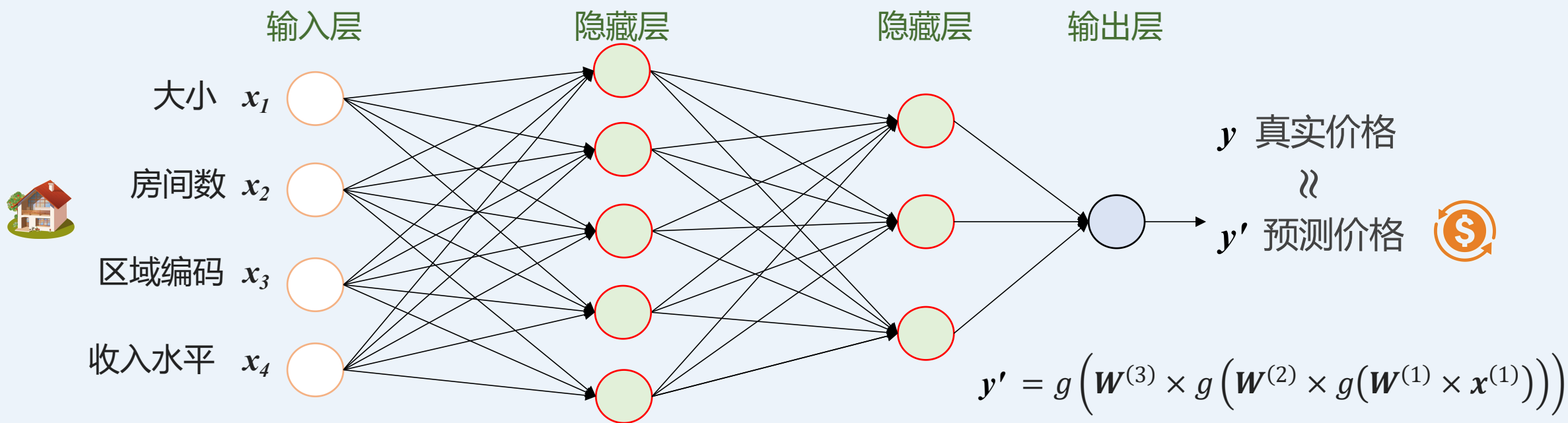


$$g\left(W^{(3)} \times g\left(W^{(2)} \times g\left(W^{(1)} \times x^{(1)}\right)\right)\right) = y'$$

神经网络：通过矩阵参数 W 与激活函数 g 来拟合特征与目标之间的真实关系

多层感知机结构与数字化

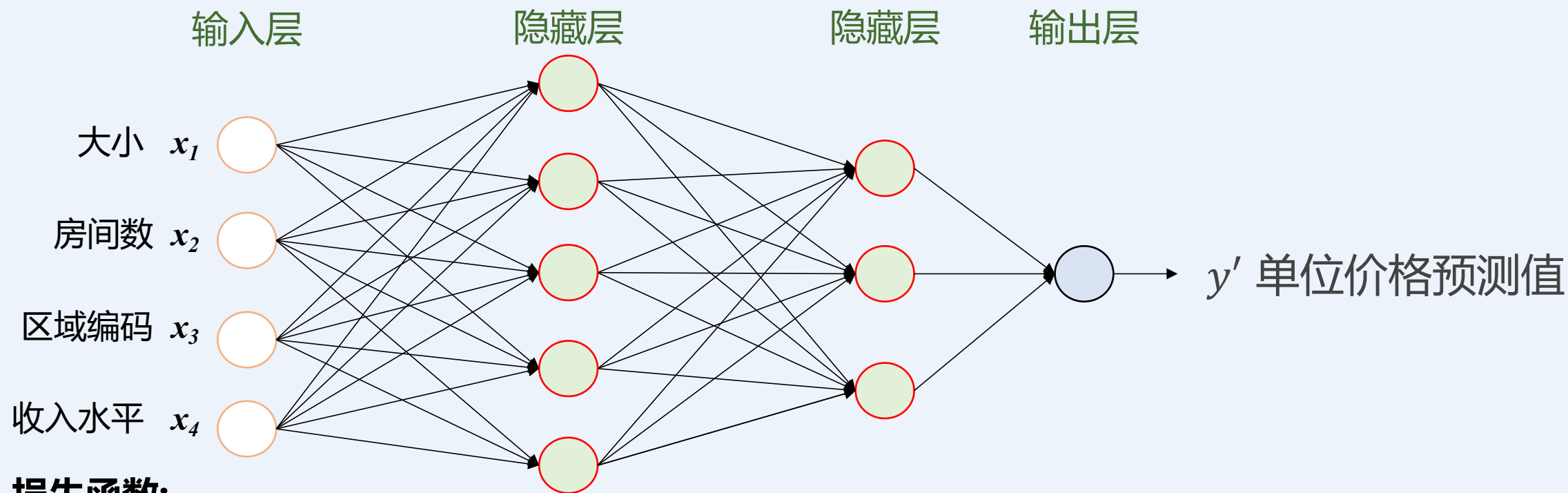
多层感知机如何训练



我国崔铁军院士课题组基于多层感知器开发了一种基于高分辨率可编程超表面的微波成像系统。相比一般光学成像，微波成像系统具有高穿透性、抗视距障碍物、全天候等特征，广泛应用于安检、自动化检测和国防等领域。

多层感知机结构与数字化

多层感知机如何训练

**损失函数:**

样本的预测结果为 $out(y')$, 真实目标为 $target(y)$ 。损失 loss 的计算公式如下, 目标是最小化预测结果与真实目标之间的距离

$$Loss = \frac{1}{2} (y - y')^2$$

训练多层感知机

- Rosenblatt提出的感知机模型中，模型中的参数可以被训练，但是使用的方法较为简单，并没有使用目前机器学习中通用的方法，这导致其扩展性与适用性非常有限
- 从两层神经网络开始，神经网络的研究人员开始使用机器学习相关的技术进行神经网络的训练。例如用大量的数据（1000-10000左右），使用算法进行优化等等，从而使得模型训练可以获得性能与数据利用上的双重优势。
- 机器学习模型训练的目的，就是使得参数尽可能的与真实的模型逼近。具体做法是这样的：首先给所有参数赋上随机值，然后使用这些随机生成的参数值，来预测训练数据中的样本，不断循环迭代更新参数。
- 样本的预测目标为 **out**，真实目标为 **target**。**损失** loss的计算公式如下，目标是使对所有训练数据的损失和尽可能的小

$$\text{loss} = (\text{target} - \text{out})^2$$

训练多层感知机

? 如何优化参数，能够让损失函数的值最小？

前向传播

按顺序（从输入层到输出层）计算和存储神经网络中每层的结果



在于**计算**当前参数下的**误差**

反向传播

计算神经网络参数梯度的方法
根据微积分中的链式规则，按相反的顺序从输出层到输入层遍历网络

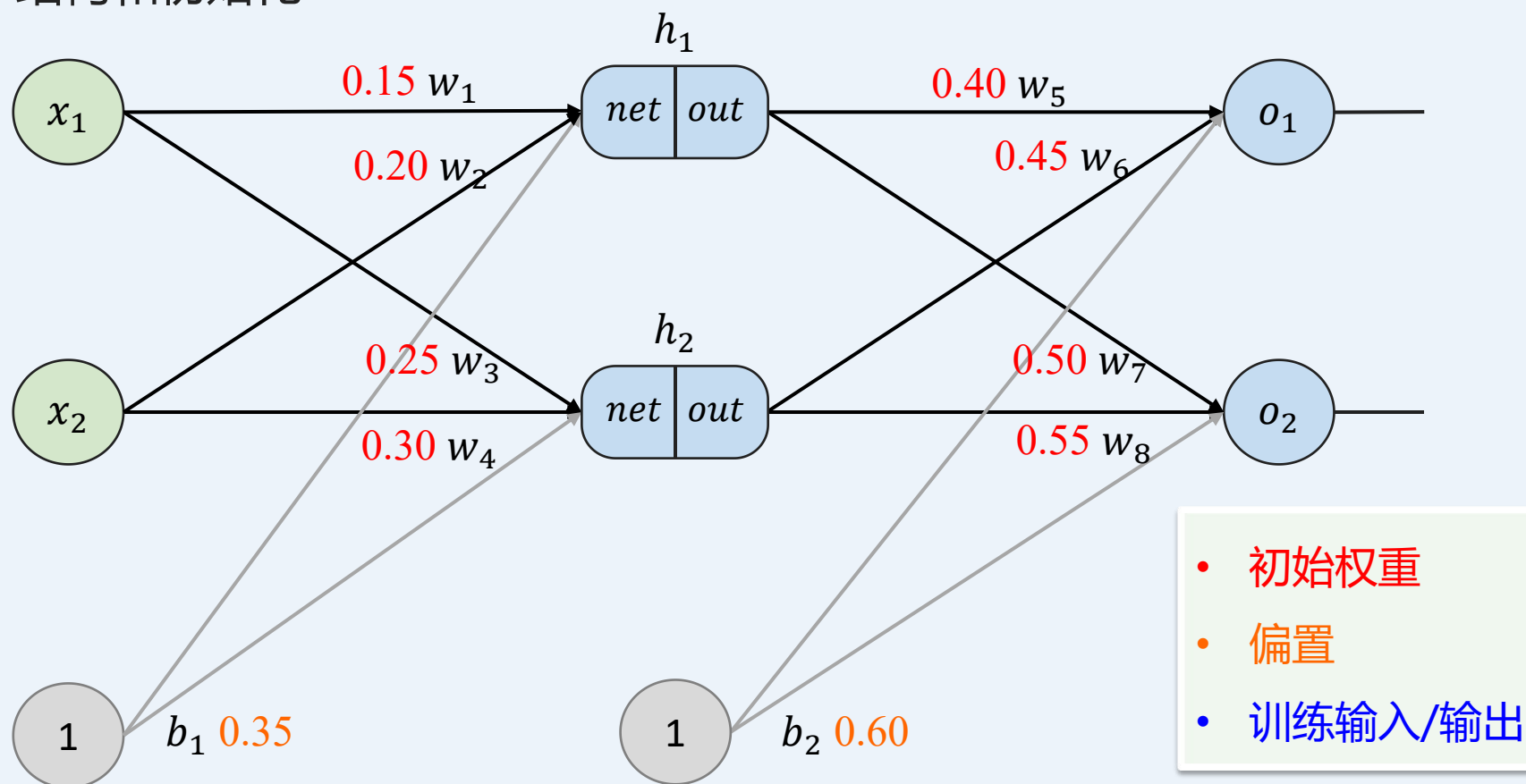


在于**根据误差调整参数**

多层感知机如何训练：前向传播+反向传播

📋 示例：一个具有两个输入、两个隐藏神经元和两个输出神经元的神经网络。

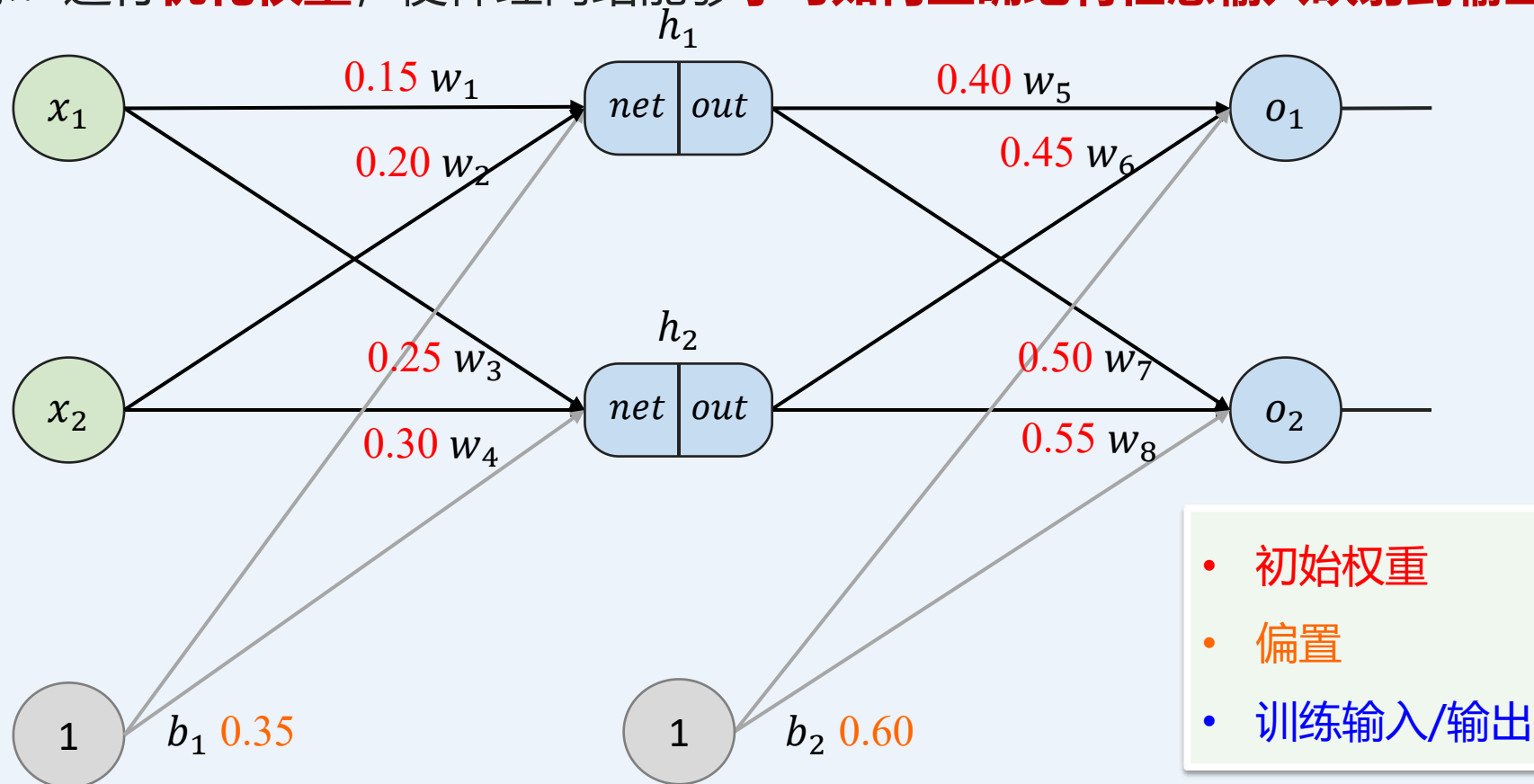
- 基本结构和初始化



多层感知机如何训练：前向传播+反向传播

 示例：一个具有两个输入、两个隐藏神经元和两个输出神经元的神经网络。

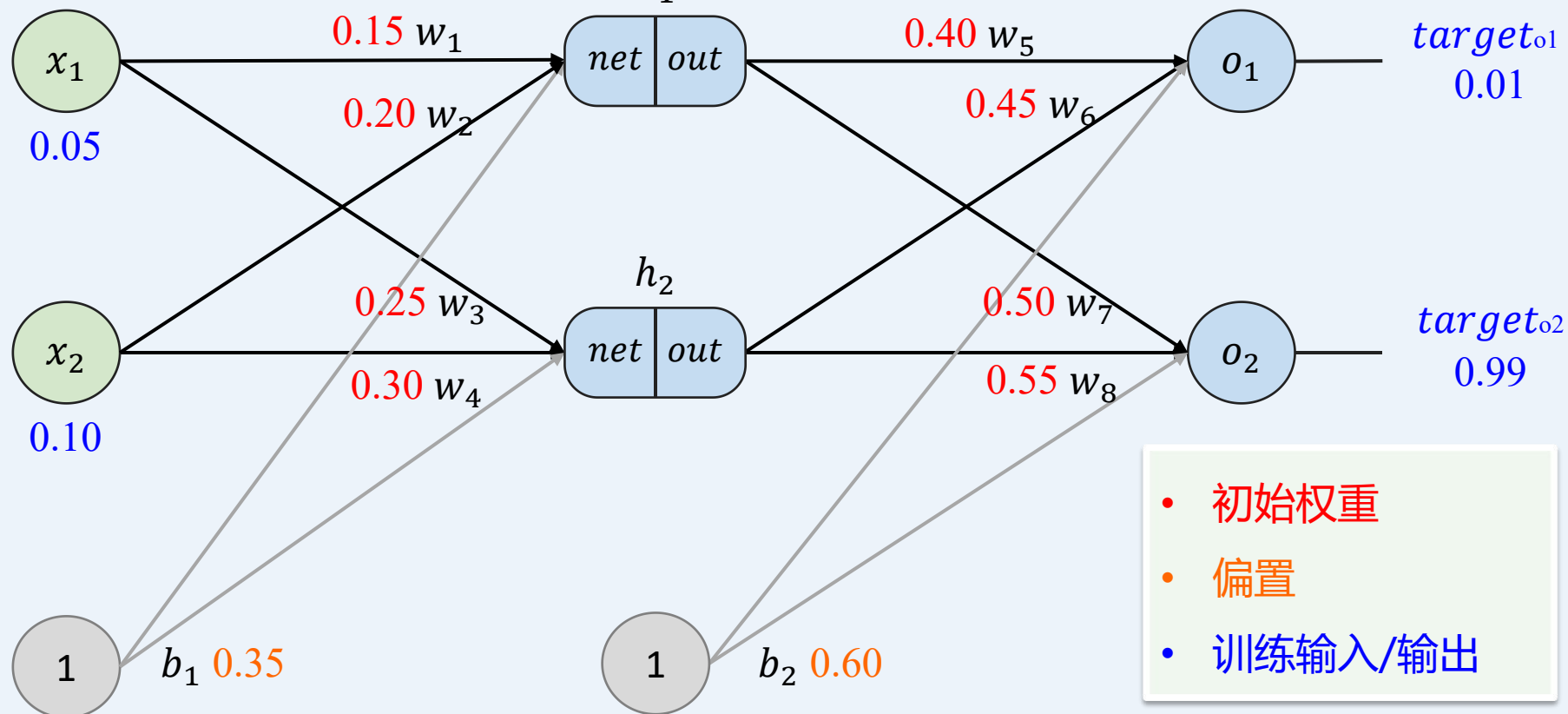
- 目标：进行**优化权重**，使神经网络能够**学习如何正确地将任意输入映射到输出**



多层感知机如何训练：前向传播+反向传播

 示例：一个具有两个输入、两个隐藏神经元和两个输出神经元的神经网络。

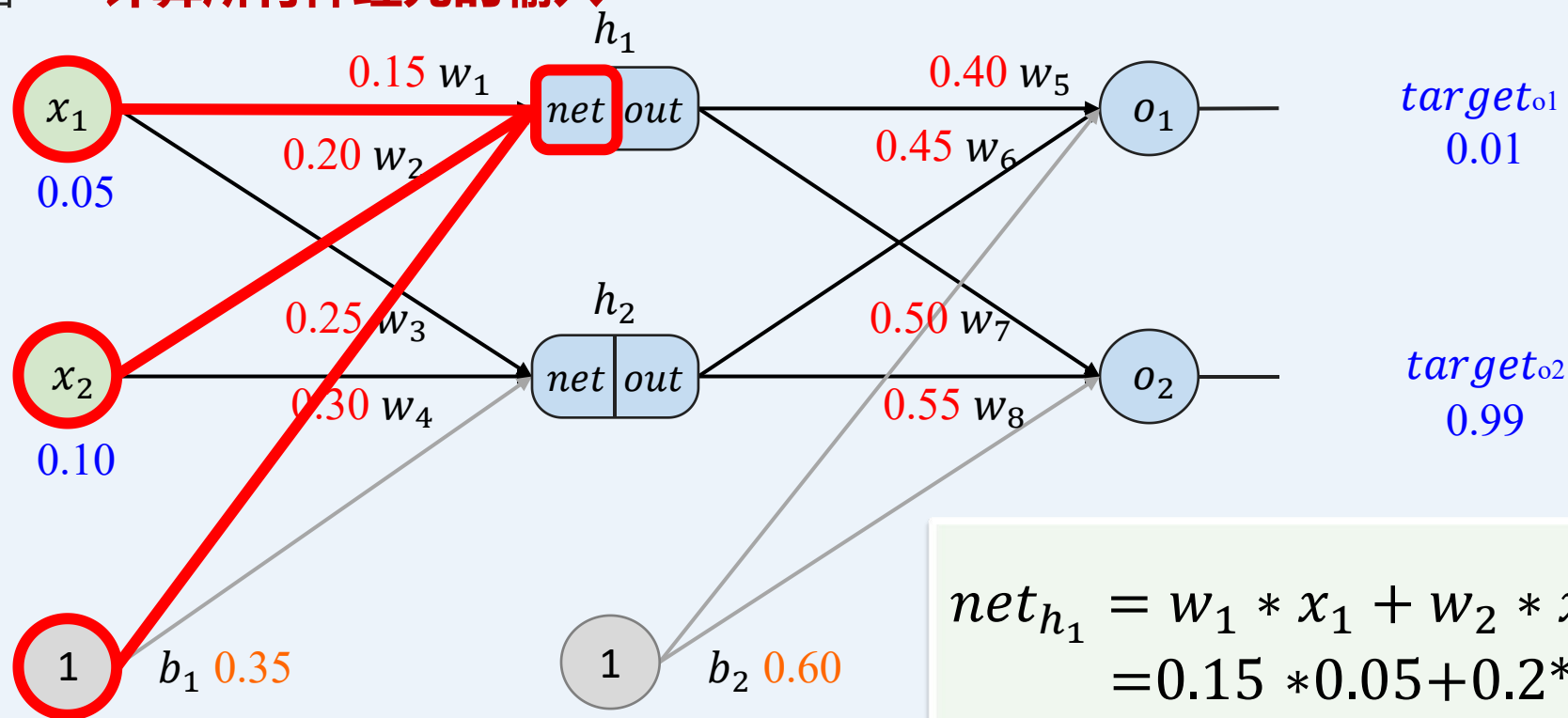
- 单个训练集：给定输入**0.05**和**0.10**，我们希望神经网络输出**0.01**和**0.99**



多层感知机如何训练：前向传播+反向传播

📋 示例：一个具有两个输入、两个隐藏神经元和两个输出神经元的神经网络。

- 前向传播----**计算所有神经元的输入**

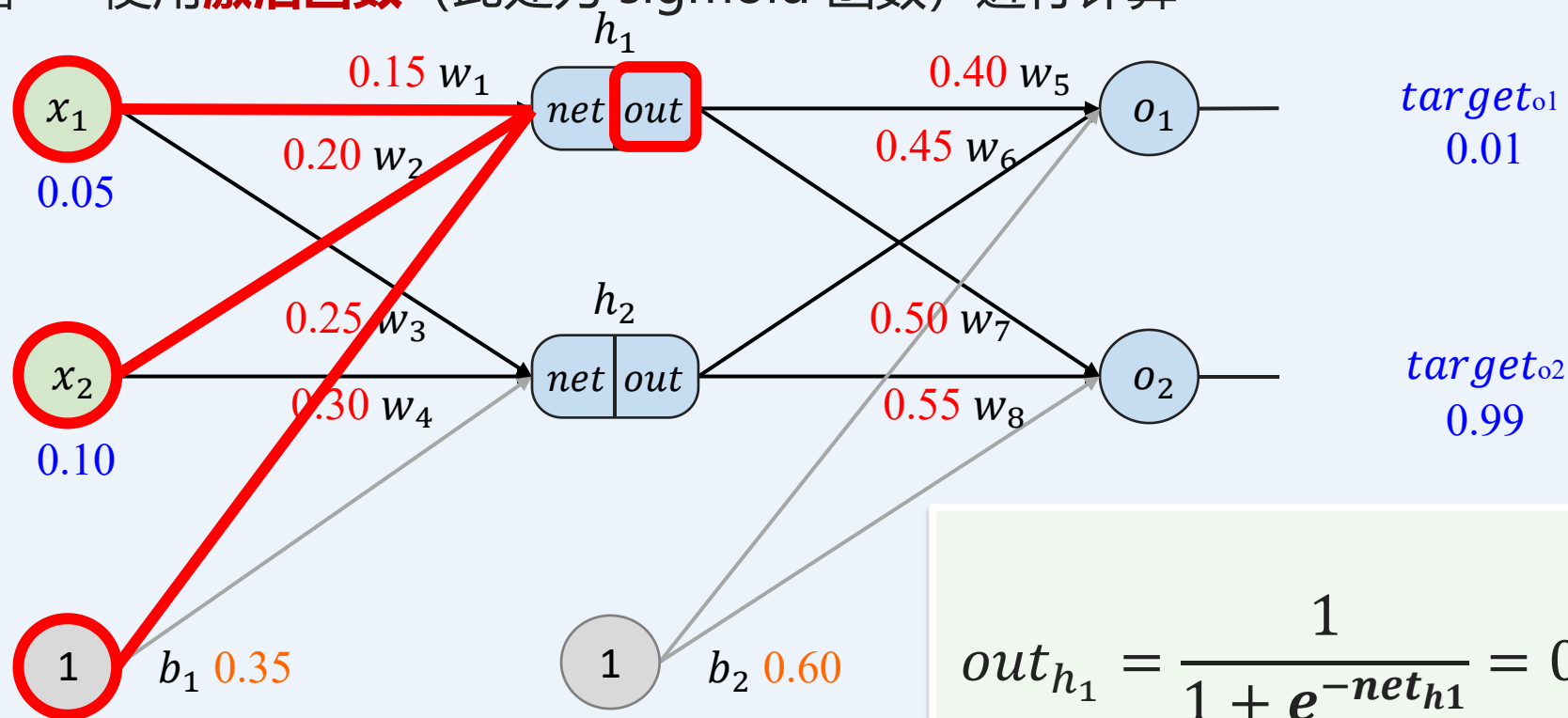


$$\begin{aligned} net_{h_1} &= w_1 * x_1 + w_2 * x_2 + b_1 * 1 \\ &= 0.15 * 0.05 + 0.2 * 0.1 + 0.35 * 1 \\ &= 0.3775 \end{aligned}$$

多层感知机如何训练：前向传播+反向传播

📋 示例：一个具有两个输入、两个隐藏神经元和两个输出神经元的神经网络。

- 前向传播---使用**激活函数**（此处为 sigmoid 函数）进行计算

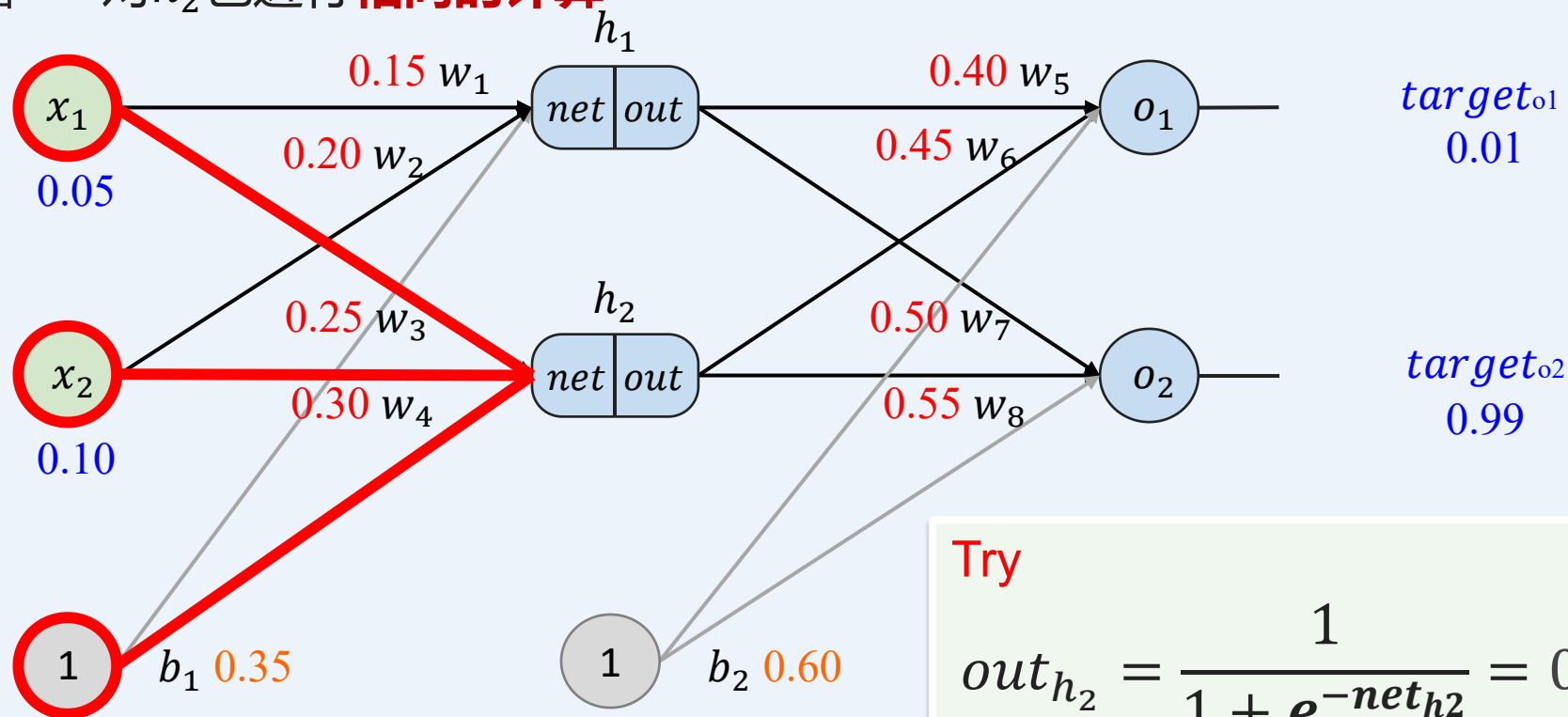


$$out_{h_1} = \frac{1}{1 + e^{-net_{h_1}}} = 0.59326992$$

多层感知机如何训练：前向传播+反向传播

📋 示例：一个具有两个输入、两个隐藏神经元和两个输出神经元的神经网络。

- 前向传播---对 h_2 也进行**相同的计算**



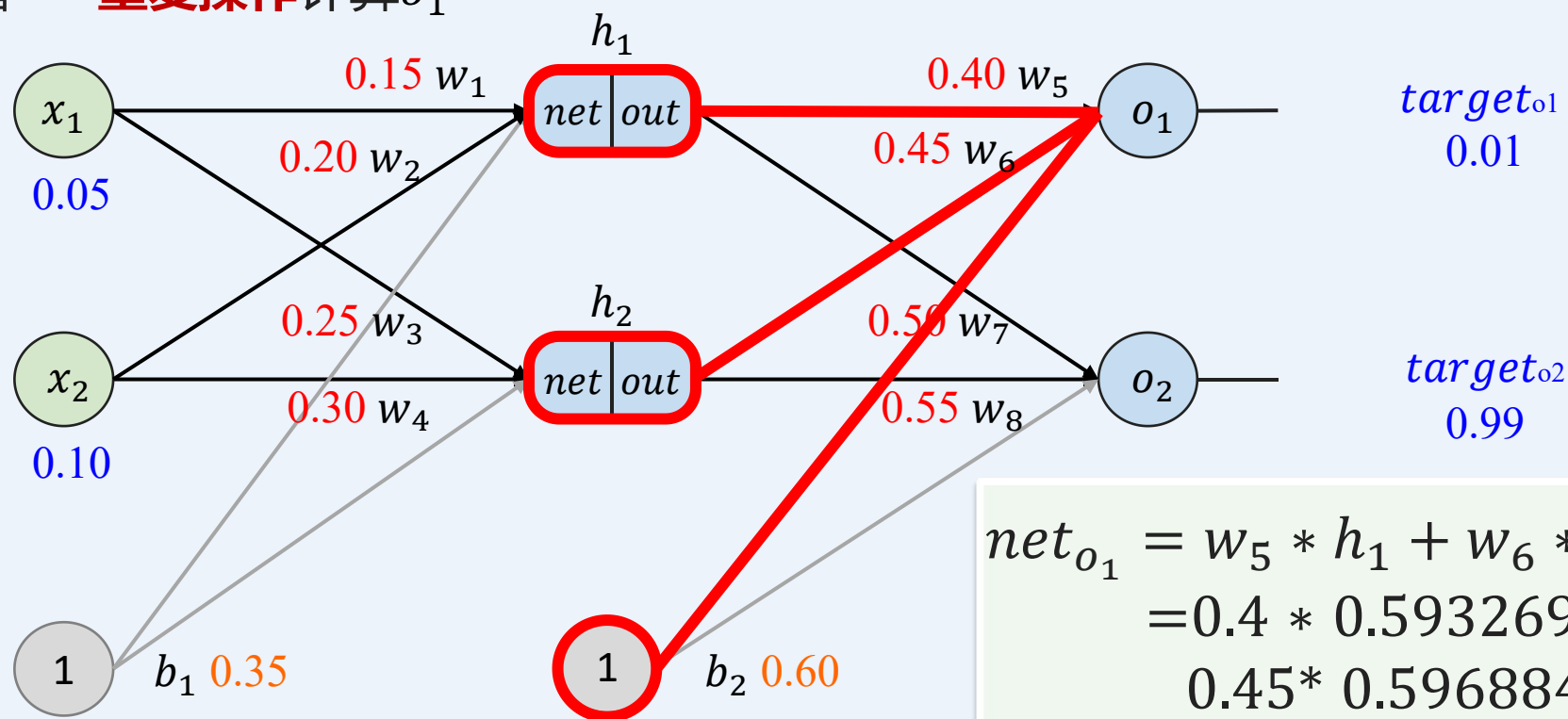
Try

$$out_{h_2} = \frac{1}{1 + e^{-net_{h_2}}} = 0.596884378$$

多层感知机如何训练：前向传播+反向传播

📋 示例：一个具有两个输入、两个隐藏神经元和两个输出神经元的神经网络。

- 前向传播---**重复操作**计算 o_1

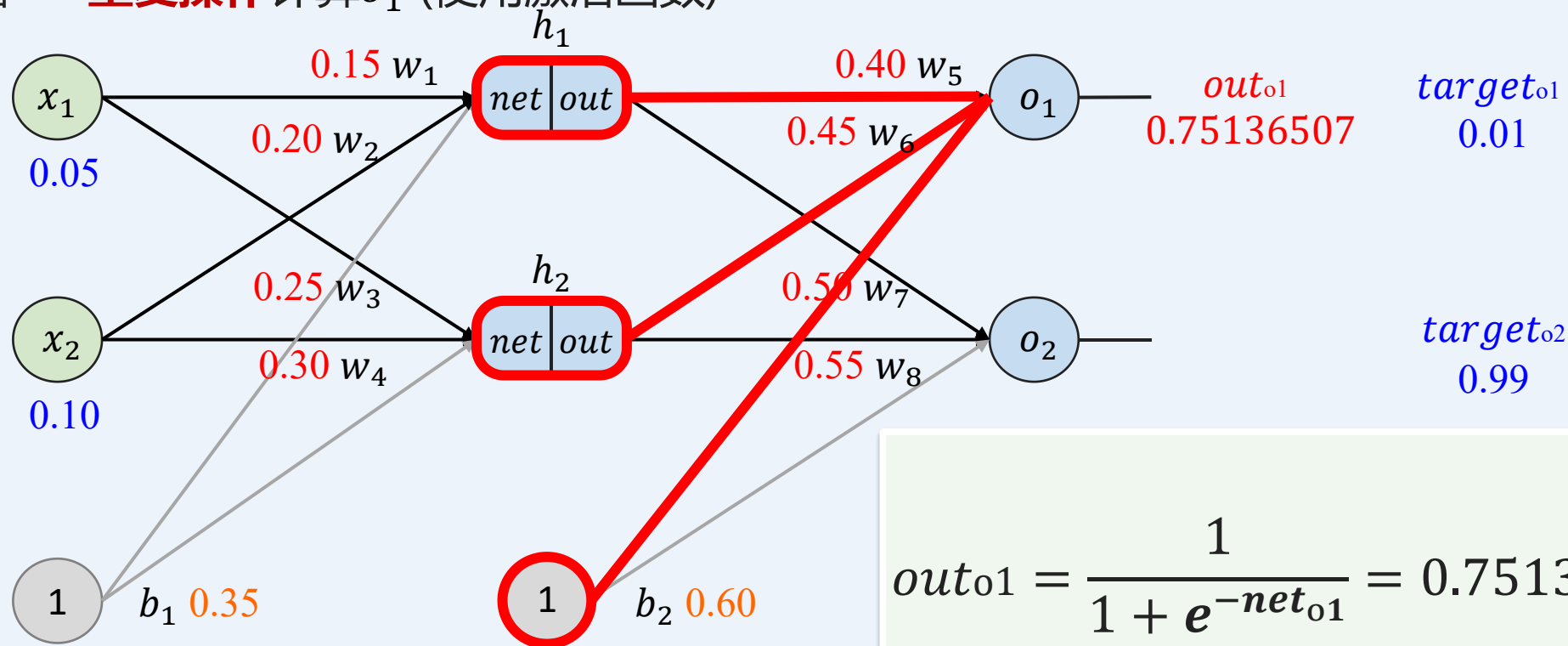


$$\begin{aligned} net_{o_1} &= w_5 * h_1 + w_6 * h_2 + b_2 * 1 \\ &= 0.4 * 0.59326992 + \\ &\quad 0.45 * 0.596884378 + 0.35 * 1 \\ &= 1.105905967 \end{aligned}$$

多层感知机如何训练：前向传播+反向传播

📋 示例：一个具有两个输入、两个隐藏神经元和两个输出神经元的神经网络。

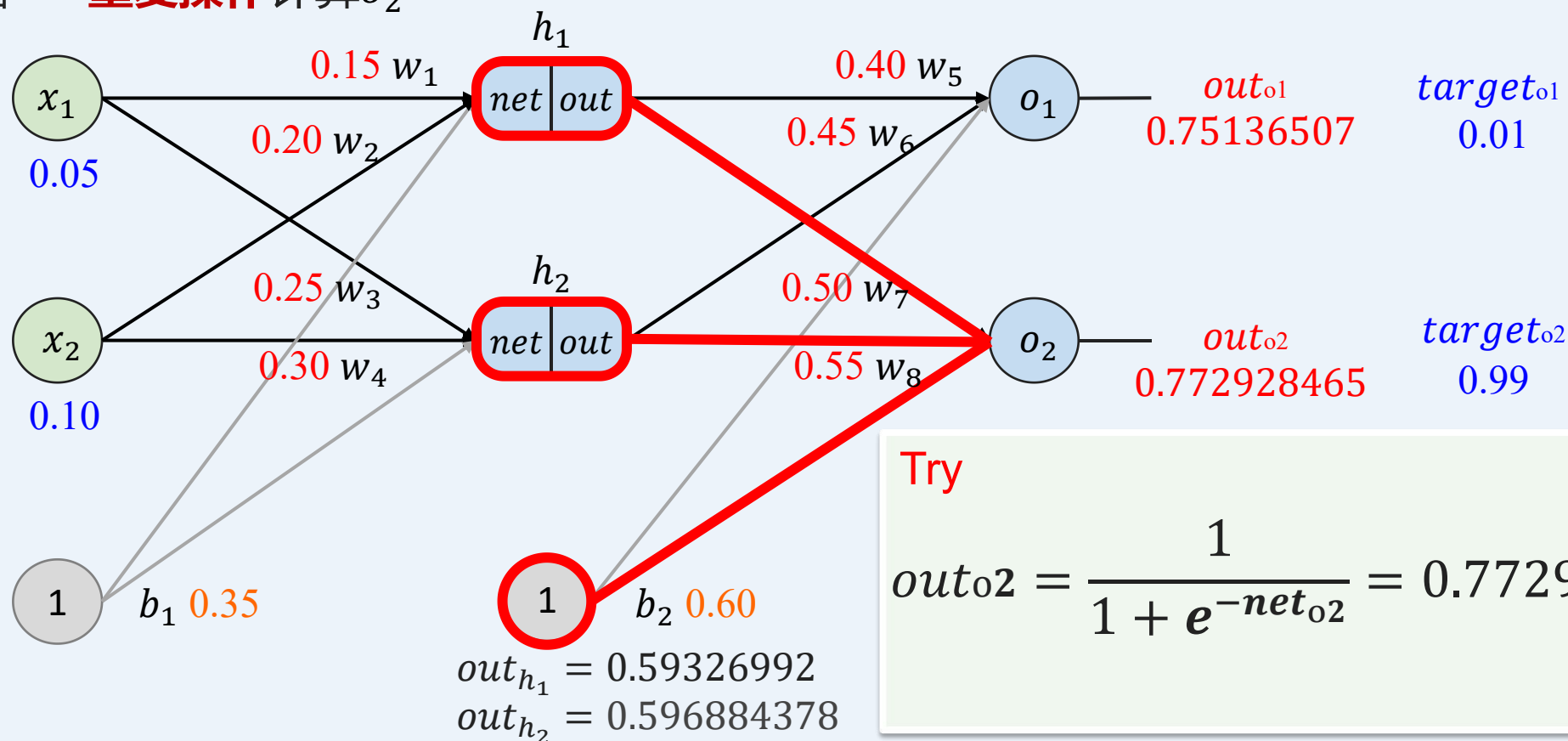
- 前向传播---**重复操作**计算 o_1 (使用激活函数)



多层感知机如何训练：前向传播+反向传播

📋 示例：一个具有两个输入、两个隐藏神经元和两个输出神经元的神经网络。

- 前向传播---**重复操作**计算 o_2



多层感知机如何训练：前向传播+反向传播

📋 示例：一个具有两个输入、两个隐藏神经元和两个输出神经元的神经网络。

$$E_{total} = \frac{1}{2} \sum (target - output)^2$$

$$\begin{aligned} E_{o1} &= \frac{1}{2} (target_{o1} - out_{o1})^2 \\ &= \frac{1}{2} (0.01 - 0.75136507)^2 = 0.274811083 \end{aligned}$$

$$\begin{aligned} E_{o2} &= \frac{1}{2} (target_{o2} - out_{o2})^2 \\ &= \frac{1}{2} (0.99 - 0.772928465)^2 = 0.023560026 \end{aligned}$$

$$E_{total} = E_{o1} + E_{o2} = 0.298371109$$

预测与目标的总误差！

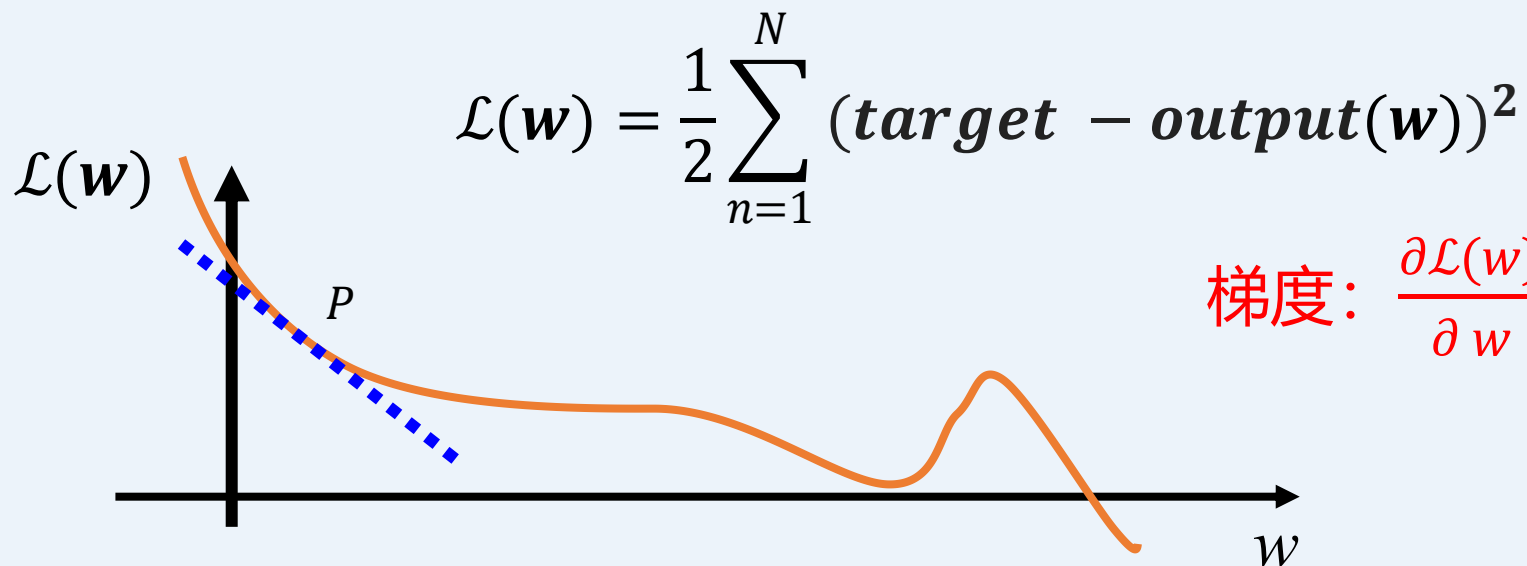
$$\begin{array}{cc} out_{o1} & \longleftrightarrow target_{o1} \\ 0.75136507 & 0.01 \end{array}$$

$$\begin{array}{cc} out_{o2} & \longleftrightarrow target_{o2} \\ 0.772928465 & 0.99 \end{array}$$

问题：如何根据总误差调整参数 w

多层感知机如何训练：梯度下降

回顾数学分析： 给定多元可微函数 $\mathcal{L}(\mathbf{w})$ 和其函数曲线上的一点 P ，函数 $\mathcal{L}$ 在点 P 上的**梯度**就是**函数 $\mathcal{L}(\mathbf{w})$ 在点 P 上的偏导数**



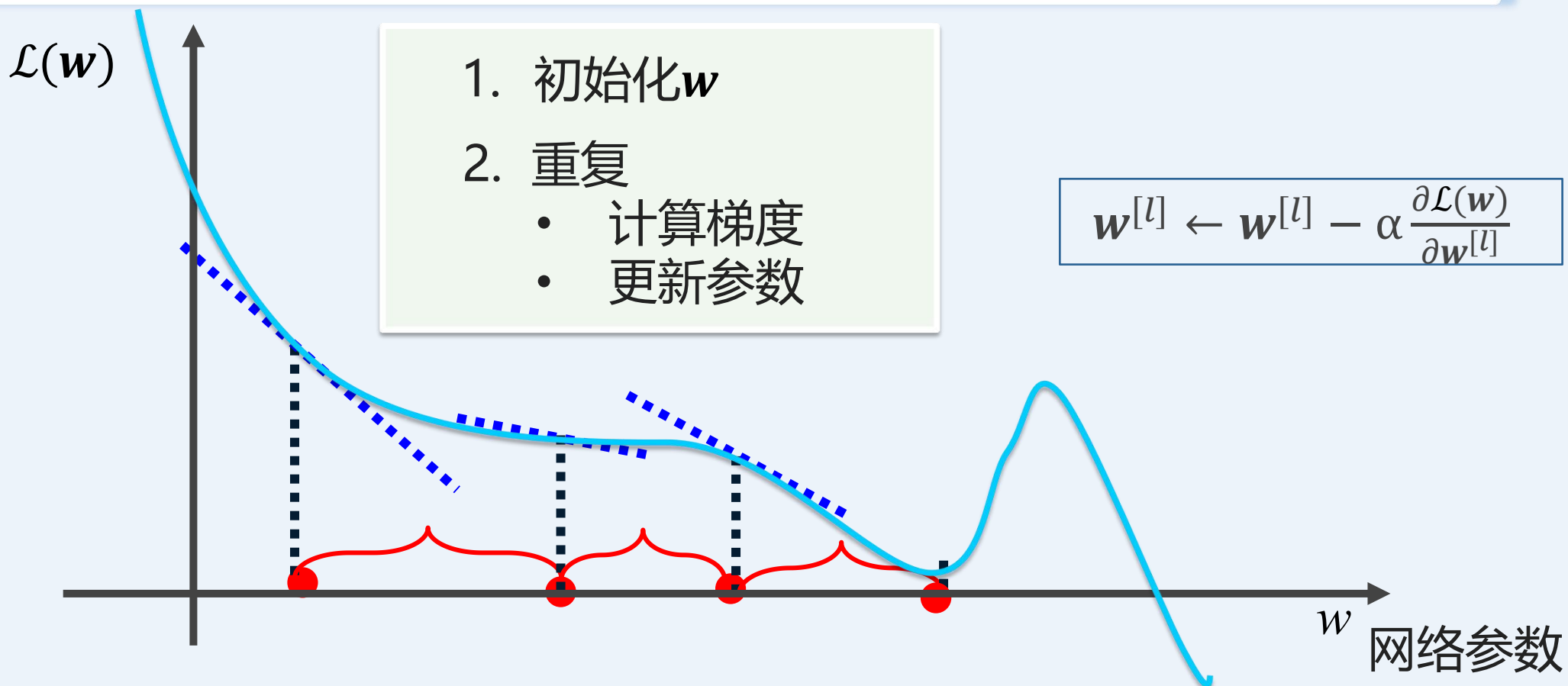
梯度: $\frac{\partial \mathcal{L}(\mathbf{w})}{\partial w} = \lim_{\Delta w \rightarrow 0} \frac{\mathcal{L}(\mathbf{w} + \Delta \mathbf{w}) - \mathcal{L}(\mathbf{w})}{\Delta w}$



路线：找到梯度下降的方向，求取 $\mathcal{L}(\mathbf{w})$ 的最小值

多层感知机如何训练：梯度下降

梯度下降法：沿着梯度下降的方向更新参数，使得 $\mathcal{L}(\mathbf{w})$ 取值最小



多层感知机如何训练：梯度下降

- 反向传播算法：**根据网络的特点而设计的高效梯度下降方法**，用于参数的学习
- 复习：前向传播中**神经网络内部的计算

$$\begin{aligned} g(\mathbf{W}^{(1)} \times \mathbf{x}^{(1)}) &= \mathbf{h}^{(1)} & g(\mathbf{W}^{(2)} \times \mathbf{h}^{(1)}) &= \mathbf{h}^{(2)} \\ g(\mathbf{W}^{(3)} \times \mathbf{h}^{(2)}) &= y' & \mathcal{L} &= \frac{1}{2} (y - y')^2 \end{aligned} \quad (\text{粗体字表示向量，上标是层号})$$

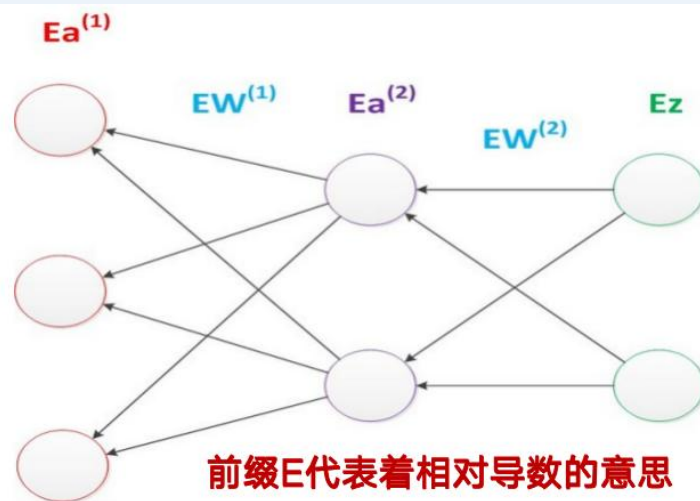
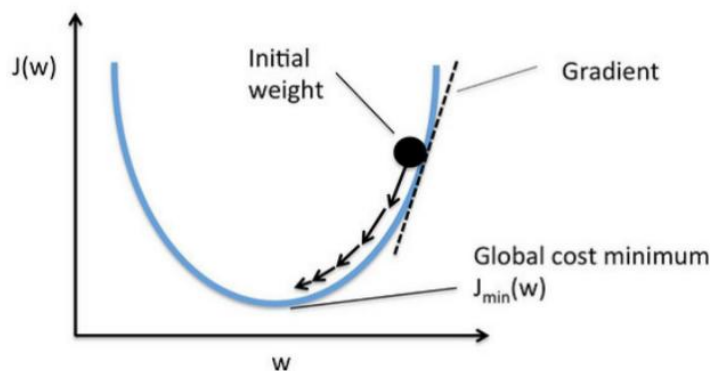
- 这是关于 $\mathbf{W}^{(1)}$ 的**复合函数**，神经网络中应用**链式求导法则**进行梯度计算

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(1)}} = \frac{\partial \mathcal{L}}{\partial y'} \times \frac{\partial y'}{\partial \mathbf{h}^{(2)}} \times \frac{\partial \mathbf{h}^{(2)}}{\partial \mathbf{h}^{(1)}} \times \frac{\partial \mathbf{h}^{(1)}}{\partial \mathbf{W}^{(1)}}$$

多层感知机-训练

梯度下降 算法：每次计算参数在当前的梯度，然后让参数向着梯度的反方向前进一段距离，不断重复，直到梯度接近零时截止。一般这个时候，所有的参数恰好达到使损失函数达到一个最低值的状态

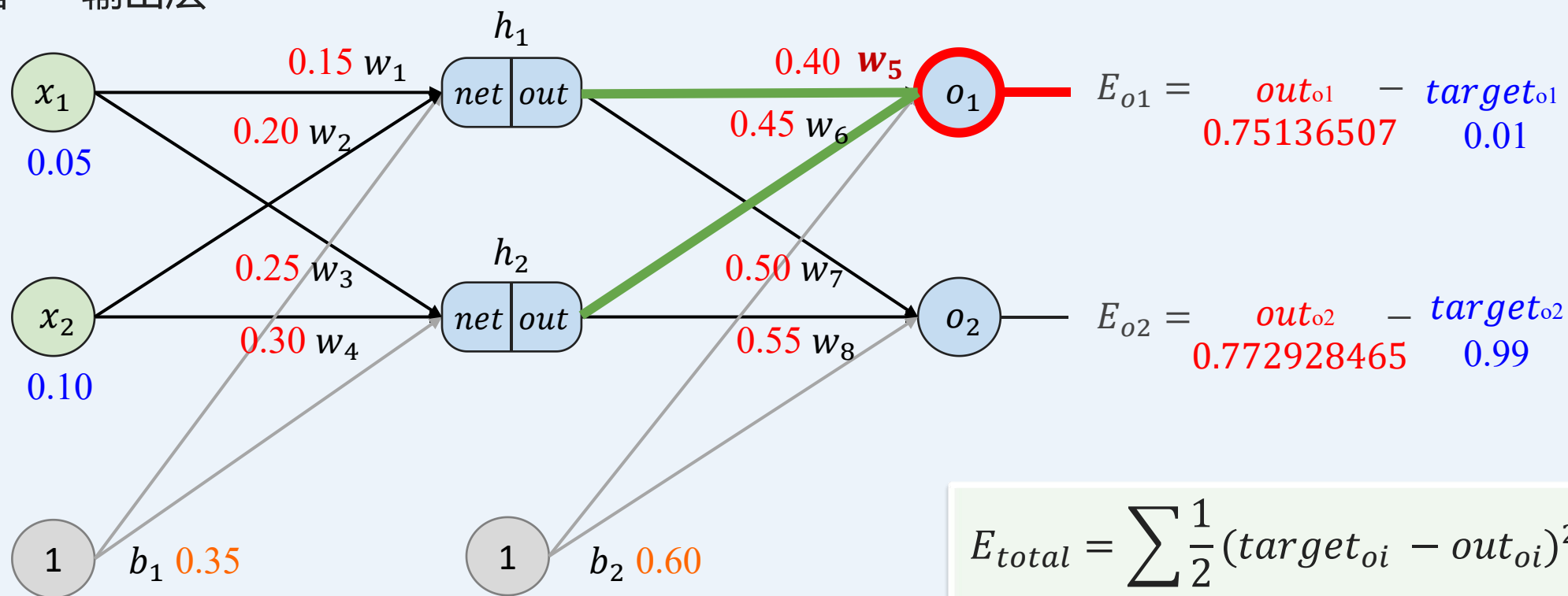
反向传播 算法：反向传播算法不一次计算所有参数的梯度，而是从后往前。首先计算输出层的梯度，然后是第二个参数矩阵的梯度，接着是中间层的梯度，再然后是第一个参数矩阵的梯度，最后是输入层的梯度。计算结束以后，所要的两个参数矩阵的梯度就都有了



多层感知机如何训练：前向传播+反向传播

📋 示例：一个具有两个输入、两个隐藏神经元和两个输出神经元的神经网络。

- 反向传播---输出层

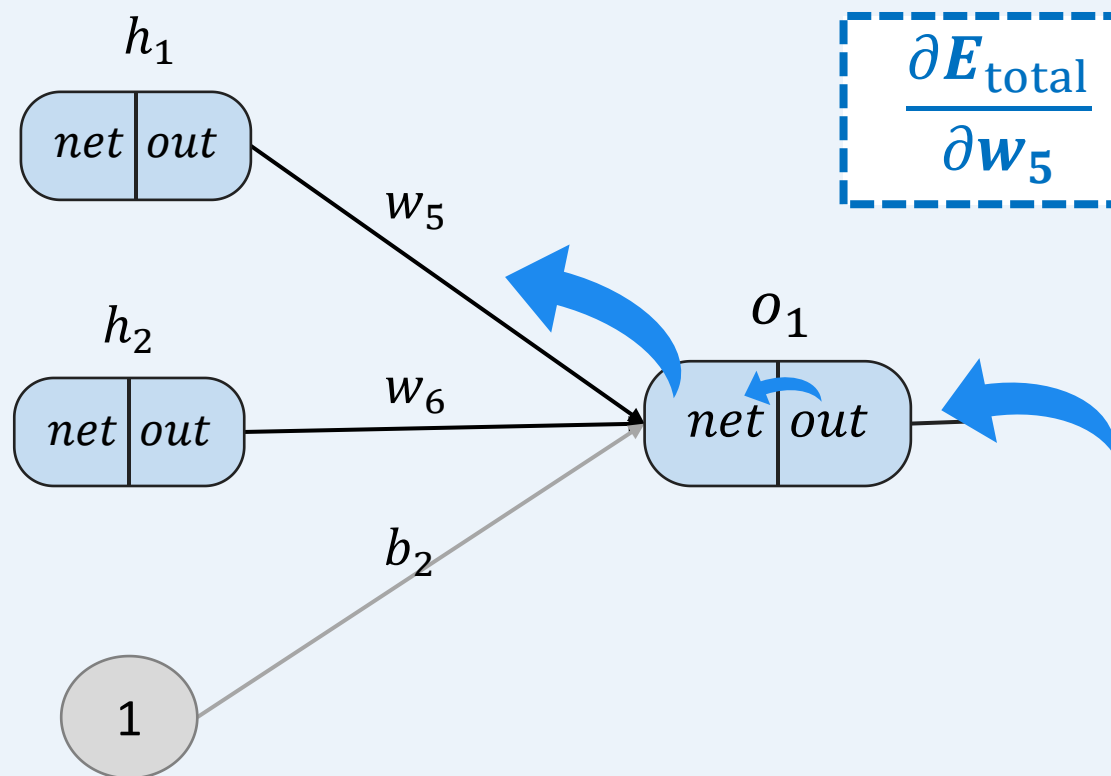


$$E_{total} = \sum \frac{1}{2} (target_{oi} - out_{oi})^2 = E_{o1} + E_{o2}$$

多层感知机如何训练：前向传播+反向传播

 示例：一个具有两个输入、两个隐藏神经元和两个输出神经元的神经网络。

- 反向传播----考虑 w_5 ，探究 w_5 的变化对总误差的影响有多大？



$$\frac{\partial E_{\text{total}}}{\partial w_5} = \frac{\partial E_{\text{total}}}{\partial \text{out}_{o1}} * \frac{\partial \text{out}_{o1}}{\partial \text{net}_{o1}} * \frac{\partial \text{net}_{o1}}{\partial w_5}$$

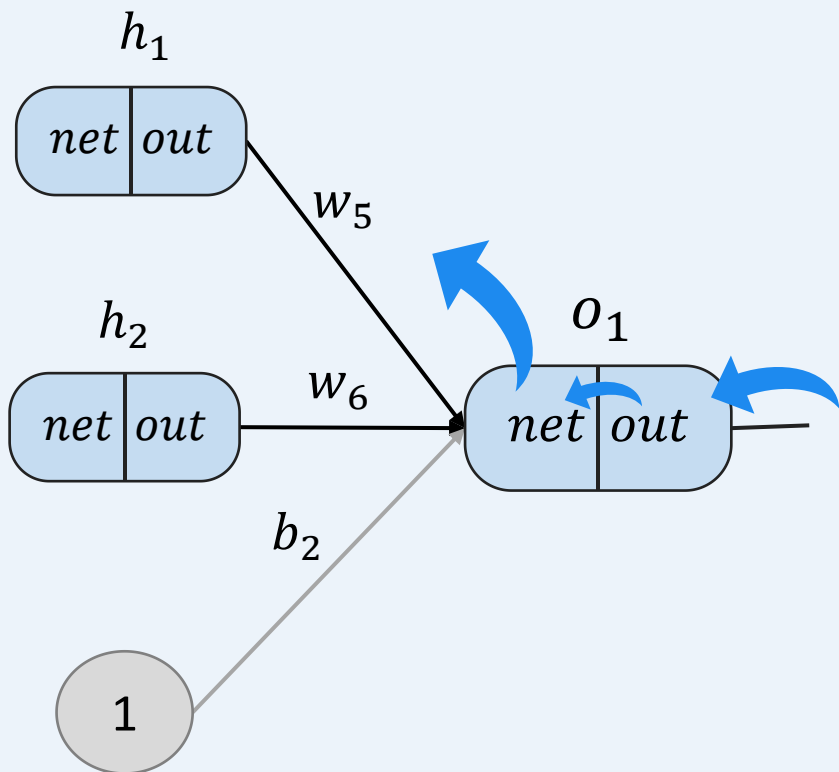
链式法则

$$E_{\text{total}} = \sum \frac{1}{2} (\text{target}_{oi} - \text{out}_{oi})^2$$
$$E_{\text{total}} = E_{o1} + E_{o2}$$

多层感知机如何训练：前向传播+反向传播

 示例：一个具有两个输入、两个隐藏神经元和两个输出神经元的神经网络。

- 反向传播----考虑 w_5 ，探究 w_5 的变化对总误差的影响有多大？



$$\frac{\partial E_{\text{total}}}{\partial w_5} = \boxed{\frac{\partial E_{\text{total}}}{\partial out_{o1}}} * \frac{\partial out_{o1}}{\partial net_{o1}} * \frac{\partial net_{o1}}{\partial w_5}$$

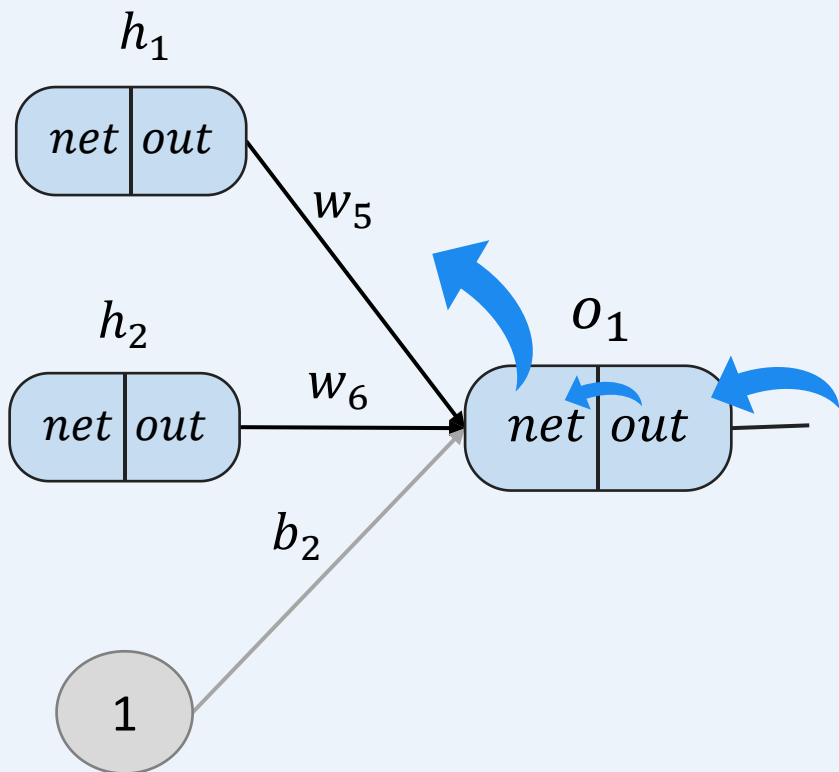
$$E_{\text{total}} = \sum \frac{1}{2} (target_i - out_{oi})^2$$

$$\begin{aligned} \frac{\partial E_{\text{total}}}{\partial out_{o1}} &= 2 * \frac{1}{2} (target_{o1} - out_{o1})^{2-1} * (-1) + 0 \\ &= -(target_{o1} - out_{o1}) \\ &= -(0.01 - 0.75136507) = 0.74136507 \end{aligned}$$

多层感知机如何训练：前向传播+反向传播

 示例：一个具有两个输入、两个隐藏神经元和两个输出神经元的神经网络。

- 反向传播----考虑 w_5 ，探究 w_5 的变化对总误差的影响有多大？



$$\frac{\partial E_{\text{total}}}{\partial w_5} = \frac{\partial E_{\text{total}}}{\partial \text{out}_{o1}} * \left[\frac{\partial \text{out}_{o1}}{\partial \text{net}_{o1}} \right] * \frac{\partial \text{net}_{o1}}{\partial w_5}$$

Sigmoid函数：

$$\text{out}_{o1} = \sigma(x) = \frac{1}{1 + e^{-x}} \quad \downarrow \quad \frac{d}{dx} \sigma(x) = \sigma(x)(1 - \sigma(x))$$

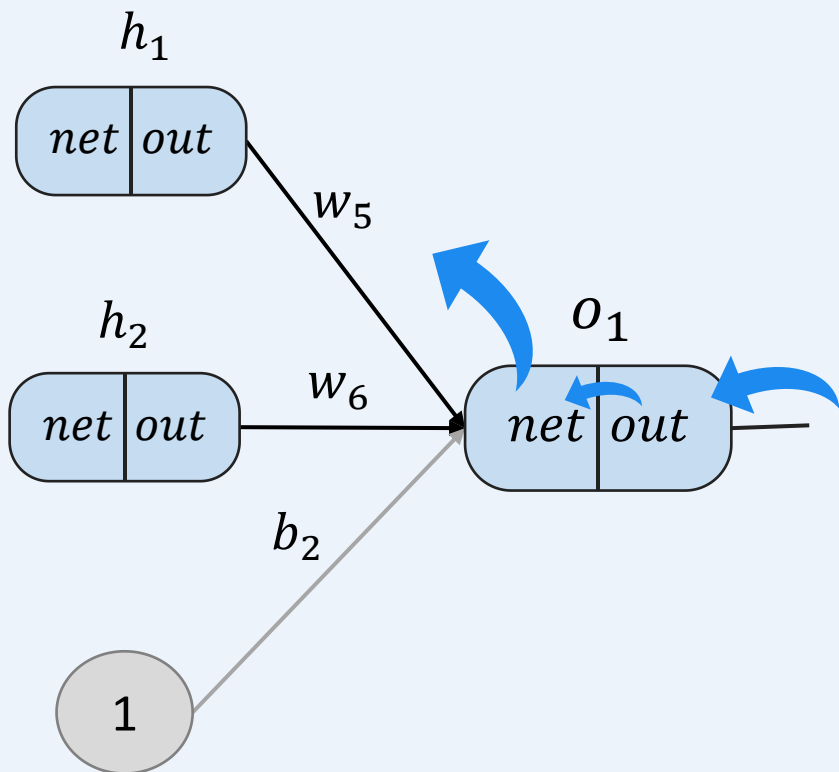
$$\text{out}_{o1} = \frac{1}{1 + e^{-\text{net}_{o1}}}$$

$$\frac{\partial \text{out}_{o1}}{\partial \text{net}_{o1}} = \text{out}_{o1}(1 - \text{out}_{o1}) = 0.186815602$$

多层感知机如何训练：前向传播+反向传播

 示例：一个具有两个输入、两个隐藏神经元和两个输出神经元的神经网络。

- 反向传播----考虑 w_5 ，探究 w_5 的变化对总误差的影响有多大？



$$\frac{\partial E_{\text{total}}}{\partial w_5} = \frac{\partial E_{\text{total}}}{\partial \text{out}_{o1}} * \frac{\partial \text{out}_{o1}}{\partial \text{net}_{o1}} * \boxed{\frac{\partial \text{net}_{o1}}{\partial w_5}}$$



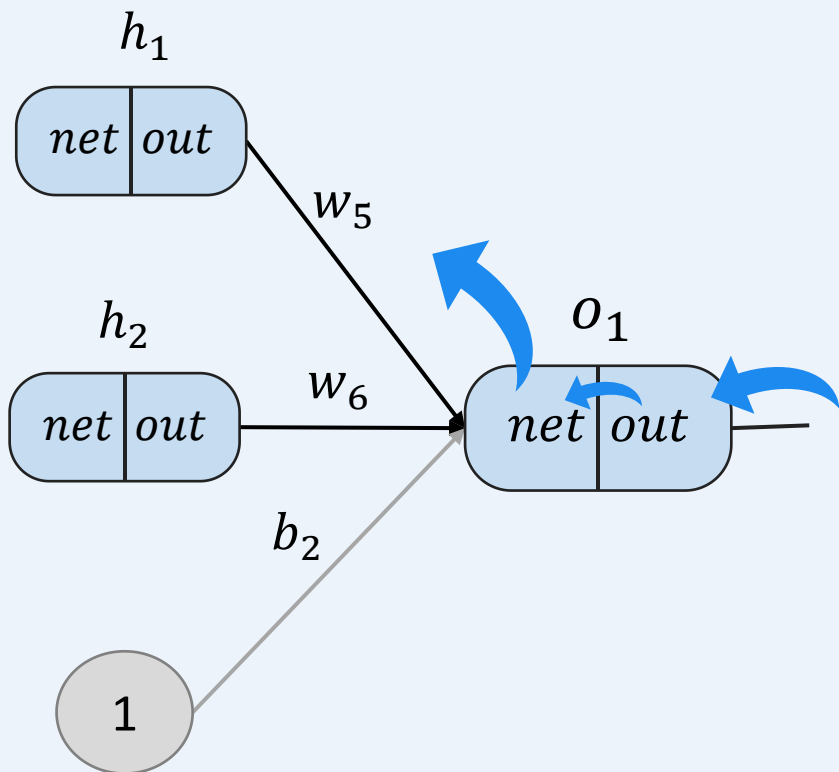
$$\text{net}_{o1} = w_5 * h_1 + w_6 * h_2 + b_2 * 1$$

$$\begin{aligned} \frac{\partial \text{net}_{o1}}{\partial w_5} &= 1 * h_1 * w_5^{(1-1)} + 0 + 0 \\ &= h_1 = 0.593269992 \end{aligned}$$

多层感知机如何训练：前向传播+反向传播

 示例：一个具有两个输入、两个隐藏神经元和两个输出神经元的神经网络。

- 反向传播----考虑 w_5 ，探究 w_5 的变化对总误差的影响有多大？



$$\begin{aligned}
 \frac{\partial E_{\text{total}}}{\partial w_5} &= \frac{\partial E_{\text{total}}}{\partial \text{out}_{o1}} * \frac{\partial \text{out}_{o1}}{\partial \text{net}_{o1}} * \frac{\partial \text{net}_{o1}}{\partial w_5} \\
 &= -(\text{target}_{o1} - \text{out}_{o1}) * \text{out}_{o1}(1 - \text{out}_{o1}) * h_1 \\
 &= 0.74136507 * 0.186815602 * 0.593269992 \\
 &= 0.082167041
 \end{aligned}$$

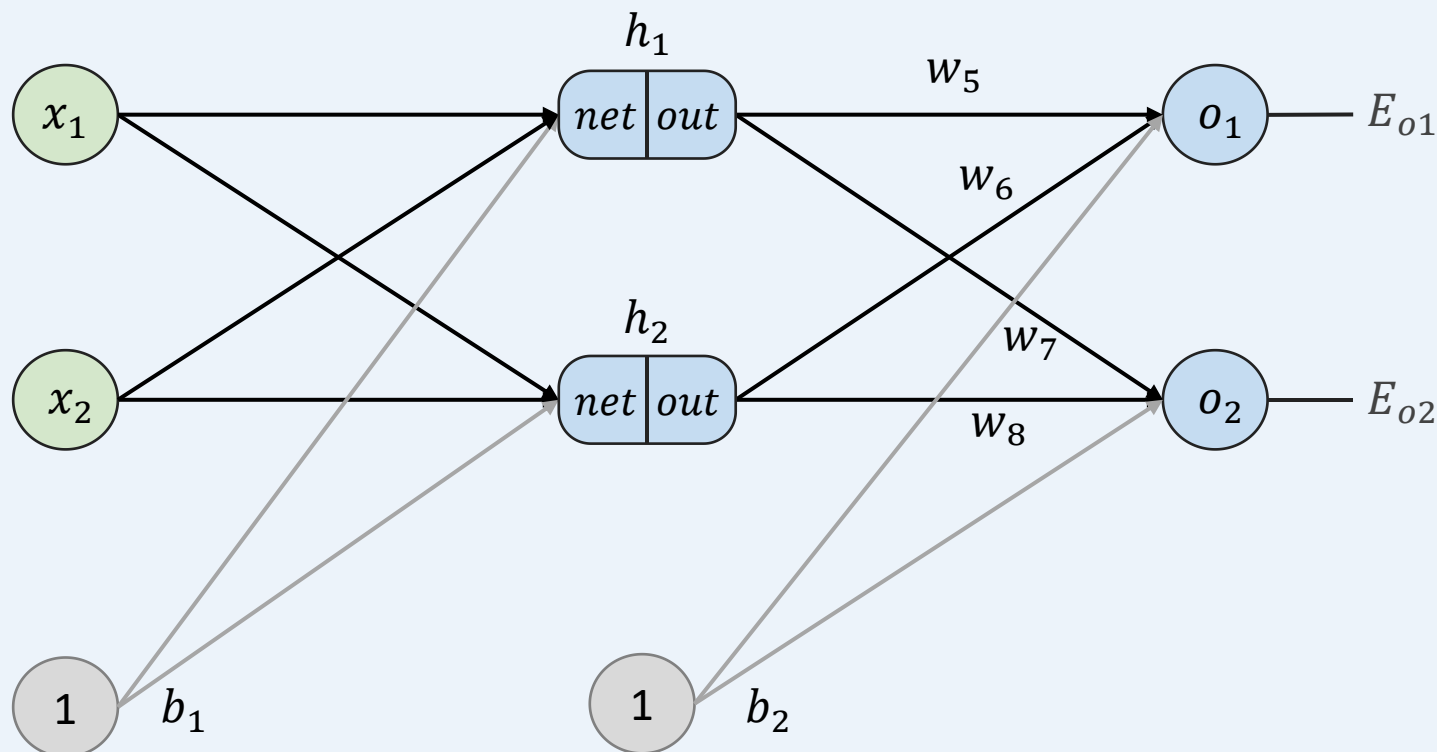
为了减少误差，设置学习率 $\eta = 0.5$

$$\begin{aligned}
 w_5^+ &= w_5 - \eta * \frac{\partial E_{\text{total}}}{\partial w_5} \\
 &= 0.4 - 0.5 * 0.082167041 = 0.35891648
 \end{aligned}$$

多层感知机如何训练：前向传播+反向传播

 示例：一个具有两个输入、两个隐藏神经元和两个输出神经元的神经网络。

- 反向传播----重复操作计算 w_6 、 w_7 、 w_8



$$w_6^+ = 0.408666186$$

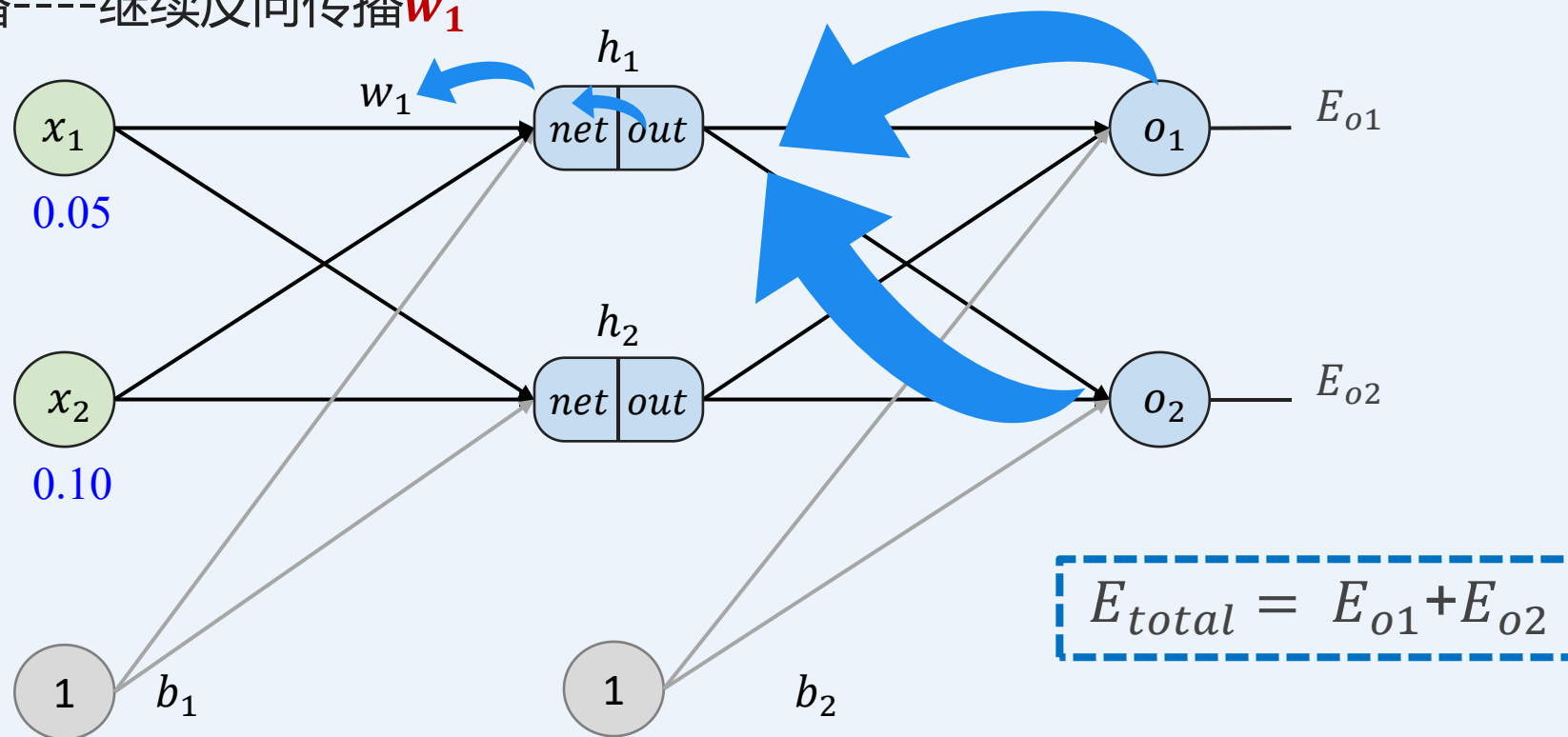
$$w_7^+ = 0.511301270$$

$$w_8^+ = 0.561370121$$

多层感知机如何训练：前向传播+反向传播

📋 示例：一个具有两个输入、两个隐藏神经元和两个输出神经元的神经网络。

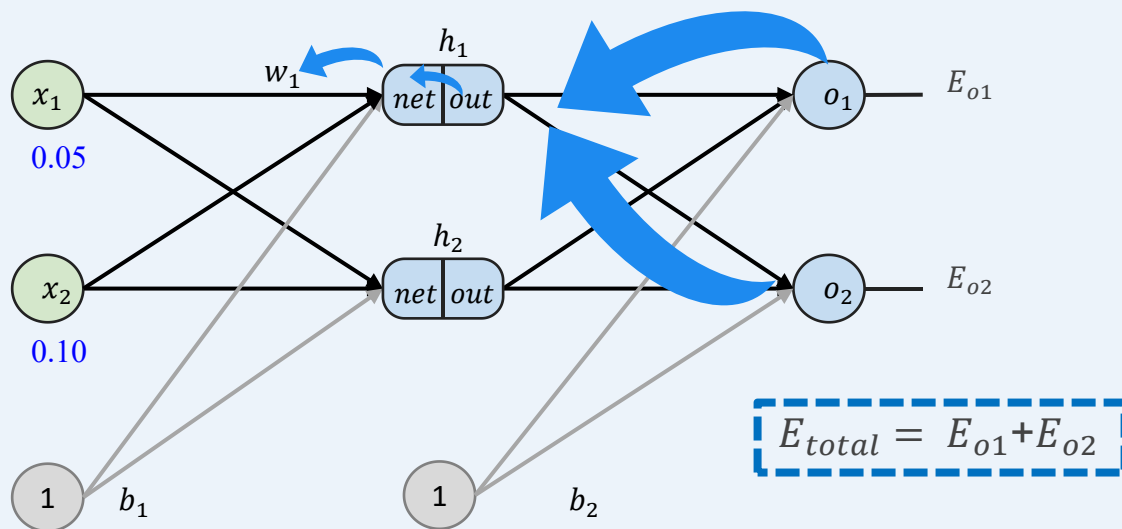
- 反向传播----继续反向传播 w_1



多层感知机如何训练：前向传播+反向传播

 示例：一个具有两个输入、两个隐藏神经元和两个输出神经元的神经网络。

- 反向传播----继续反向传播 w_1



$$\frac{\partial E_{total}}{\partial w_1} = \boxed{\frac{\partial E_{total}}{\partial out_{h1}}} * \frac{\partial out_{h1}}{\partial net_{h1}} * \frac{\partial net_{h1}}{\partial w_1}$$

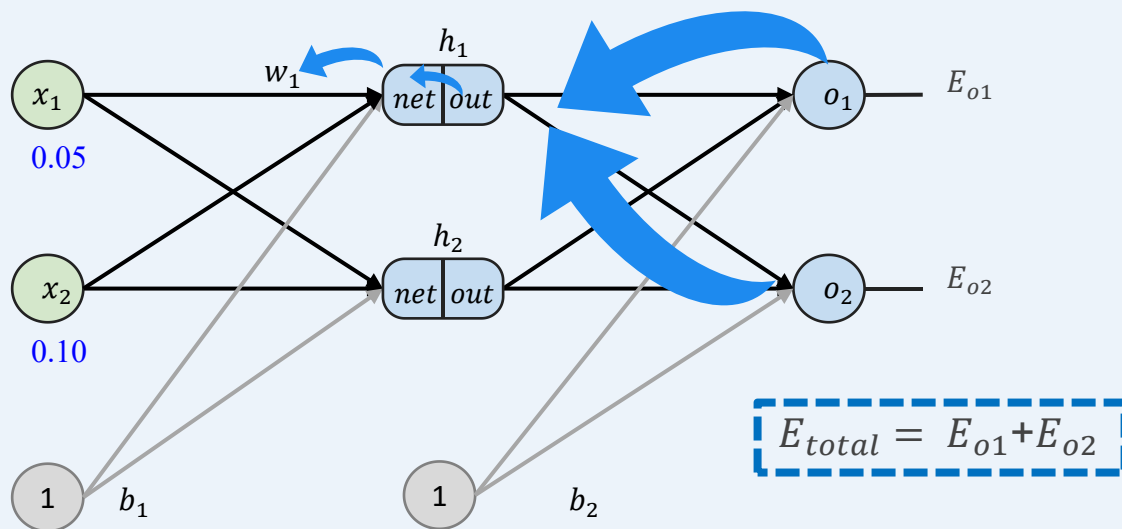
$$\frac{\partial E_{total}}{\partial out_{h1}} = \boxed{\frac{\partial E_{o1}}{\partial out_{h1}}} + \frac{\partial E_{o2}}{\partial out_{h1}}$$

$$\frac{\partial E_{o1}}{\partial out_{h1}} = \frac{\partial E_{o1}}{\partial out_{o1}} * \frac{\partial out_{o1}}{\partial net_{o1}} * \frac{\partial net_{o1}}{\partial out_{h1}}$$

多层感知机如何训练：前向传播+反向传播

 示例：一个具有两个输入、两个隐藏神经元和两个输出神经元的神经网络。

- 反向传播----像以前一样**计算并更新** w_1



$$w_1^+ = w_1 - \eta * \frac{\partial E_{total}}{\partial w_1} = 0.149780716$$

重复操作**计算** w_2 、 w_3 、 w_4

$$w_2^+ = 0.19956143$$

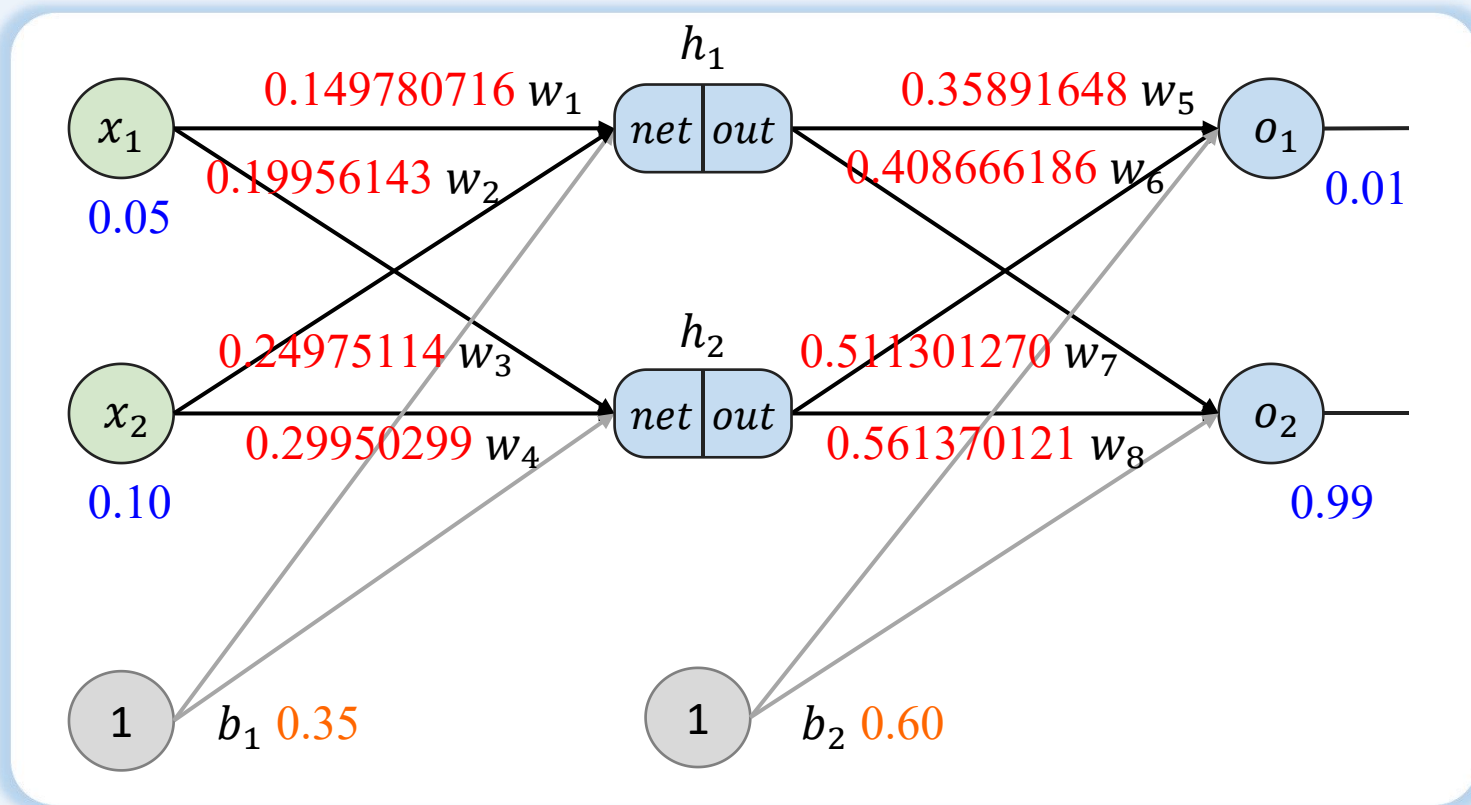
$$w_3^+ = 0.24985114$$

$$w_4^+ = 0.29950229$$

多层感知机如何训练：前向传播+反向传播

📋 示例：一个具有两个输入、两个隐藏神经元和两个输出神经元的神经网络。

- 反向传播----更新参数并重复训练



反向传播总结

1.

第一轮反向传播后，总误差：

0.298371109 → 0.291027924

2.

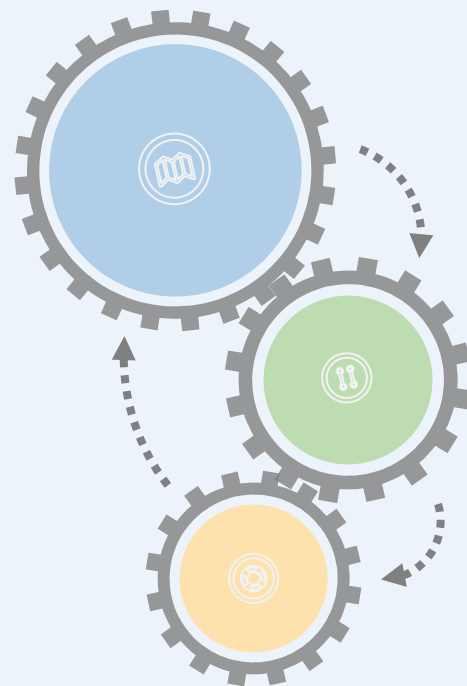
重复此过程10000次后，误差骤降至 **0.000035085**

3.

当我们输入0.05和0.1时，两个输出神经元产生

0.015912196 (相对于0.01目标)

0.984065734 (相对于0.99目标)



神经网络带来的启示



- 反向传播算法的启示是数学中的链式法则
- 从BP算法开始，研究者们更多地从数学上寻求问题的最优解。不再盲目模拟人脑网络（神经网络研究走向成熟的标志）。正如科学家们可以从鸟类的飞行中得到启发，但没有必要一定要完全模拟鸟类的飞行方式，也能制造可以飞天的飞机

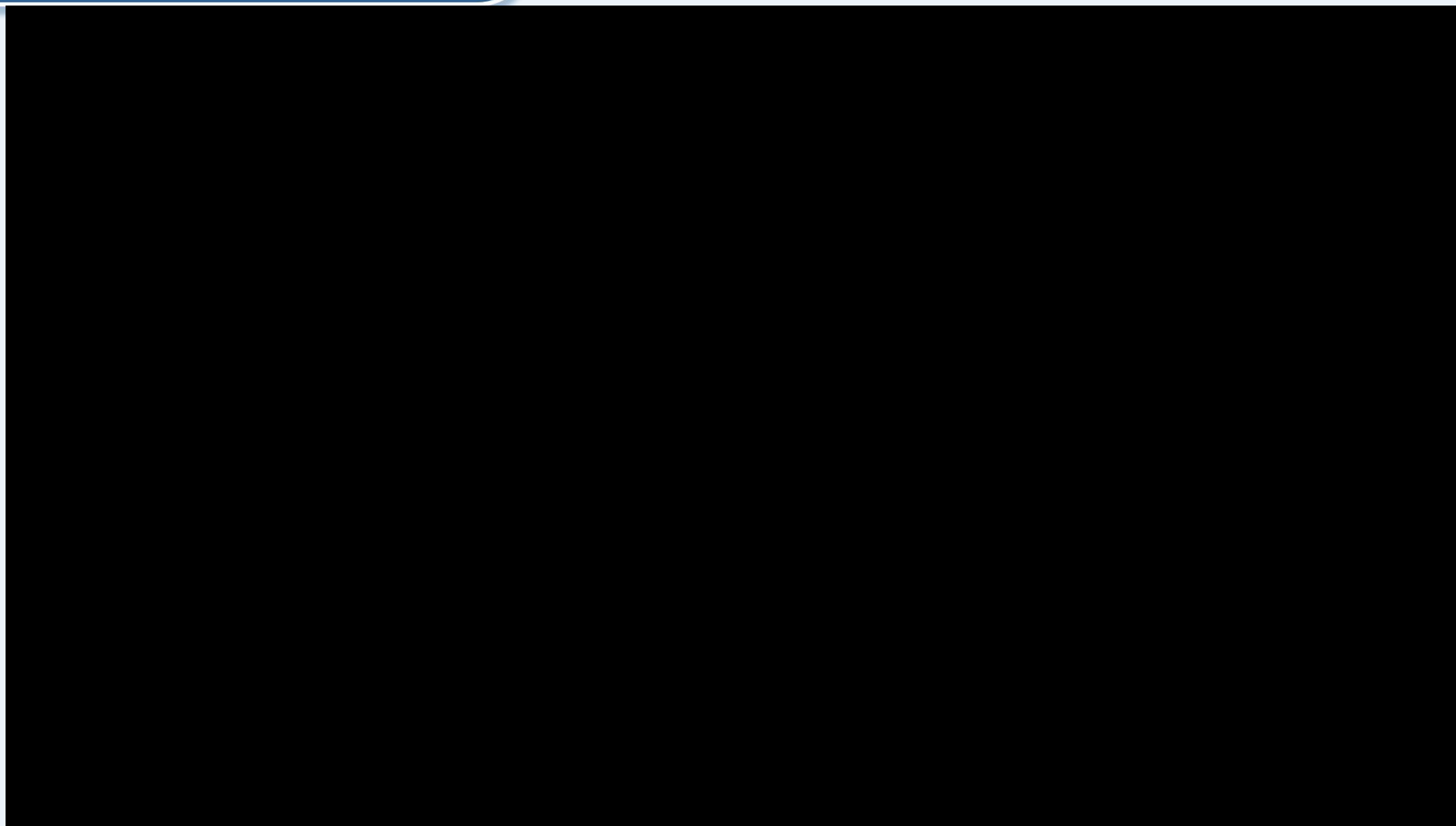


神经网络带来的影响

- 神经网络的学者们再次登上了《纽约时报》的专访。人们认为神经网络可以解决许多问题。就连娱乐界都开始受到了影响，《终结者》电影中的阿诺都赶时髦地说：“我的CPU是一个神经网络处理器，一个会学习的计算机”。



神经网络应用

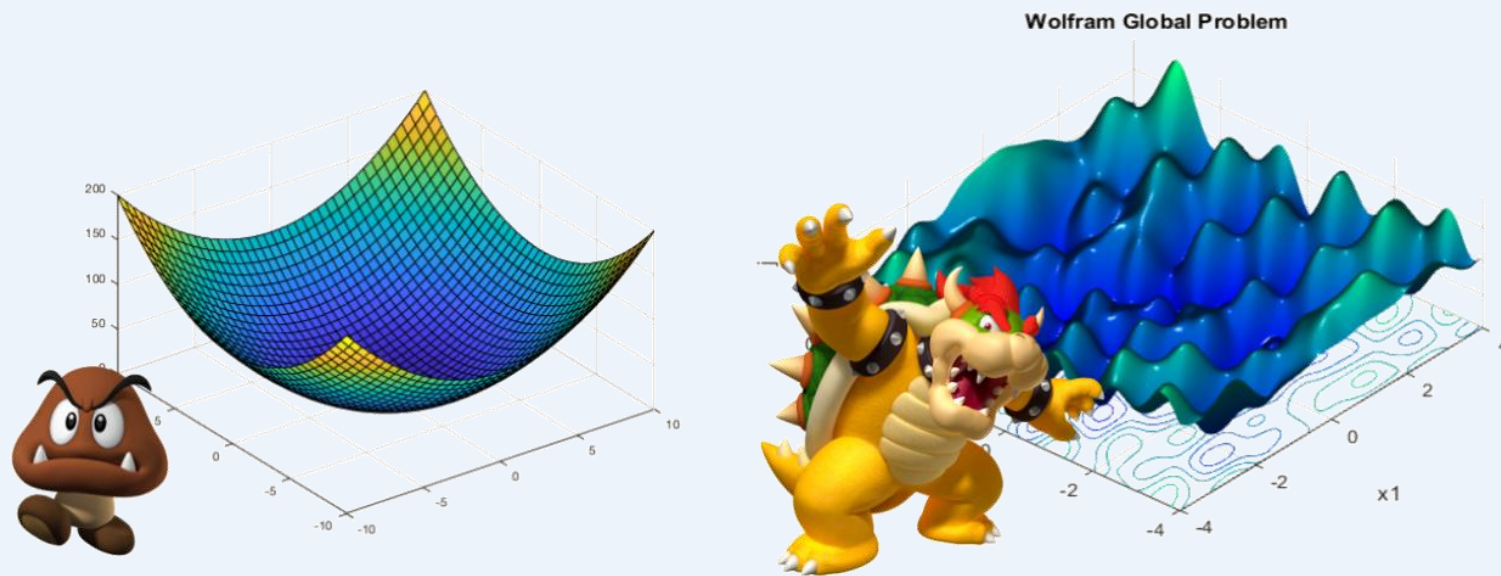


多层神经网络的问题

存在问题：尽管使用了BP算法，一次多层神经网络的**训练仍然耗时太久**，而且困扰训练优化的一个问题就是**局部最优解**问题，这使得神经网络的优化较为困难。同时，隐藏层的**节点数需要调参**，这使得使用不太方便，工程和研究人員对此多有抱怨

非凸模型优化问题

- 问题 a: 非凸模型优化的一个问题就是**局部最优解**问题, 这使得神经网络的优化较为困难。(Hmm... 真的非常难...)



- 非凸模型: $g(\dots g(W (g(W^{(1)} * a^{(1)}) + g(W^{(2)} * a^{(2)}))) \dots)$
- 优化效率慢**限制了感知机规模**, **节点数也需要精确调配**。

多层感知机激活函数的影响

问题：为什么会产生这样的问题？

问题：现实应用中，人们发现多层感知机不同的层的学习速度差异很大。

- 使用sigmoid作为神经元的输入输出函数，在神经网络反向传播梯度时，每传递一层梯度衰减为原来的0.25。层数一多，梯度指数衰减后低层基本上接受不到有效的训练信号。

2层

$$\frac{\partial E_{\text{total}}}{\partial w_1} = \frac{\partial E_{\text{total}}}{\partial \text{out}_{h1}} * \frac{\partial \text{out}_{h1}}{\partial \text{net}_{h1}} * \frac{\partial \text{net}_{h1}}{\partial w_1}$$

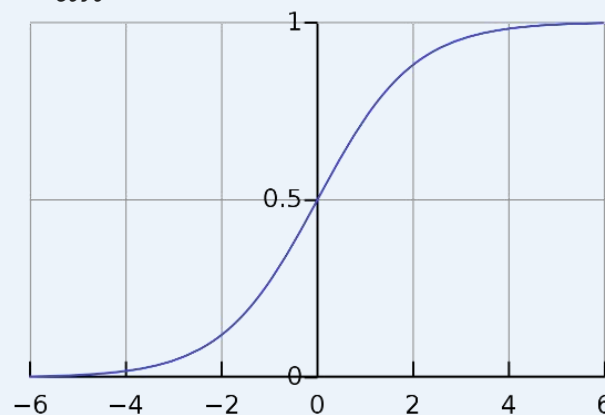
$$\frac{\partial E_{\text{total}}}{\partial \text{out}_{h1}} = \frac{\partial E_{o1}}{\partial \text{out}_{h1}} + \frac{\partial E_{o2}}{\partial \text{out}_{h1}}$$

1层

$$\frac{\partial E_{o1}}{\partial \text{out}_{h1}} = \frac{\partial E_{o1}}{\partial \text{out}_{o1}} * \frac{\partial \text{out}_{o1}}{\partial \text{net}_{o1}} * \frac{\partial \text{net}_{o1}}{\partial \text{out}_{h1}}$$

Sigmoid函数: $\sigma(x) = \frac{1}{1+e^{-x}}$

$$0 \leq \frac{d}{dx} \sigma(x) = \sigma(x)(1 - \sigma(x)) \leq 0.25$$



逐层递减直到**梯度消失**，使得多层网络等价于浅层网络的学习

非凸模型优化问题

- 90年代中期，由Vapnik等人发明的SVM（Support Vector Machines，支持向量机）算法诞生，很快就在若干个方面体现出了对比神经网络的优势：**无需调参；高效；全局最优解**。基于以上种种理由，SVM迅速打败了神经网络算法成为主流。



Vladimir Vapnik



- 神经网络的研究再次陷入了冰河期。当时，只要你的论文中包含神经网络相关的字眼，非常容易被会议和期刊拒收，研究界那时对神经网络的不待见可想而知。

深度学习-引入

- | 在被人摒弃的10年中，有几个学者仍然在坚持研究，包括加拿大多伦多大学的Geoffery Hinton教授
- | 2006年，Hinton在《Science》和相关期刊上发表了论文，首次提出了“深度信念网络”的概念
- | 与传统的训练方式不同，“深度信念网络”有一个“预训练”（pre-training）的过程，这可以方便的让神经网络中的权值找到一个接近最优解的值，之后再使用“微调”（fine-tuning）技术来对整个网络进行优化训练。这两个技术的运用大幅度减少了训练多层神经网络的时间。他给多层神经网络相关的学习方法赋予了一个新名词--“深度学习”（深度学习是多个算法的总称）

深度学习-引入

参数越多的模型复杂度越高

- 优点：“容量” (capacity) 越大，能完成更复杂的任务
- 缺点：训练效率低，数据量少时易陷入过拟合

缺点解决方案

- 云计算，大数据时代下，计算能力大幅提升，可缓解训练低效性
- 训练数据大幅增加，降低过拟合风险

以深度学习为代表的复杂模型开始受人关注

深度学习-引入

- | 很快，深度学习在语音识别领域暂露头角。接着，2012年，Hinton 与他的学生在 ImageNet 竞赛中，用多层的卷积神经网络成功地对包含一千类别的一百万张图片进行了训练，取得了分类错误率 15 % 的好成绩，这个成绩比第二名高了近 11 个百分点，充分证明了多层神经网络识别效果的优越性
- | 在这之后，关于深度神经网络的研究与应用不断涌现。



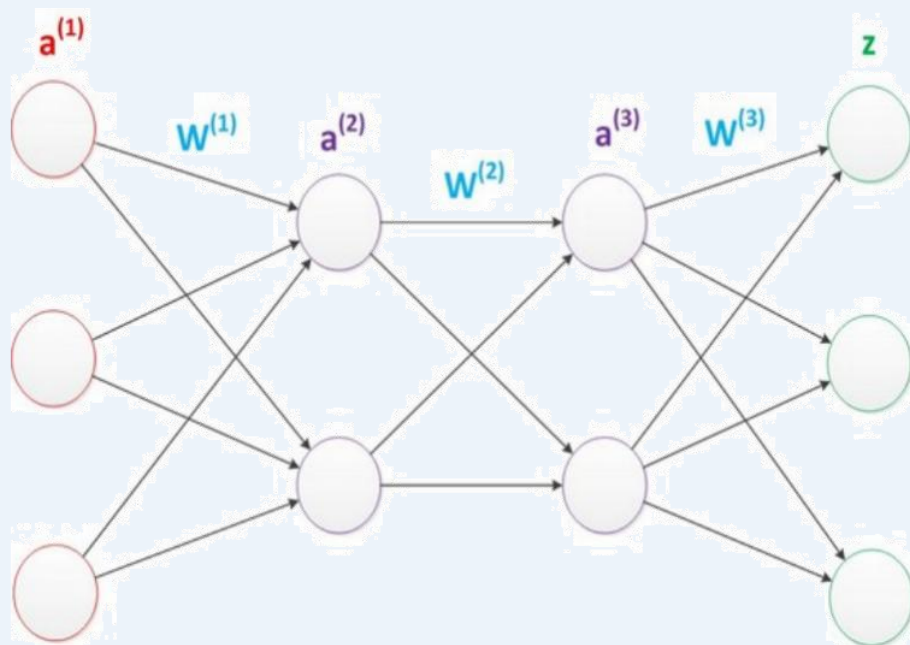
Geoffery Hinton
1986, BP算法



Geoffery Hinton

深度学习网络框架

- | 延续两层神经网络的方式来设计一个多层神经网络
- | 在两层神经网络的输出层后面，继续添加层次。原来的输出层变成中间层，新加的层次成为新的输出层。所以可以得到下图。



多层神经网络

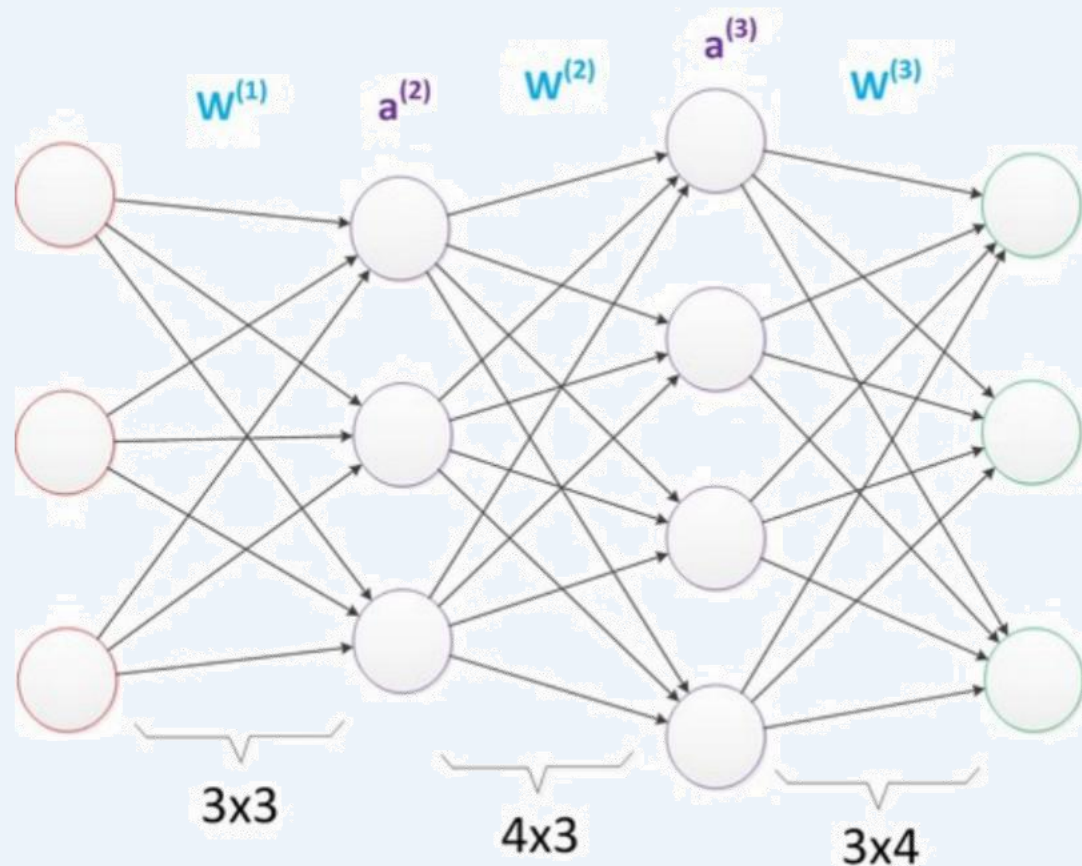
$$g(W^{(1)} * a^{(1)}) = a^{(2)};$$

$$g(W^{(2)} * a^{(2)}) = a^{(3)};$$

$$g(W^{(3)} * a^{(3)}) = z;$$

深度学习网络框架

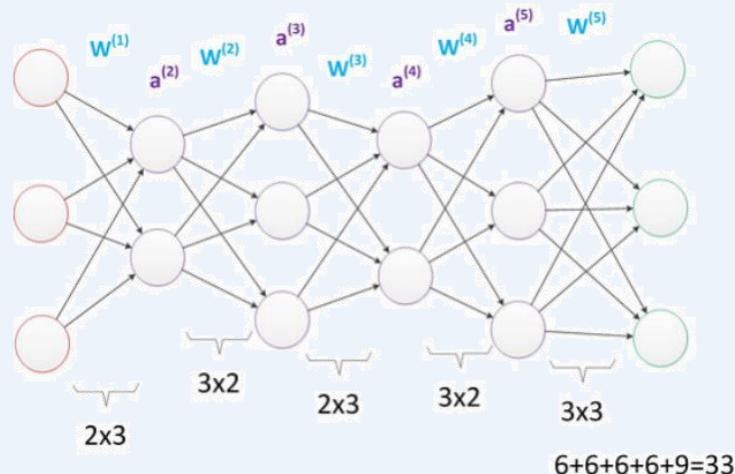
网络的参数为33个。



$$9+12+12=33$$

深度学习网络框架

- 在参数一致的情况下，我们也可以获得一个“更深”的网络

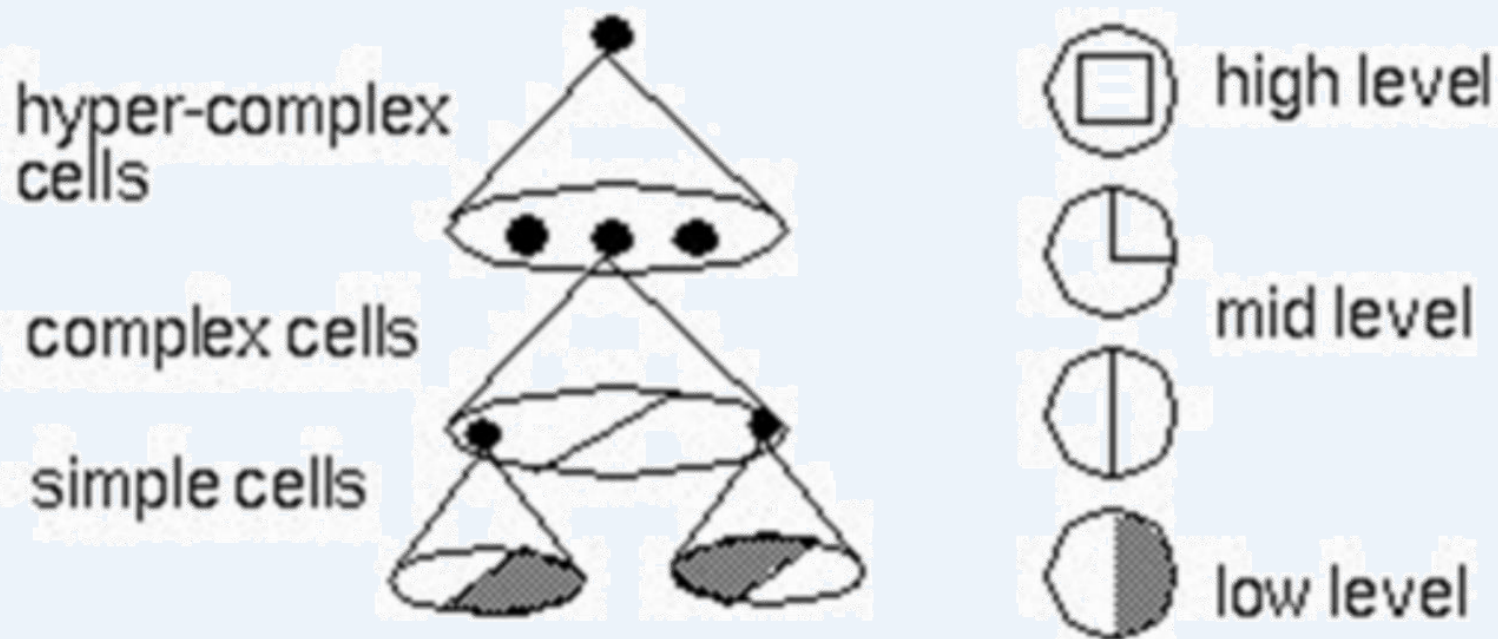


- 参数数量仍然是33，但却有4个中间层，更多的层次带来更深入的特征表示能力，以及更强的函数模拟能力
- 随着层数增加，每一层对于前一层次的抽象表示更深入。每一层神经元学习到的是前一层神经元值的更抽象的表示。例如第一个隐藏层学习到的是“边缘”的特征，第二个隐藏层学习到由“边缘”组成的“形状”，第三个隐藏层学习到的是由“形状”组成的“图案”的特征，最后的隐藏层学习到的是由“图案”组成的“目标”的特征
- 通过抽取更抽象的特征来对事物进行区分，从而获得更好的区分与分类能力

深度学习网络框架

逐层特征学习例子：

featural hierarchy



多层神经网络（特征学习）

深度学习优越效果

- | 更强的函数模拟能力是由于随着层数的增加，整个网络的参数就越多。而神经网络其实本质就是模拟特征与目标之间的真实关系函数的方法，更多的参数意味着其模拟的函数可以更加的复杂，可以有更多的 容量（capacity）去拟合真正的关系（强悍的非线性函数拟合）
- | 在参数数量一样的情况下，更深的网络往往具有比浅层的网络更好的识别效率
 - 2012年起，每年获得ImageNet冠军的深度神经网络的层数逐年增加
 - 2015年最好的方法GoogleNet是一个多达22层的神经网络
 - 2016拿到最好成绩的MSRA团队的方法使用的更是一个152层的网络！

多层训练解决手段

2006年Hinton提出的逐层预训练方法

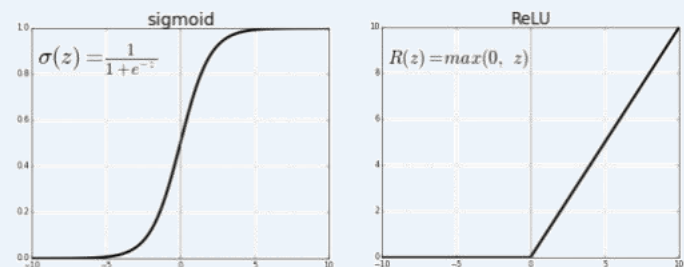
❖ pre-training + fine-tuning...

使用Rectified Linear Unit(ReLu)激活函数

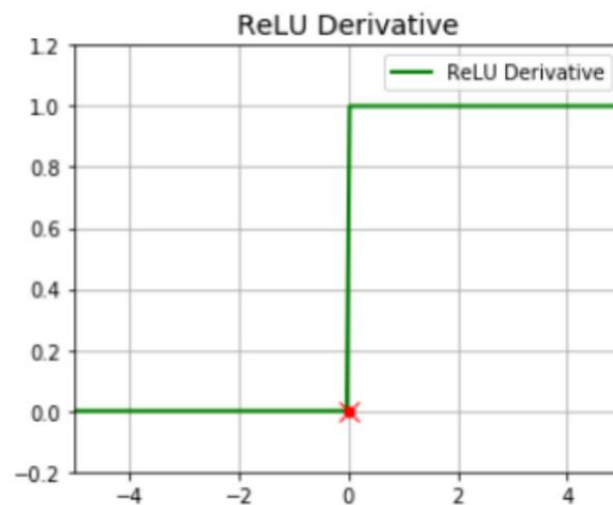
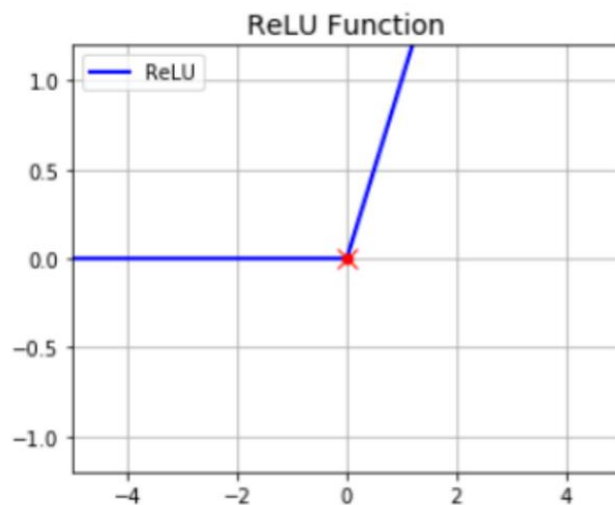
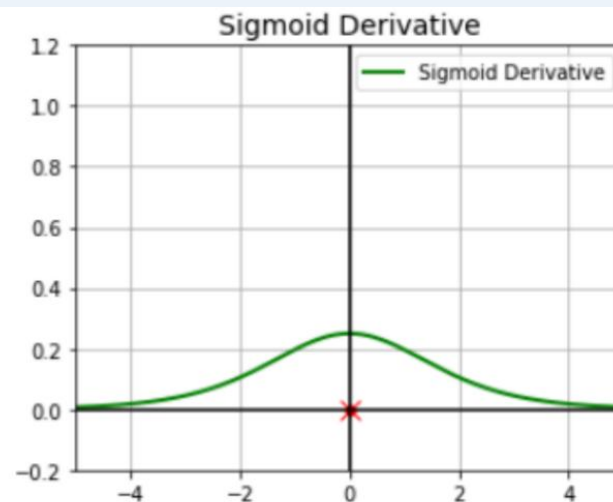
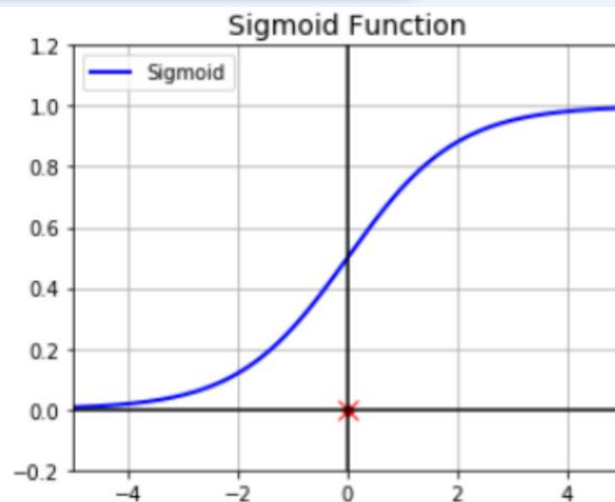
- ❖ sigmoid等函数，算激活函数时（指数运算），计算量大，反向传播求误差梯度时，求导涉及除法，计算量相对大，而采用Relu激活函数，整个过程的计算量节省很多
- ❖ sigmoid函数反向传播时，容易出现梯度消失
- ❖ Relu会使一部分神经元的输出为0，这样就造成了网络的稀疏性，并且减少了参数的相互依存关系，缓解了过拟合问题的发生

Dropout

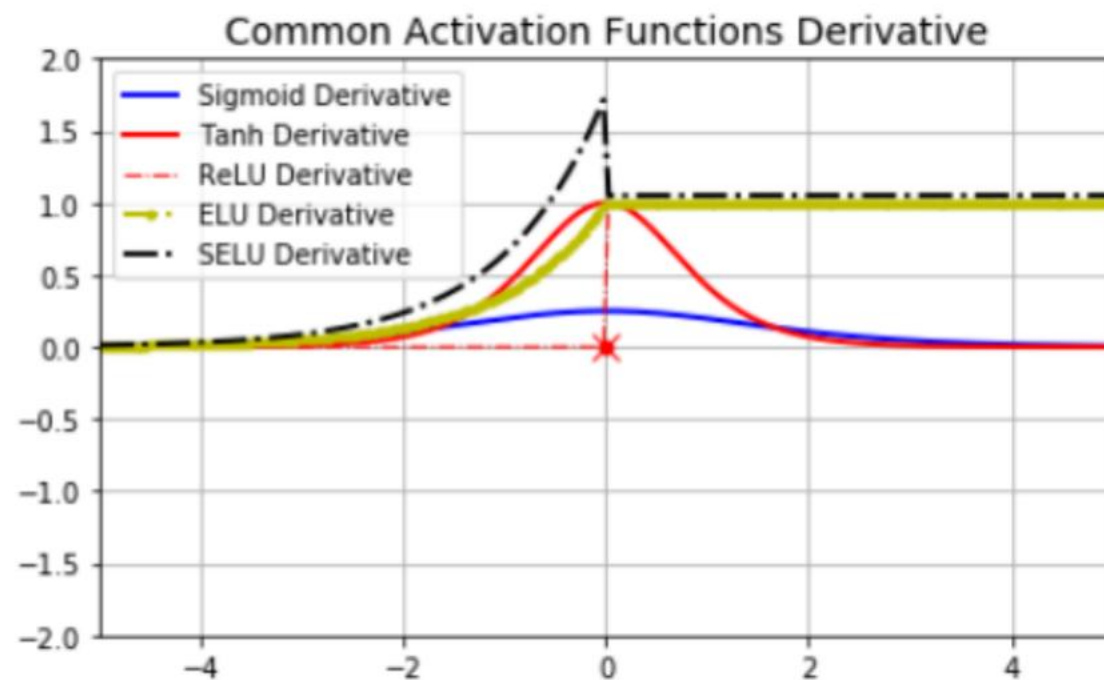
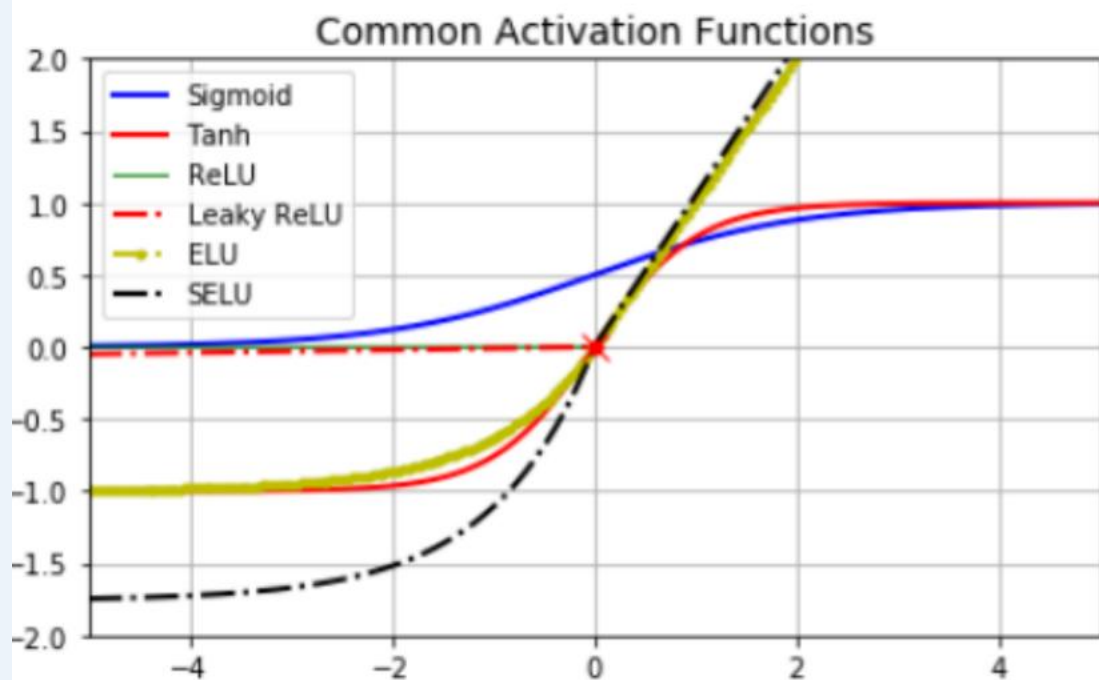
- ❖ 模型训练时随机让网络某些隐含层节点的权重不工作，防止过拟合



不同激活函数求导

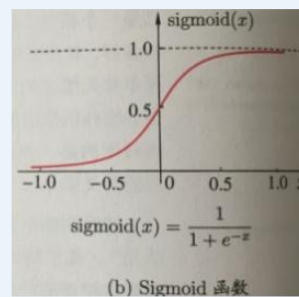
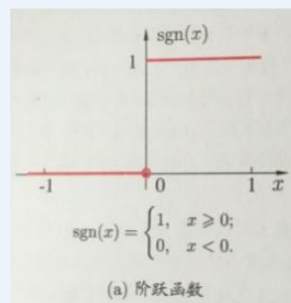


不同激活函数求导

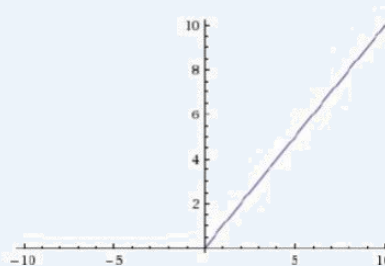


训练细节

- | 单层神经网络：sgn函数
- | 两层神经网络：sigmoid函数



- | 多层神经网络：ReLU函数， $y = \max(x, 0)$ ，在 x 大于0，输出就是输入，而在 x 小于0时，输出就保持为0。函数设计启发来自于生物神经元对于激励的线性响应，以及当低于某个阈值后就不再响应的模拟



- | 神经网络的层数增加了，参数也增加了，表示能力大幅度增强，很容易出现过拟合现象。因此正则化技术就显得十分重要。Dropout技术是目前使用的最多的正则化技术

不同激活函数求导

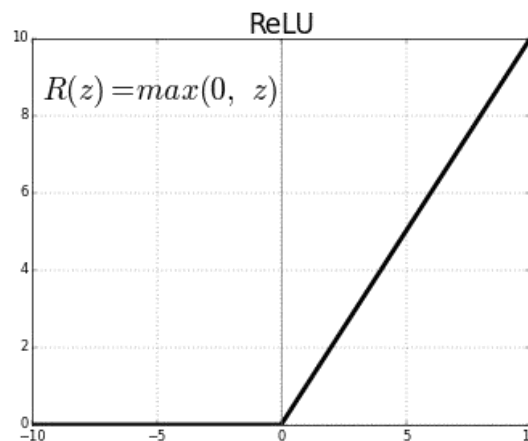
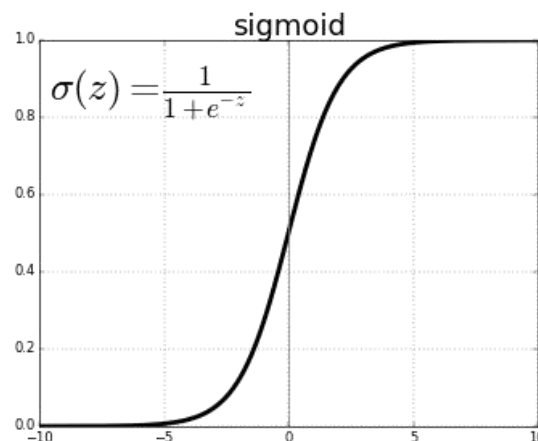


解决：使用Rectified Linear Unit(Relu)激活函数

Sigmoid函数及求导：

$$\sigma(x) = \frac{1}{1+e^{-x}}$$

$$0 \leq \frac{d}{dx} \sigma(x) \leq 0.25$$



ReLU函数及求导：

$$\sigma(x) = \max(0, x)$$

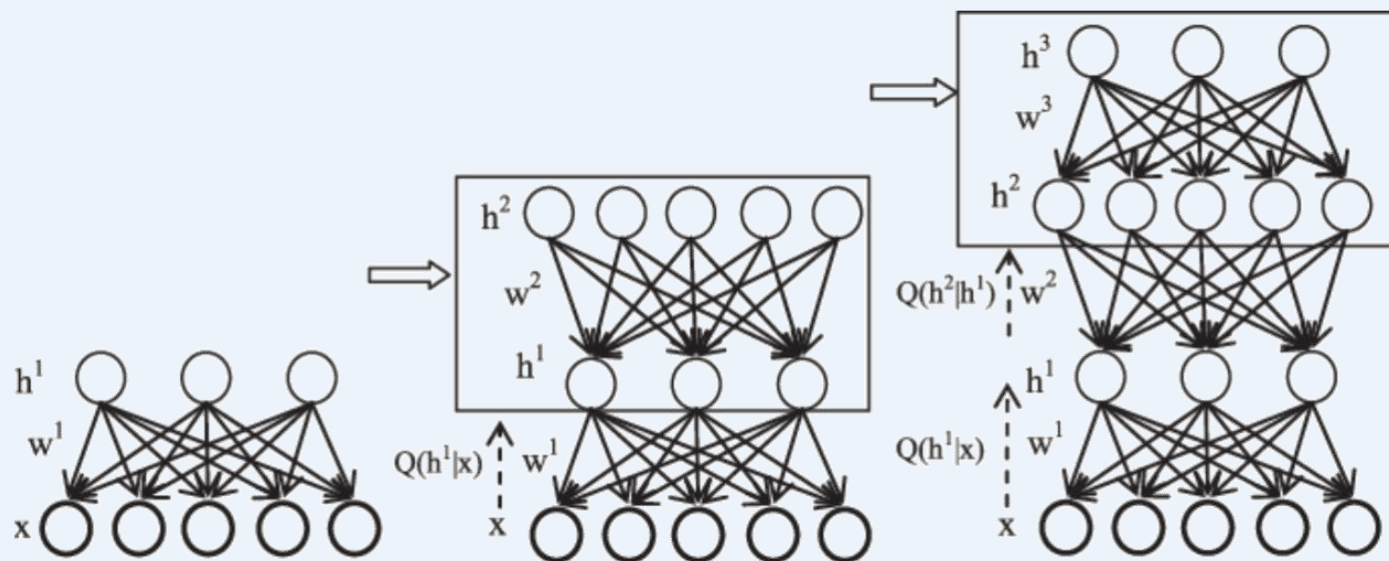
$$\frac{d}{dx} \sigma(x) = \begin{cases} 0 & x \leq 0 \\ 1 & x > 0 \end{cases}$$

- Sigmoid函数反向传播时，容易出现梯度消失， Relu函数**避免梯度消失**
- Sigmoid等函数计算量大，反向传播时计算量相对大，采用Relu激活函数的**计算量节省**很多
- Relu会使部分神经元的输出为0，造成了网络的稀疏性，减少了参数的相互依存关系，**缓解过拟合问题**的发生

无监督逐层训练

多隐层网络训练的方法：无监督逐层训练 (unsupervised layer-wise training)

- 预训练：每次训练一层隐结点，训练时将上一层隐结点的输出作为输入，而本层隐结点的输出作为下一层隐结点的输入。
- 在预训练全部完成后，对整个网络进行“微调” (fine-tuning) 训练。



DBN网络

- | 举例 - - 深度信念网络 (Deep belief network, DBN):
 - 每层都是一个受限Boltzmann机，即整个网络可视为若干个受限玻尔兹曼机 (RBM) 堆叠而成
 - 首先训练第一层（是关于训练样本的RBM模型，可按标准的RBM训练）
 - 将第一层预训练好的隐结点视为第二层的输入结点；对第二层进行预训练；... ..
 - 各层预训练完成后，利用BP算法对整个网络进行训练

新范式提出

“预训练” + “微调”

- 可视为将大量参数分组，对每组先找到局部看来比较好的设置
- 再基于这些局部较优的结果联合起来进行全局寻优
- 这样利用了模型大量参数所提供的自由度的同时，有效地节省了训练开销

可视化过程



深度学习的影响

- 目前，深度神经网络在人工智能界占据统治地位。神经网络界当下的四位引领者除了Ng, Hinton以外，还有CNN的发明人Yann Lecun，以及《Deep Learning》的作者Bengio
- 马斯克的OpenAI项目，邀请Bengio作为高级顾问。马斯克认为，人工智能技术不应该掌握在大公司如Google, Facebook的手里，更应该作为一种开放技术，让所有人都可以参与研究。



Yann LeCun



Yoshua Bengio

神经网络中的不同类别:



前馈神经网络：有向图是没有回路的

反馈神经网络：有向图是有回路的，Hopfield网络是反馈神经网络

RNN也属于一种反馈神经网络



神经网络的应用：疾病早期检测

中大眼科中心团队通过捕捉儿童眼球运动数据, 研发**全球首个婴幼儿视功能损伤手机智能筛查系统**, 为破解婴幼儿视功能损伤及相关眼病早期筛诊难题提供有效技术手段, 研究成果发表于《Nature Medicine》



通过婴幼儿观看动画片的注视习惯和行为模式的实时捕捉, **实现16种婴幼儿常见眼病的早期检测, 筛查准确率超85%, 降低儿保, 社区筛查等多场景中婴幼儿眼病筛诊的难度**

nature medicine

[Explore content](#) [About the journal](#) [Publish with us](#)

[nature](#) > [nature medicine](#) > [articles](#) > [article](#)

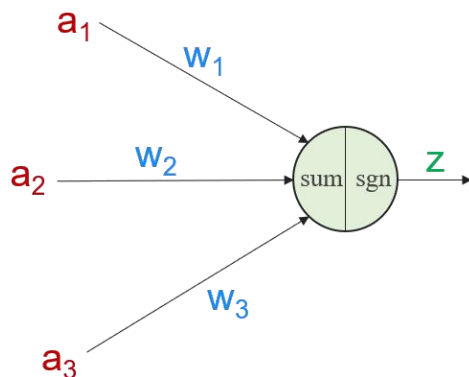
Article | [Published: 26 January 2023](#)

Early detection of visual impairment in young children using a smartphone-based deep learning system

数据采集

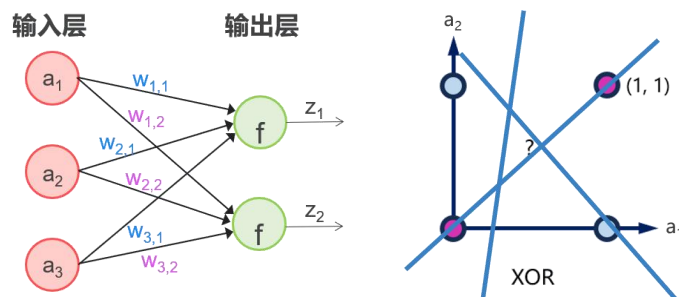
本章小结

人工神经元结构



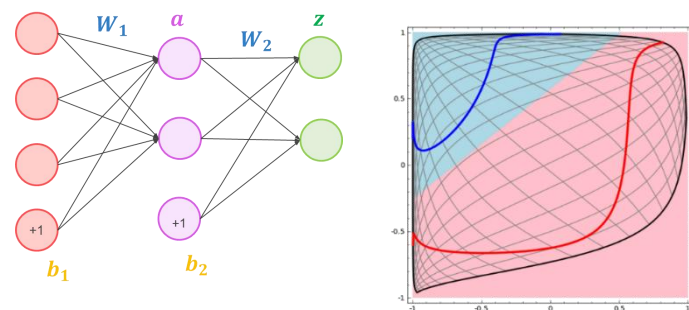
- 建立了人工神经元的结构
 - **N输入**
 - **1输出**
 - **2计算**

感知机



- 学习感知机的概念，并分析出它的局限性
 - **数学化表达**
 - **线性分类边界**

多层感知机



- 多层神经网络的结构及数学表达式
 - **通用近似定理**解释其拟合原理
 - **非线性问题：隐藏层与非线性激活函数**

课堂练习

考虑以下神经网络，其中 node1 和 node2 为输入节点，node3 为输出节点，且输入节点均没有应用激活函数。输出节点 node3 的输入 $I_3 = w_{13} * x_1 + w_{23} * x_2 + \theta$ ，输出节点采用 sigmoid 激活函数，即 $O_3 = \frac{1}{1+e^{-I_3}}$ ，假定一个训练样本， $x_1 = 1, x_2 = 0.5$ ，其真实的类标签 $y = 1$ ，设损失函数采用均方误差，即 $L = 0.5 * (y - O_3)^2$ ，用以更新网络参数。当前网络的参数初始值为： $\theta = 0, w_{13} = 0.5, w_{23} = -1$ 。请基于上述训练样本的 x_1, x_2, y 的取值，以及网络中 θ, w_{13}, w_{23} 的初始值，计算损失函数 L 对 w_{13} 的偏导，即 $\frac{\partial L}{\partial w_{13}}$ 的值

($\sqrt{e} = 1.65$)。

解答：

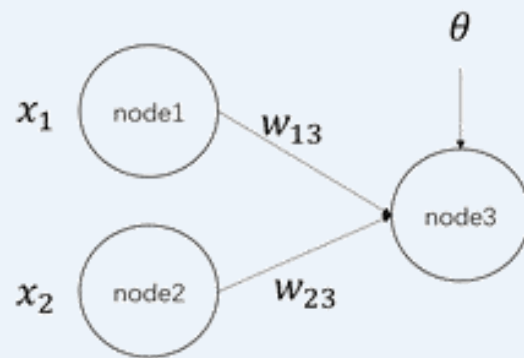
正向传播：

$$I_3 = w_{13} * x_1 + w_{23} * x_2 + \theta = 0$$

$$O_3 = \frac{1}{1+e^{-I_3}} = \frac{1}{2}$$

链式法则：

$$\frac{\partial L}{\partial w_{13}} = \frac{\partial L}{\partial O_3} * \frac{\partial O_3}{\partial I_3} * \frac{\partial I_3}{\partial w_{13}} = (O_3 - y) * \frac{e^{-I_3}}{(1+e^{-I_3})^2} * x_1 = -\frac{1}{2} * \frac{1}{4} * 1 = -\frac{1}{8}$$



谢谢大家！

相关课程资源及参考文献请浏览

超算习堂：<https://easyhpc.net/course/143>